

Cloud Computing (BCS071/BCS713)

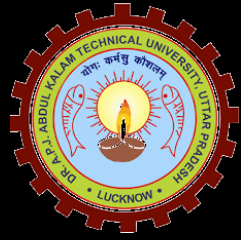
Unit -1 Introduction to Cloud Computing

Unit	Topic
I	Introduction To Cloud Computing: Definition of Cloud – Evolution of Cloud Computing – Underlying Principles of Parallel and Distributed Computing – Cloud Characteristics – Elasticity in Cloud – On- demand Provisioning.

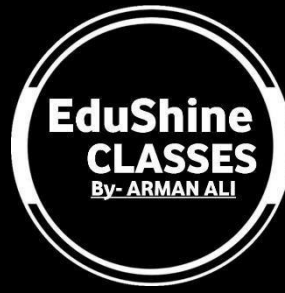
 EduShine Classes – Arman Ali



Download Notes : <https://rzp.io/rzp/IHQaLO0N>



Cloud Computing (BCS071/BCS713)



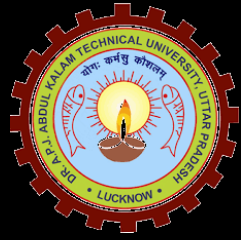
⚡ What is Cloud Computing?

- Cloud computing means **using computer services (like storage, servers, databases, software, and networking) over the Internet** instead of keeping them on your personal computer.
- Cloud computing is like renting computing power, storage, and software over the internet instead of owning and managing your own physical hardware and software

So in simple words:

Cloud = Internet-based computer services.

You just need a good Internet connection, and you can use powerful services from anywhere.



Cloud Computing (BCS071/BCS713)



⚡ Need of Cloud Computing (Why we use it?)

1. Cost Saving 💰

No need to buy expensive hardware/software.
Just pay for what you use (like electricity bill).

2. Scalability 📈

Easily increase or decrease resources.

Example: Amazon adds more servers automatically during Diwali sales.

3. Accessibility 🌐

You can access your files/apps from anywhere in the world, just need the Internet.

Example: Google Drive lets you open files from laptop, phone, or office PC.

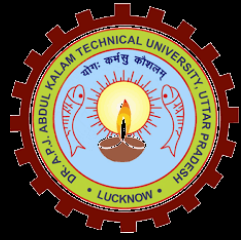
4. Data Backup and Recovery 🔒

Even if your laptop crashes, your files in cloud (like Google Photos or Drive) remain safe.

5. Collaboration 👥

Multiple people can work on the same project at the same time.

Example: In Google Sheets, many friends can edit together.



Cloud Computing (BCS071/BCS713)



6. Automatic Updates ↻

Cloud services get updated automatically (no need to install new versions manually).

✍ Needs of Cloud Computing (Where do we need it?)

1. For Businesses 📁

Startups don't need to buy costly servers, they rent from cloud providers like AWS, Azure, or GCP.

2. For Students/Education 📖

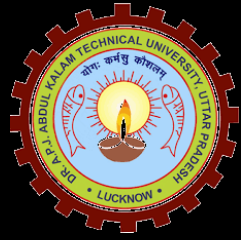
Online classes, assignments, storage of notes, YouTube tutorials all use cloud.

3. For Developers 👤💻

Can test and run apps without buying big servers.

4. For Daily Life 📱

Using Gmail, Google Drive, WhatsApp backup, Netflix – all are examples of cloud computing.



Cloud Computing (BCS071/BCS713)



☁️ Characteristics of Cloud Computing(V.V.IMP)

Cloud computing has some special qualities that make it different from traditional computing. Let's go one by one:

1. On-Demand Self Service

☞ **Means:** You can use cloud services whenever you want, without asking anyone.

💡 **Example:** Just like ordering food from Zomato/Swiggy anytime you want – you don't need to call the restaurant owner; you just click and get it.

2. Broad Network Access

☞ **Means:** Cloud services can be accessed through the internet from anywhere and from any device (computer, laptop, mobile, tablet).

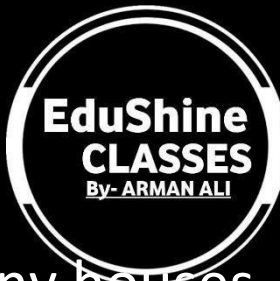
💡 **Example:** You can open **Google Drive** on your phone, laptop, or tablet – doesn't matter where you are.

3. Resource Pooling

☞ **Means:** The cloud provider shares its big resources (like servers, storage, network) with many users. These resources are allocated dynamically depending on who needs them.



Cloud Computing (BCS071/BCS713)



💡 **Example:** Like an electricity board – one power station provides electricity to many houses. You don't know which exact line is giving you power, but you still get it.

4. Rapid Elasticity

👉 **Means:** You can easily increase or decrease your usage of resources. The cloud scales up or down automatically as per your needs.

💡 **Example:** Amazon website gets millions of users during Diwali sale. Cloud automatically provides more servers. After the sale ends, servers are reduced to save cost.

5. Measured Service (Pay-as-you-use)

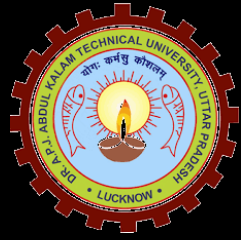
👉 **Means:** You only pay for what you use. The system measures your usage automatically.

💡 **Example:** Like your electricity or mobile data bill – you pay only for the units (GBs or kWh) you use, not for the whole power plant.

7. High Availability and Reliability

👉 **Means:** Cloud is designed to be available 24x7. If one server fails, another one takes its place. So, downtime is very low.

💡 **Example:** Gmail almost never goes down, because Google's cloud system keeps backups ready.



Cloud Computing (BCS071/BCS713)



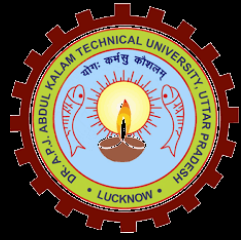
8. Security

☞ **Means:** Cloud providers give strong security like firewalls, encryption, authentication, so your data stays safe.

💡 **Example:** Like keeping your money in the bank – they have strong vaults and guards. Similarly, the cloud keeps your data safe.

❖ Disadvantages of Cloud Computing :

- Needs internet,
- Risk of security,
- Downtime issues,
- Less control,
- Can be costly,
- Data transfer speed depends on internet,
- Hard to switch providers.



Cloud Computing (BCS071/BCS713)



❖ Cloud Computing Service Provider : (IMP)

Top 10 Cloud Providers





Cloud Computing (BCS071/BCS713)



☁ Evolution of Cloud Computing(V.V.V.IMP)

Cloud Computing did not come suddenly; it evolved slowly with improvements in computing technologies.

1. Distributed Computing (1950s onwards)

Computers were connected together to share tasks.

Work was divided into small parts and executed on different machines.

Goal: Faster problem-solving by sharing resources.

2. Mainframe Computing

Big and powerful computers (mainframes) used by many users through "dumb terminals" (screens without processors).

Goal: Centralized powerful machine serving multiple users.

3. Cluster Computing

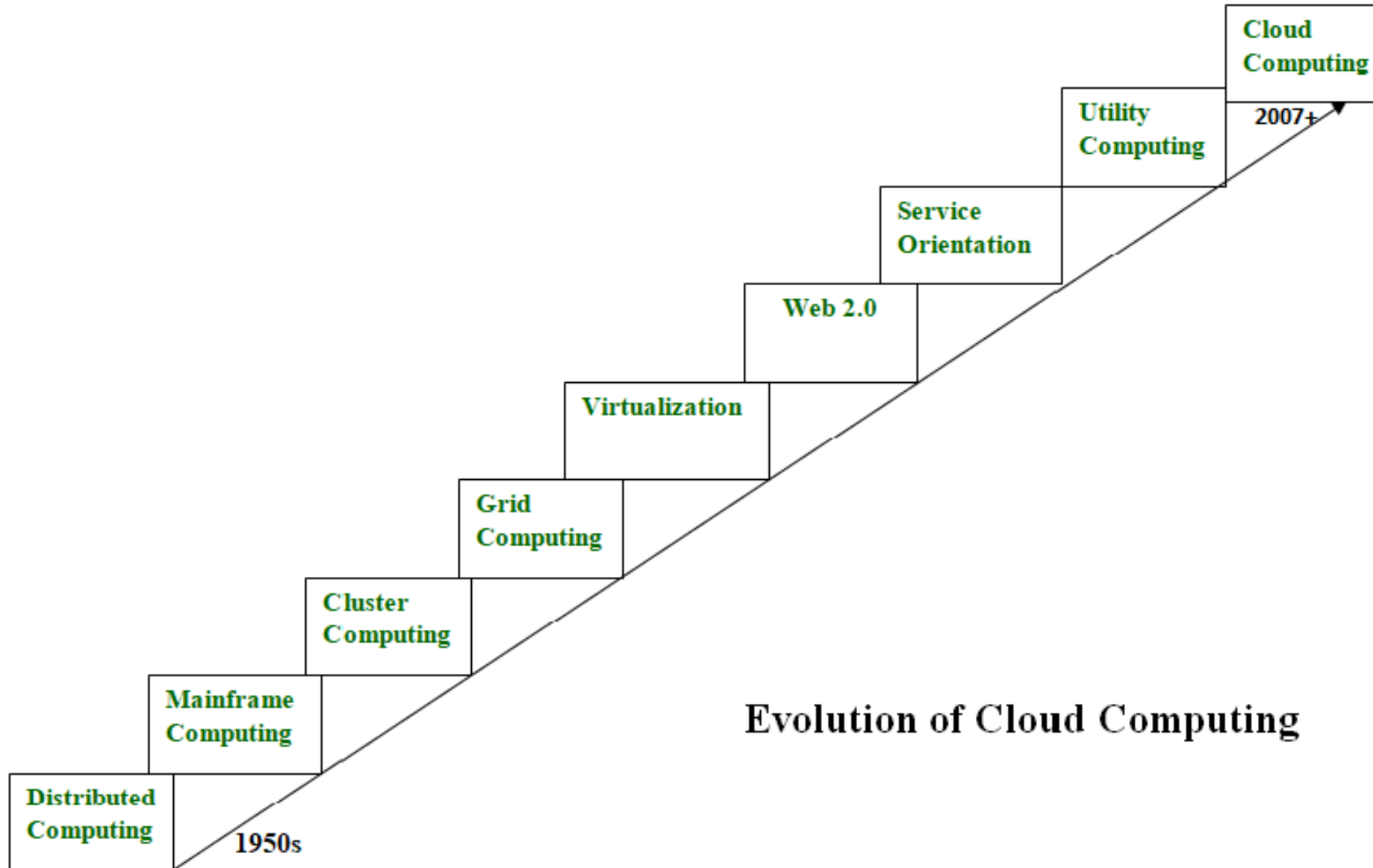
Many computers connected and working as a single powerful system.

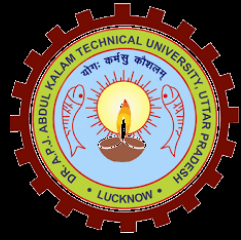
Goal: High performance and reliability at lower cost than a mainframe.



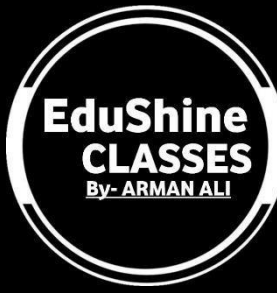


Cloud Computing (BCS071/BCS713)





Cloud Computing (BCS071/BCS713)



4. Grid Computing

Resources (computers) from different locations were connected to work together like a "virtual supercomputer".

Used for big scientific research, weather forecasting, etc.

Goal: Use idle resources worldwide efficiently.

5. Virtualization

Multiple virtual machines (VMs) could run on a single physical machine.

Improved hardware utilization, cost saving, and isolation.

Goal: Better use of one machine for multiple purposes.

7. Service Orientation

Services were delivered as **Software as a Service (SaaS)** (like Gmail, Salesforce).

Instead of buying software, people could use them online.

Goal: Provide services on demand.



Cloud Computing (BCS071/BCS713)



8. Utility Computing

Computing became like electricity → Pay for what you use.

Example: Amazon Web Services (AWS) started offering servers on rent.

Goal: On-demand computing resources at low cost.

9. Cloud Computing (2007 onwards)

All previous ideas merged to form Cloud Computing.

Provides IaaS, PaaS, SaaS on demand.

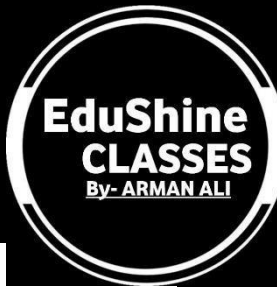
Accessible from anywhere, anytime, with scalability and flexibility.

Goal: Easy access, cost saving, global availability, and elasticity.

Distributed → Mainframe → Cluster → Grid → Virtualization → Web 2.0 → Service Orientation → Utility → Cloud.



Cloud Computing (BCS071/BCS713)



☁ Cloud Computing Architecture(VV.IMP)

Cloud computing has **two main parts**:

1. **Front-end (Client side)**
2. **Back-end (Cloud side)**

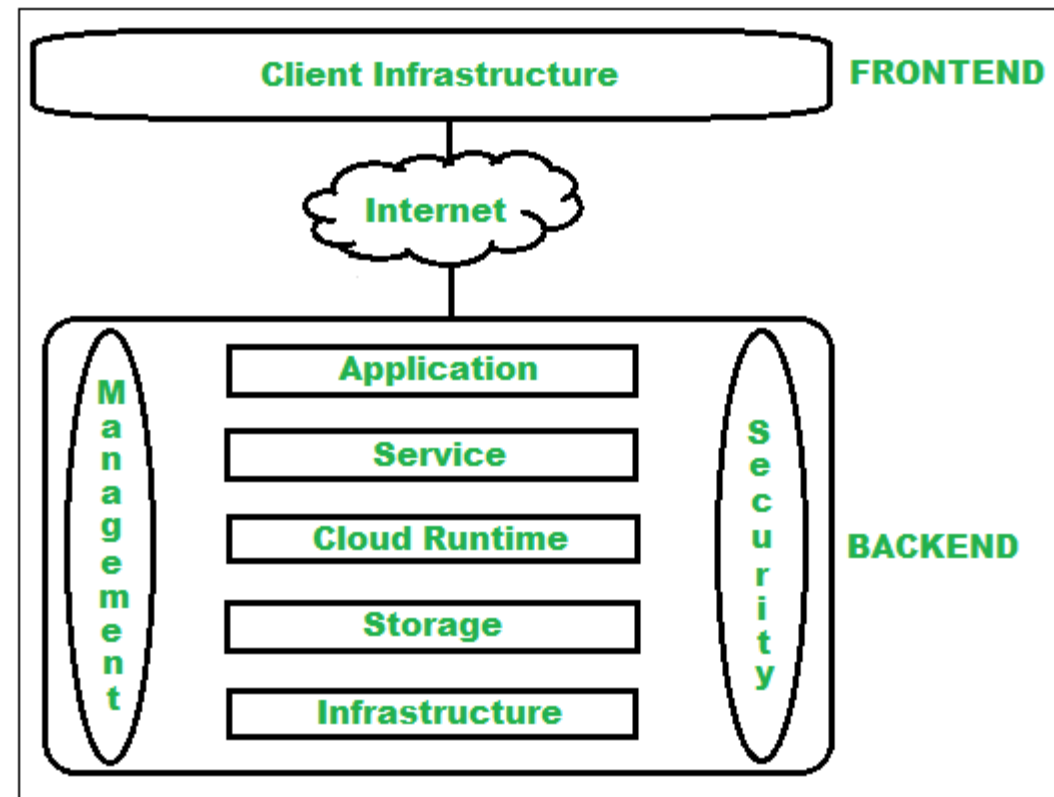
Both are connected through the

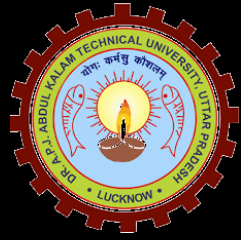
3. **Internet.**

1. Front-end (Client Infrastructure)

This is the **user side**.

Includes the **devices and applications** the user uses to access the cloud (like browsers, laptops, mobiles, thin clients).
Example: You open Gmail using your phone browser → that's the **front-end**.





Cloud Computing (BCS071/BCS713)



2. Internet (Medium)

The internet connects the client (front-end) with the cloud servers (back-end). All communication (requests & responses) happens through the internet.

3. Back-end (Cloud Infrastructure)

This is the **cloud provider side** where actual processing, storage, and service delivery happens. It includes multiple layers:

◆ a) Infrastructure

The base layer → physical hardware like servers, storage devices, and networking.

Example: Data centers that run Google Cloud, AWS, Azure.

◆ b) Storage

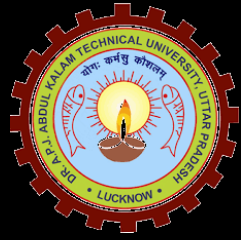
Stores all the cloud data securely.

Scalable → increases storage when demand grows.

Example: Google Drive, Dropbox.

◆ c) Cloud Runtime (Virtualization Layer)

Provides a **virtual environment** where applications run.



Cloud Computing (BCS071/BCS713)



Manages resources like CPU, memory, and storage using virtualization.

Example: VMware, Hyper-V.

◆ d) Services

Provides cloud services like SaaS, PaaS, and IaaS.

Example: Gmail (SaaS), Google App Engine (PaaS), AWS EC2 (IaaS).

◆ e) Application

The actual applications that users access.

Example: Google Docs, Microsoft Office 365, Netflix.

4. Management

Controls and manages all resources in the cloud (applications, services, runtime, storage, infrastructure).

Ensures smooth functioning and resource allocation.

5. Security

Provides **data protection, authentication, and privacy**.

Very important because cloud is shared among many users.



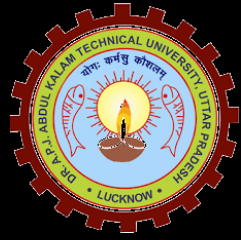
Cloud Computing (BCS071/BCS713)



Example: Firewalls, encryption, access control.

❖ Application of Cloud Computing :

- **Data Storage & Backup** – Store files, photos, and videos online (e.g., Google Drive, Dropbox).
- **Web Hosting** – Hosting websites and applications on cloud servers.
- **Software as a Service (SaaS)** – Using apps like Gmail, MS Office 365 without installing.
- **E-commerce** – Online shopping platforms like Amazon, Flipkart use cloud.
- **Social Networking** – Facebook, Instagram, Twitter run on cloud servers.
- **Education** – Online classes, e-learning platforms (Coursera, Google Classroom).
- **Banking & Finance** – Online transactions, mobile banking apps.
- **Entertainment** – OTT platforms like Netflix, Hotstar use cloud for video streaming.
- **Business Applications** – CRM tools, project management, and collaboration (Zoom, Slack).
- **Healthcare** – Storing patient records and telemedicine services.



Cloud Computing (BCS071/BCS713)



❖ What is Parallel Computing?(V.VVIMP)

Parallel computing means **dividing a big problem into smaller parts**.

These smaller parts are solved **at the same time** (simultaneously) using **multiple processors (CPUs/cores)**.

Finally, the results are combined to get the **final output**.

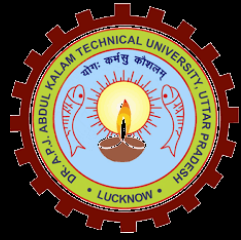
☞ **Example:** Imagine you want to wash 100 clothes.

Normal way (sequential): 1 person washes all clothes one by one.

Parallel way: 4 people wash 25 clothes each at the same time → work finishes faster.

➤ Steps in Parallel Computing :

1. **Problem** – A large task that needs to be solved.
2. **Divide into Instructions** – Break the problem into smaller instructions or tasks.
3. **Assign to Processors** – Each processor gets a part of the work.
4. **Execute Simultaneously** – All processors work at the same time.
5. **Combine Results** – The outputs from all processors are merged into the final solution.



Cloud Computing (BCS071/BCS713)

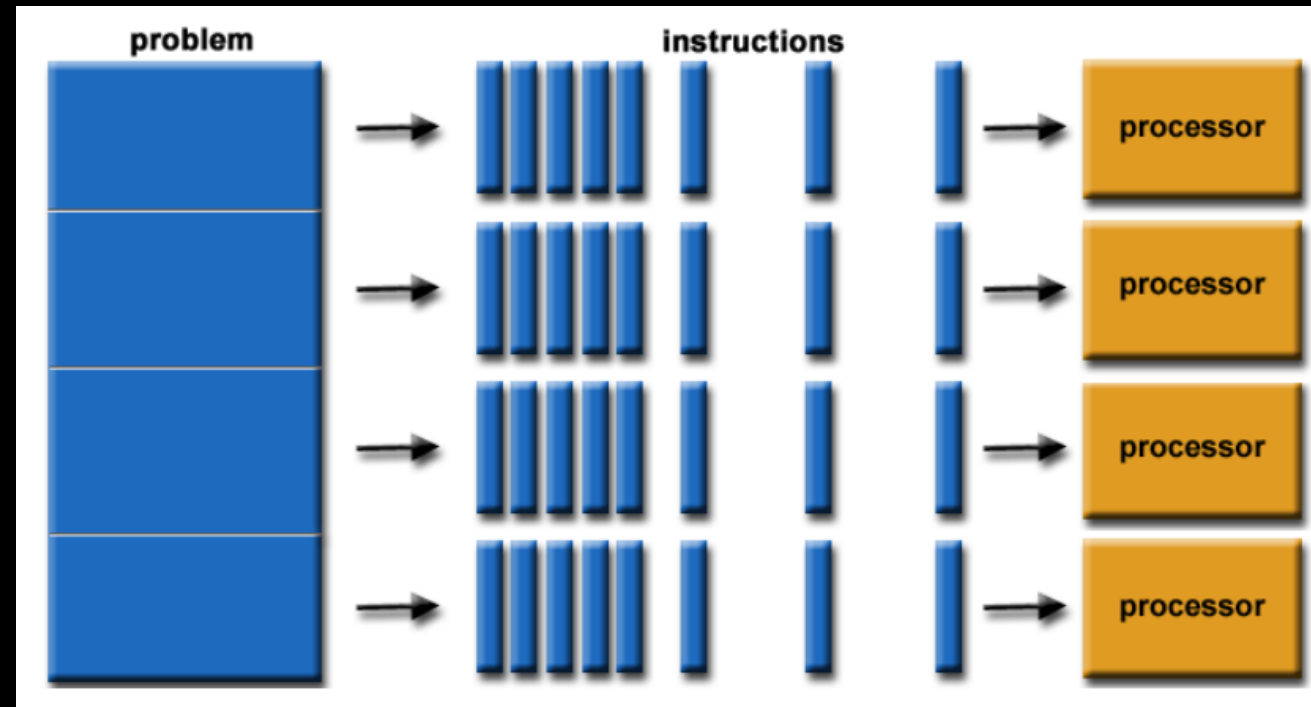


◆ Advantages of Parallel Computing over Serial Computing

- ✓ Saves **time and money** (work is done faster).
- ✓ Can solve **big and complex problems** (serial computing fails here).
- ✓ Uses **resources more effectively** (no idle CPU parts).
- ✓ Allows use of **distributed resources** (not only local machines).

❖ Disadvantages :

- Difficult to divide some problems.
- Requires special hardware/software.
- Costly setup if many processors are needed.





Cloud Computing (BCS071/BCS713)



❖ ♦ Types of Parallelism

1. Bit-level Parallelism

Processor handles more bits at a time.

Example: An 8-bit processor adds a 16-bit number in 2 steps, but a 16-bit processor does it in 1 step.

2. Instruction-level Parallelism

Processor rearranges and executes multiple instructions in one clock cycle (without changing result).

3. Task Parallelism

A big task is divided into smaller subtasks.

Subtasks run at the same time on different processors.

4. Data Parallelism (DLP)

Same instruction is applied on multiple data sets at once.

Example: Adding two arrays by splitting data among processors.



Cloud Computing (BCS071/BCS713)



2. Distributed Computing

Distributed computing is a system where **many computers (or machines) work together to solve a single big problem.**

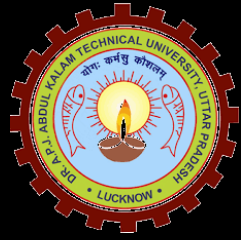
Instead of one computer doing all the work, the work is **divided (distributed)** among many computers connected through a network.

✓ Main Idea:

- A big task is **broken into small parts.**
- These parts are given to different computers.
- All computers work at the same time (parallel).
- At the end, results are combined to get the final answer.

❖ Key Features:

1. **Multiple Computers** – connected by a network.
2. **Shared Workload** – tasks are divided among them.
3. **Faster Processing** – work is done quickly.
4. **Fault Tolerance** – if one machine fails, others can continue.



Cloud Computing (BCS071/BCS713)



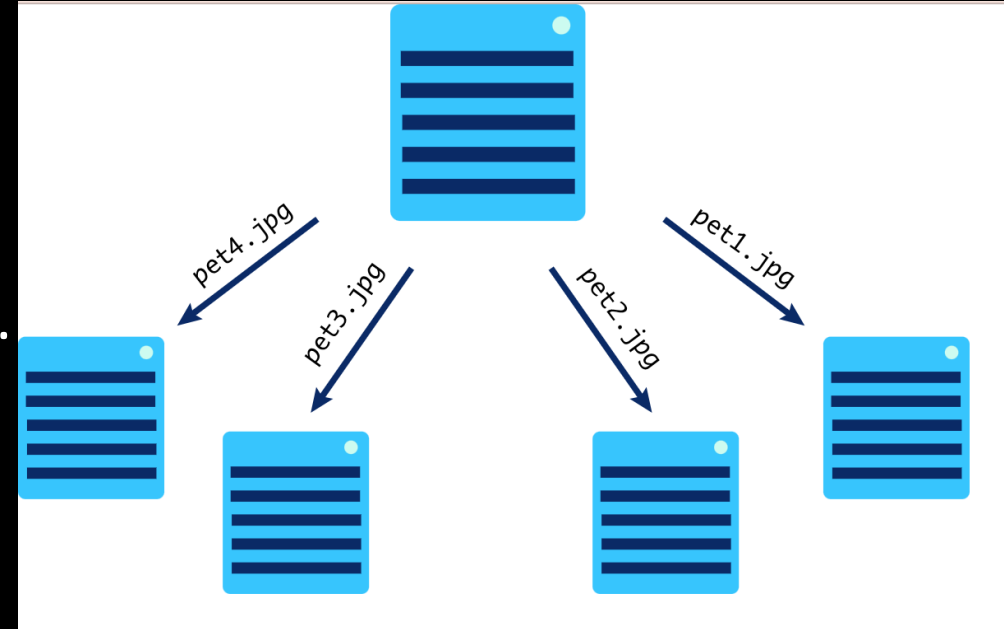
Advantages:

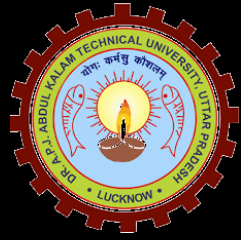
- Saves **time** (parallel processing).
- Uses resources more efficiently.
- Can handle **big problems** that one computer cannot.

Simple Real-Life Examples:

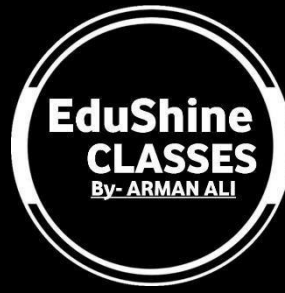
1. **Google Search** – thousands of servers work together.
2. **Banking Systems** – transactions handled by multiple computers.
3. **Weather Forecasting** – uses many supercomputers together.

Distributed computing is a model where many computers work together on a problem by dividing tasks among them and connecting through a network. It improves speed, resource sharing, fault tolerance, and can solve very large problems. Example: Google search engine and weather forecasting use distributed computing.



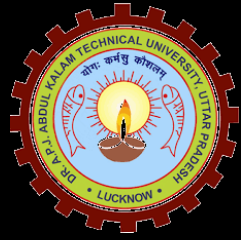


Cloud Computing (BCS071/BCS713)



❖ Difference between Parallel Computing Vs Distributed Computing :

S.No.	Parallel Computing	Distributed Computing
1	Many operations are performed simultaneously.	System components are located at different locations.
2	Single computer is required.	Uses multiple computers.
3	Multiple processors perform multiple operations.	Multiple computers perform multiple operations.
4	It may have shared or distributed memory.	It only has distributed memory.
5	Processors communicate via a bus.	Computers communicate through message passing.
6	Improves system performance.	Enhances system scalability, fault tolerance, and resource sharing.



Cloud Computing (BCS071/BCS713)



☁️ Cloud Elasticity

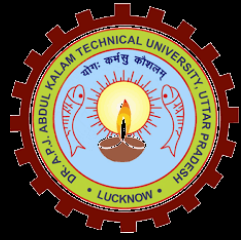
- Elasticity means **automatic adjustment of resources** (CPU, RAM, storage, bandwidth) depending on sudden demand.
- When more people use your app/website → cloud **adds extra resources automatically**.
- When fewer people use → cloud **removes extra resources automatically**.
- Saves **money** because you only pay for what you use (pay-per-use).
- Best for **temporary / seasonal demand**.

👉 Example:

An online shopping site gets huge traffic during **Christmas sales**. Elasticity helps it automatically add more servers during sales and remove them when sales end

☁️ Cloud Scalability

Scalability means **growing resources steadily (manually or automatically)** to handle **permanent increase in workload**.



Cloud Computing (BCS071/BCS713)



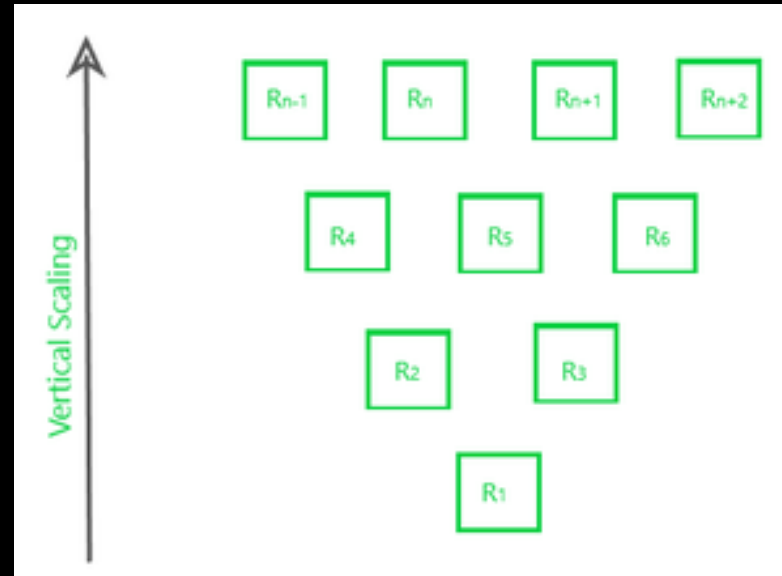
- It is **planned growth**, not sudden like elasticity.
- Best for **long-term resource needs**.

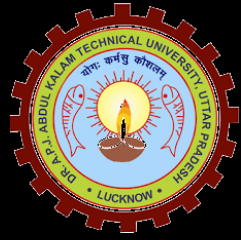
➡ Example:

A company's **database** keeps growing every year as business expands. They **scale up** storage and computing power to handle the larger workload.

↻ Types of Scalability

1. **Vertical Scalability (Scale Up):** Increase the power of existing machine (e.g., add more RAM or CPU to the same server).

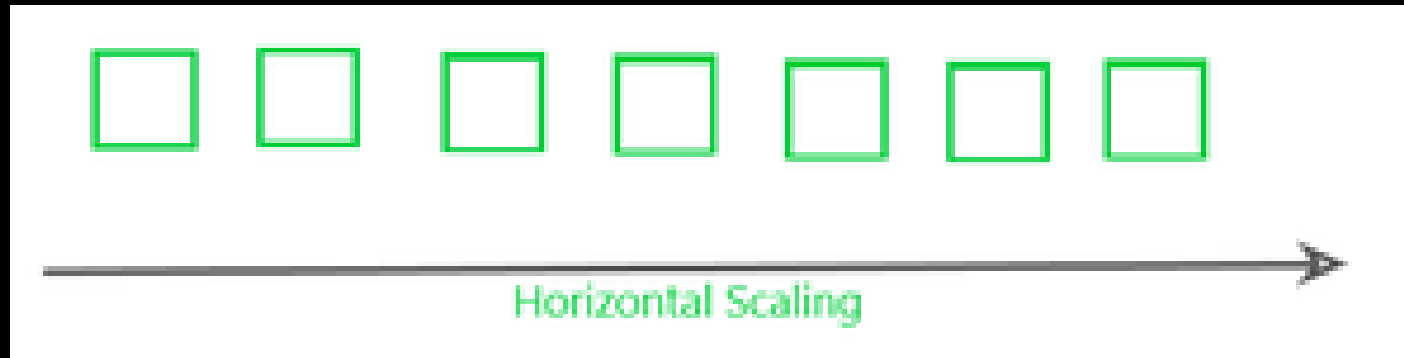




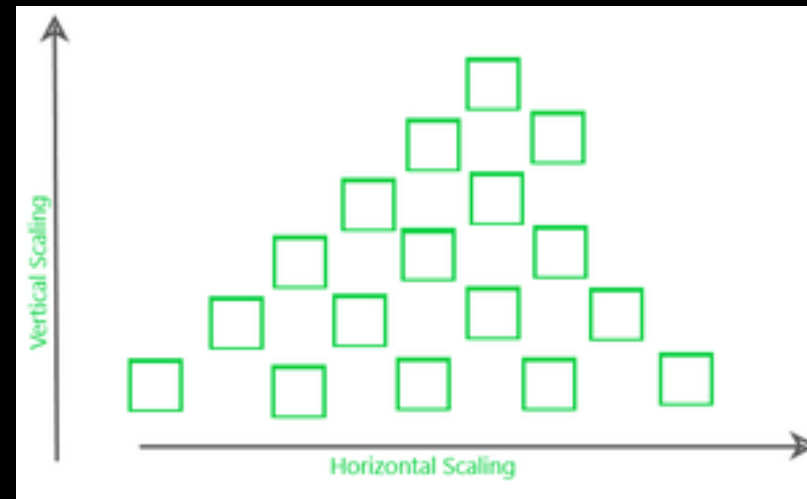
Cloud Computing (BCS071/BCS713)

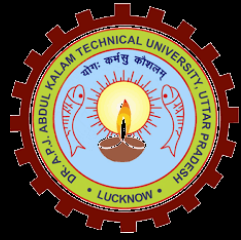


2. Horizontal Scalability (Scale Out): Add more machines/servers to share the load.

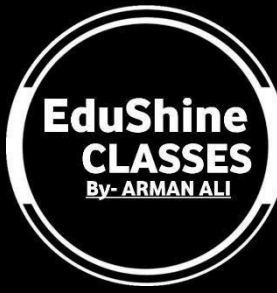


3. Diagonal Scalability: A mix of both vertical + horizontal scaling.





Cloud Computing (BCS071/BCS713)



❖ Difference between Elasticity & Scalability : (IMP)

Elasticity

Handles **sudden increase or decrease** in workload.

Dynamic (resources go up & down automatically).

Used by **small/medium businesses** with seasonal demands.

Short-term solution.

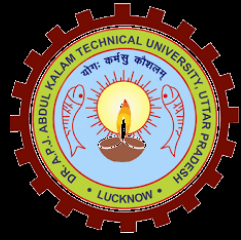
Scalability

Handles **steady, predictable increase** in workload.

Static/Planned (resources are increased when needed permanently).

Used by **big companies** with continuously growing users.

Long-term solution.



Cloud Computing (BCS071/BCS713)



❖ On-Demand Provisioning in Cloud Computing :

On-demand provisioning means getting computing resources (like servers, storage, or applications) whenever you need them, and releasing them when you don't.

It is just like ordering electricity, water, or food online – you get it instantly when required, and stop when you don't need it.

1 .Traditional Way (Before Cloud):

- Companies had to buy and install their own servers.
- Even if they used them for 2 hours only, they had to pay for the whole machine.
- This was expensive and time-wasting.

2. With On-Demand Provisioning (Cloud):

- You don't buy hardware.
 - You just request resources (CPU, storage, database, etc.) from the cloud.
 - Cloud provider (like AWS, Azure, Google Cloud) gives you resources within seconds.
 - You pay only for what you use.
-



Cloud Computing (BCS071/BCS713)



❖ Key Features :

- **Instant availability** – Resources are ready quickly.
- **Scalability** – Increase or decrease resources anytime.
- **Pay-as-you-go** – No need to spend extra money.
- **Flexibility** – Use resources only when needed.

Example

- Imagine you are building a website.
- During normal days → You need only 2 servers.
- During festival season (Diwali sales) → You suddenly need 20 servers.
- With on-demand provisioning → You can instantly add 18 extra servers and remove them later when traffic reduces.

On-demand provisioning in cloud computing means **providing IT resources (like servers, storage, databases) instantly whenever needed, instead of pre-installing them.** It saves **time, money, and effort**, and allows **scaling up and down** as per demand.



Cloud Computing (BCS071/BCS713)



Thank You....



Download Notes : <https://rzp.io/rzp/IHQaLO0N>



Cloud Computing (BCS071/BCS713)



❖ What is Cloud enabling Technologies :

☞ Cloud Enabling Technologies are those basic tools and technologies that make cloud computing possible.

Without these, cloud can't work.

Think of it like this ☞

If cloud computing = a house, then

Cloud Enabling Technologies = bricks, cement, electricity, water → the things needed to build and run the house.

❖ Main Cloud Enabling Technologies:

1. **Virtualization** – allows one physical machine (computer) to run many virtual machines. (Like dividing one big cake into many small slices).
2. **Broadband Networks & Internet** – cloud works only because of fast internet.
3. **Service-Oriented Architecture (SOA)** – designing software in small services so they can easily run on cloud.
4. **Utility Computing** – pay only for what you use (like electricity bill).





Cloud Computing (BCS071/BCS713)



5. Grid Computing – combining power of many computers to work together.

✓ In short:

Cloud Enabling Technologies = the backbone technologies that make cloud services like Google Drive, AWS, Azure work.

❖ **Service Oriented Architecture (SOA) :**

- It is a **way of designing software** where different parts of the software are made as **separate services**.
- Each service does **one specific job** (like login, payment, search, etc.).
- These services can **talk to each other** through the internet or a network.
- The main idea is **reusability** → once a service is created, it can be used in different applications.





Cloud Computing (BCS071/BCS713)



✓ Example:

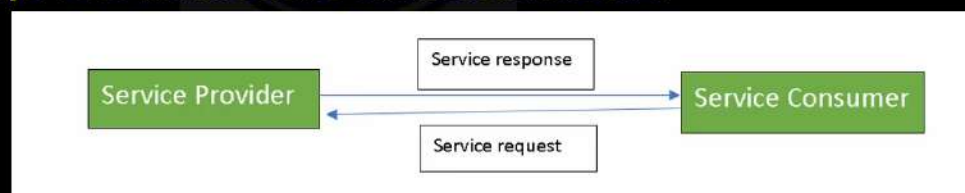
Think of a **restaurant** 🍽️👤

- There is a **chef service** (cooks food),
- **Waiter service** (takes order),
- **Cashier service** (handles payment).
- All of them work separately but **together they make the restaurant run**.

Similarly, in SOA:

- **Payment service, User service, Email service** etc. are separate, but together they make the whole application.

There are **two major roles** within Service-oriented Architecture:





Cloud Computing (BCS071/BCS713)



1. Service provider:

The service provider is the maintainer of the service and the organization that makes available one or more services for others to use. To advertise services, the provider can publish them in a registry, together with a service contract that specifies the nature of the service, how to use it, the requirements for the service, and the fees charged.

2. Service consumer:

The service consumer can locate the service metadata in the registry and develop the required client components to bind and use the service.

Now let Discuss main Key characteristics of SOA :





Cloud Computing (BCS071/BCS713)



❖ Key Characteristics of SOA :

1. Loose Coupling

Services are independent.

They don't depend too much on each other, so one service can change without breaking others.

2. Reusability

A service created once can be used in many applications.

Example: A "Payment Service" can be used in shopping apps, ticket booking apps, etc.

3. Interoperability

Services can work together even if they are built in different programming languages or run on different systems.

4. Discoverability

Services are easy to find (like searching in a directory).

Developers can search and use them without building from scratch.





Cloud Computing (BCS071/BCS713)



5. Composability

Small services can be combined together to make bigger, complex applications.

6. Statelessness

Services don't remember past information.

Each request is independent (like every time you open Google, it doesn't remember your last search unless you log in).

7. Autonomy

Each service controls itself.

It works independently without needing full control from others.

8. Standardized Service Contract

Services follow rules and standards for communication (like using common languages such as XML/JSON).

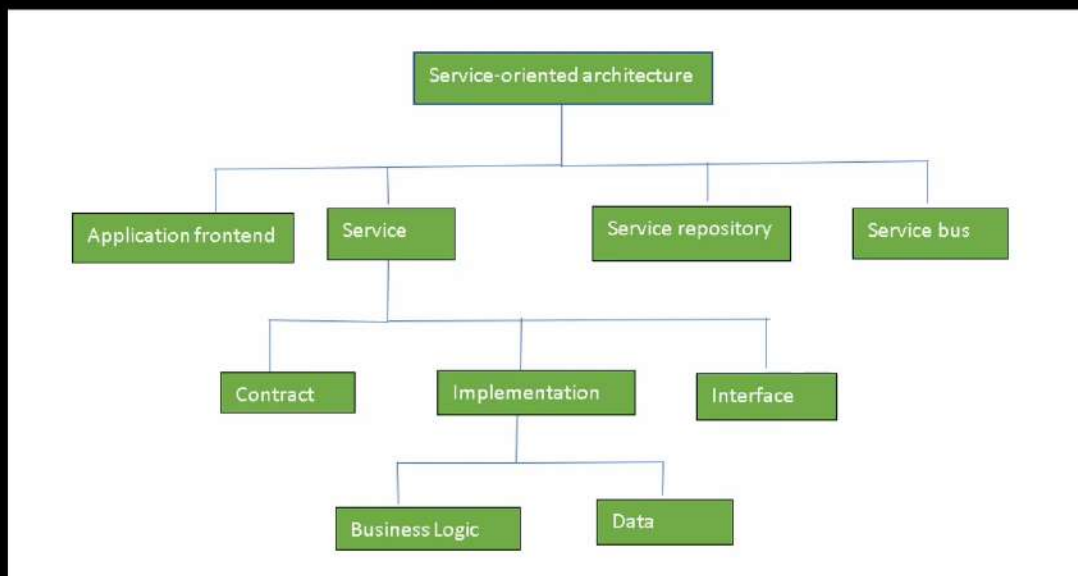




Cloud Computing (BCS071/BCS713)



❖ Service Oriented Architecture (SOA) :





Cloud Computing (BCS071/BCS713)



1. Application Frontend

This is the **user side** (what we see and interact with).

Example: A shopping app's webpage or mobile app.

2. Service

- The **core building block** of SOA.
- Each service does a specific job (like "Payment Service," "Search Service").
- Inside a service, we have three parts:
 - **Contract** → The rules (what the service offers and how to use it).
 - **Implementation** → The actual working code of the service.
 - Inside implementation:
 - **Business Logic** → The main rules/process (e.g., how to calculate payment).
 - **Data** → The information the service uses (like product list, user info).
 - **Interface** → The entry point (like a door to the service). Other services or apps use this to communicate.





Cloud Computing (BCS071/BCS713)



3. Service Repository

Think of this as a **library or dictionary** of all available services.

Developers can look here to find and reuse services instead of making new ones.

4. Service Bus

The **communication system** (like a post office or internet router).

It helps different services talk to each other smoothly, even if they are built differently.

★ Advantages

1. **Time Saving** – People don't need to check multiple sources; they can quickly see the weather on your app.
2. **Accuracy** – Since it uses real-time **Weather API**, the data is updated and reliable.
3. **User-Friendly** – Simple design helps even non-technical users understand weather details.
4. **Accessibility** – Can be used anywhere, anytime (website or app).
5. **Decision Making** – Helps people plan travel, farming, outdoor events, or daily routine.





Cloud Computing (BCS071/BCS713)



⚡ Challenges

1. **Internet Dependency** – App only works if there is an internet connection.
2. **API Limitations** – Free APIs have request limits, so too many users can cause errors.
3. **Accuracy Issues** – Weather prediction is not always 100% correct (nature is unpredictable).
4. **Data Cost** – If you use premium APIs, it may cost money.
5. **Technical Maintenance** – The app must be updated to handle bugs, server issues, or API changes.

🌐 Applications

1. **Daily Life** – People can check weather before going to office, college, or travel.
2. **Agriculture** – Farmers can plan irrigation, sowing, or harvesting.
3. **Transportation** – Airlines, shipping, and railways depend on weather info for safety.
4. **Event Planning** – Outdoor functions, weddings, or sports matches can be planned better.
5. **Disaster Management** – Helps warn people about storms, floods, or heavy rain.





Cloud Computing (BCS071/BCS713)



❖ What is REST?

REST = Representational State Transfer

It is a set of **rules/standards** used to design web APIs so that systems can talk to each other easily.

REST stands for **Representational State Transfer** and **API** stands for **Application Program Interface**. REST is a software architectural style that defines the set of rules to be used for creating web services. Web services that follow the REST architectural style are known as RESTful web services. It allows requesting systems to access and manipulate web resources by using a uniform and predefined set of rules. Interaction in REST-based systems happens through the Internet's Hypertext Transfer Protocol (HTTP).

Example: When you use **Instagram**, your app talks to Instagram's server using REST API (like "get my profile", "post a photo").





Cloud Computing (BCS071/BCS713)



A Restful system consists of a:

- ✓ A client who requests for the resources.
- ✓ server who has the resources.

It is important to create REST API according to industry standards which results in ease of development and increase client adoption.

Client interact with the resources by HTTP methods –

- GET – Retrieve the data
- POST – Post the into Database
- PUT – To Update the data
- DELETE - To delete the data





Cloud Computing (BCS071/BCS713)



🌐 Architectural Constraints of REST

REST has **6 main rules (constraints)**. If an API follows these, we call it a **RESTful API**.

1. Uniform Interface
2. Stateless
3. Cacheable
4. Client-Server
5. Layered System
6. Code on Demand

URI	HTTP verb	Description
api/users	GET	Get all users
api/users/new	GET	Show form for adding new user
api/users	POST	Add a user
api/users/1	PUT	Update a user with id = 1
api/users/1/edit	GET	Show edit form for user with id = 1
api/users/1	DELETE	Delete a user with id = 1
api/users/1	GET	Get a user with id = 1





Cloud Computing (BCS071/BCS713)



1. Uniform Interface

☞ Rule: Use the same standard way to talk to the server.

- Example: Always use HTTP methods like
 - GET → Read data
 - POST → Add data
 - PUT → Update data
 - DELETE → Remove data
- Why? So all apps talk in the same "language" with the server.

2. Stateless

☞ Rule: Server does not remember old requests.

Every request must carry all information needed.

Example: If you log in, every time you ask for data, you send your **token/ID**. The server doesn't "remember" who you are unless you tell it.





Cloud Computing (BCS071/BCS713)



3. Cacheable

☞ Rule: Responses should say whether they can be saved for later.

Example: Weather app can **cache** today's weather for 10 minutes instead of asking the server again and again.

Benefit: Faster and reduces server load.

4. Client-Server

☞ Rule: Client and server should be separate.

Client = front-end (app/browser).

Server = back-end (database, logic).

Example: Your Zomato app (client) asks for food items → the Zomato server sends the list. Both are separate, so they can evolve independently.





Cloud Computing (BCS071/BCS713)



5. Layered System

☞ Rule: There can be many layers between client and server, but client doesn't need to know.

Example: When you request YouTube video:

It may go through **load balancer** → **cache server** → **main server**, but you don't care. You just get your video.

6. Code on Demand (optional)

☞ Rule: Server can send extra code to client.

Example: Server sends a small JavaScript program to your browser to improve UI.

Note: This is optional, not always used.





Cloud Computing (BCS071/BCS713)



❖ System of Systems (SoS)

A **System of Systems** is when many **independent systems** (that can work on their own) come together and connect to form a **larger, more complex system** that provides new capabilities.

◆ Each smaller system inside SoS is:

1. **Independent** (it can work alone)
2. **Operational** (already useful by itself)
3. **Managed separately** (different organizations/people may control them)

But when these systems are **linked together**, they achieve something **bigger and better** than what each system could do individually.

✓ Key Idea:

A **system of systems** = many useful systems working together to provide **greater functionality** than the sum of their parts.





Cloud Computing (BCS071/BCS713)



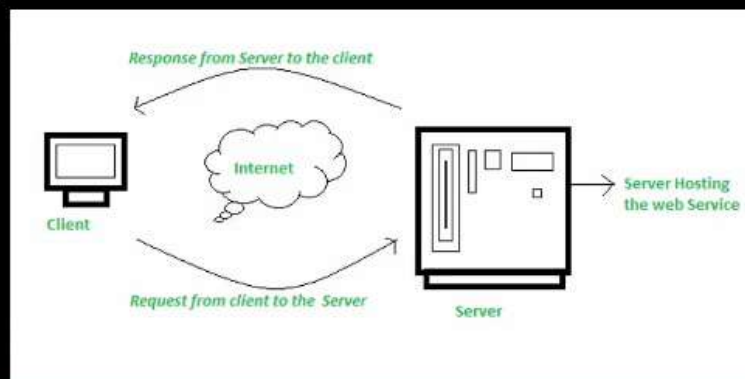
🌐 Web Services in Cloud Computing

❏ What are Web Services?

Web services are like **online messengers** that allow **two different applications** (even if they are built in different languages or platforms) to **talk to each other** over the internet.

They help in **sharing data** and **functions** between systems.

Example: A **flight booking website** uses web services to talk with **airline servers** and show available tickets.





Cloud Computing (BCS071/BCS713)



❖ Main Components of Web Services

1. XML (Extensible Markup Language)

A language for structuring and sharing data.

Works like a **common format** so all applications can understand.

2. SOAP (Simple Object Access Protocol)

A protocol (rule) to send messages using **XML** over the internet (usually with **HTTP**).

3. WSDL (Web Services Description Language)

A file that **describes what a web service does** and how to use it.

Like a **user manual** for the web service.

4. UDDI (Universal Description, Discovery, and Integration)

A **directory** where web services are published and discovered.

Like a **yellow pages** for web services.

5. REST (Representational State Transfer)

A simpler way than SOAP, uses **HTTP (GET, POST, PUT, DELETE)**.

Shares data mostly in **JSON** format.





Cloud Computing (BCS071/BCS713)



6. HTTP (HyperText Transfer Protocol)

Standard web protocol to exchange data between client and server.

7. JSON (JavaScript Object Notation)

Lightweight and easy-to-read data format, mainly used in REST.

❖ Features of Web Services

1. Interoperability → Different platforms and languages can work together.

2. Reusability → Services can be reused in different apps.

3. Scalability → Can handle more users by adding servers.

4. Loose Coupling → Services are independent, not tightly linked.

5. Security → Uses SSL/TLS, OAuth, WS-Security.

6. Discoverability → Services can be searched and used via UDDI.

7. Platform Independence → Works on any OS or language.

Web services in cloud computing are online tools that allow different applications to communicate and share data over the internet using standard protocols like HTTP, XML, SOAP, REST, and JSON.





Cloud Computing (BCS071/BCS713)



❖ Web services are mainly of two types. Let me explain in easy words 🗣️

1. SOAP Web Services (Simple Object Access Protocol)

It is an **old/traditional** type of web service.

It uses **XML format** for sending and receiving data.

Very **secure and reliable**, mostly used in banking, telecom, or large organizations.

But it is **slow and heavy** because XML is big in size.

✓ Example: A bank system sending secure transaction details to another bank.

2. RESTful Web Services (Representational State Transfer)

It is a **modern and simple** web service.

It uses **HTTP methods** (GET, POST, PUT, DELETE).

Data can be sent in **JSON, XML, or text**, but mostly JSON because it is **lightweight and fast**.

Easy to use, scalable, and works with mobile apps, websites, and cloud services.

✓ Example: Weather app getting temperature details from an online weather API.





Cloud Computing (BCS071/BCS713)



★ What is Publish-Subscribe Model?

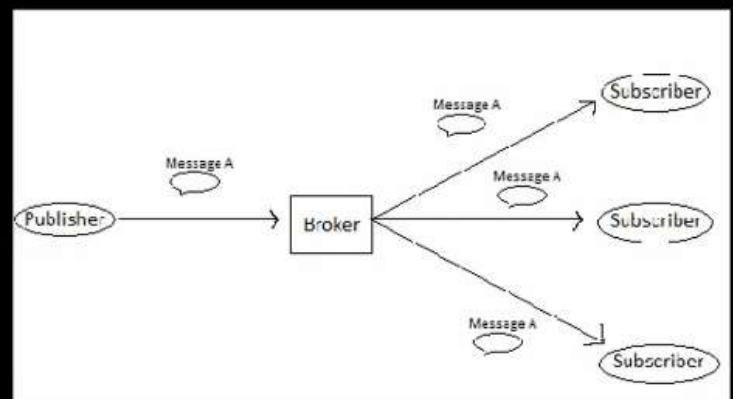
The **Publish-Subscribe** model is a messaging pattern where **senders (Publishers)** do not directly send messages to **receivers (Subscribers)**.

Instead, there is a **broker (or message channel)** in the middle that delivers the messages only to those subscribers who are interested.

☞ Example in real life:

Think about **YouTube** –

- You (Publisher) upload a video.
- People who **subscribed** to your channel get notified.
- People who didn't subscribe don't get the notification.





Cloud Computing (BCS071/BCS713)



★ Components of Publish-Subscribe Model

1. Publisher

The sender of the message.

It publishes (sends) messages without worrying about who will read them.

2. Subscriber

The receiver of the message.

It subscribes (shows interest) to certain topics/events.

3. Broker (Message Channel / Middleware)

Acts like a **postman**.

Takes the message from the publisher and delivers it to all the interested subscribers.

Example: **Kafka, RabbitMQ, MQTT Broker.**

4. Topic/Event

A category or subject where messages are published.

Subscribers choose the topics they care about.





Cloud Computing (BCS071/BCS713)



★ How it Works (Step by Step)

- ✓ **Publisher sends message** → Publishes a message to a specific topic (e.g., "Weather Updates").
- ✓ **Broker receives message** → The broker holds the message temporarily.
- ✓ **Subscribers express interest** → They subscribe to that topic (e.g., "Weather Updates").
- ✓ **Broker delivers message** → Broker forwards the message only to subscribers of that topic.

◆ Example

- **Publisher:** "EduShine Classes" channel uploads a video on *Cloud Computing*.
- **Broker:** YouTube platform stores, processes, and manages the video.
- **Subscriber:** You and other 10,000 subscribers.
- **Topic:** "Unit 2 One shot Uploaded".
- **Event:** Subscribers get a notification → "New Unit 2 One shot video is live!"

Publishers don't directly send content to subscribers.

Instead, the **broker (YouTube)** takes care of delivering content only to those who subscribed to that topic.





Cloud Computing (BCS071/BCS713)



❖ Basic of Virtualization : (kahani smjho Virtualization ki)

🏢 The Company's Problem (Without Virtualization)

A company needs 4 different servers for:

- Storing **customer data** (needs Windows + lots of storage).
- Running **online shopping website** (needs Linux + can handle many visitors).
- Doing **employee payroll** (needs more RAM + specific OS).
- Running **social media campaigns** (needs its own system).

👉 Without virtualization:

They must buy 4 **separate physical servers** (big computers).

Each server costs money, uses electricity, needs maintenance.

Managing 4 machines separately = **expensive + complicated**.

Now Lets Solve this problem using Virtualization Concept →





Cloud Computing (BCS071/BCS713)



💡 The Solution (With Virtualization)

Instead of buying 4 physical servers, they buy **just 1 or 2 powerful servers**.

Then, using **virtualization software**, they create **4 Virtual Machines (VMs)** inside it:

- VM1 → for customer data (Windows + storage).
- VM2 → for website (Linux + handles traffic).
- VM3 → for payroll (special RAM + OS).
- VM4 → for marketing (custom system).

☞ Each VM works like a **separate server**, but all are running on **the same physical server**.

👉 Benefits :

Save **money** 💰 (fewer servers to buy).

Save **space & energy** ⚡.

Easy to **manage** (all VMs in one place).

Flexible → can add more VMs anytime without buying new machines.





Cloud Computing (BCS071/BCS713)



☞ Virtualization means creating a **virtual (not real, but working like real)** version of something instead of using the physical one.

That "something" can be:

- **Operating System**
- **Server**
- **Storage**
- **Network**

It allows **one physical computer** to run **many virtual computers** at the same time.

Suppose you have **one powerful laptop**.

Normally, you can install only **one operating system (like Windows)**.

- But with virtualization software (like VMware, VirtualBox, Hyper-V):
- You can install **Windows, Linux**, and even **Mac OS** on the same laptop.
- Each OS will run as if it has its **own computer**.
- But in reality, all of them are running on **one physical machine**.





Cloud Computing (BCS071/BCS713)



Working of Virtualization :

1. Virtual Machines (VMs)

Imagine you have **one big computer** (the real physical computer).

Virtualization lets you **divide it into many small computers** inside it.

These “small computers” are called **Virtual Machines (VMs)**.

Each VM works like a **real computer**: it can have its own **operating system, apps, and settings**.

 **Example:** On one laptop, you can run **Windows VM, Linux VM, and Ubuntu VM** all at the same time.

2. Host and Guest

The **real physical computer** = **Host**.

The **virtual machines running inside it** = **Guests**.

One Host can run **many Guests** at the same time.





Cloud Computing (BCS071/BCS713)



3. Hypervisor (the manager 📁)

The magic tool that makes virtualization possible is called a Hypervisor.

It sits **between the hardware and VMs**.

It **controls how CPU, RAM, storage, and network** are shared between different VMs.

👉 **Example:** If one VM needs more memory, it asks the **hypervisor**, and the hypervisor gives it from the Host.

Types of Hypervisors

1. Type 1 (Bare-Metal Hypervisor):

Installed **directly on the computer hardware** (no OS in between).

Super fast & efficient.

Used in data centers and cloud (like VMware ESXi, Microsoft Hyper-V).

2. Type 2 (Hosted Hypervisor):

Installed **on top of an Operating System** (like a normal app).

Easier for personal use/testing.

Examples: VirtualBox, VMware Workstation.

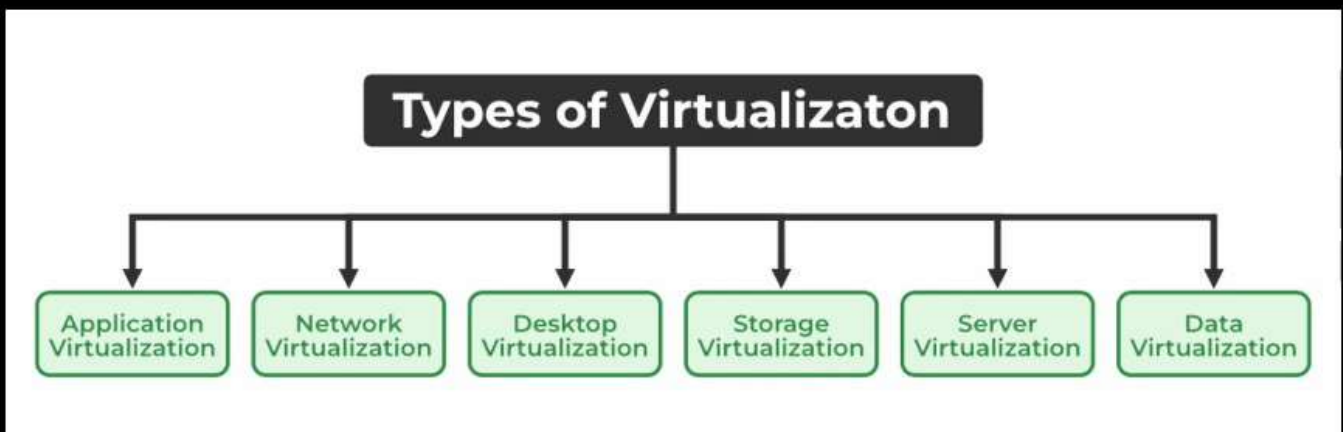




Cloud Computing (BCS071/BCS713)



Types of Virtualization :





Cloud Computing (BCS071/BCS713)



1. Application Virtualization:

- Normally, to use software (like MS Word, Photoshop, or Payroll software), you have to install it on your own computer.
- But with Application Virtualization, you don't need to install it.
- The software is actually running on a server (cloud or company server), and you just use it through the internet on your device.
- It looks and feels like the app is on your computer, but it's really somewhere else.

Example :

☞ Microsoft Azure Virtual Apps

- Company puts its business apps (like Payroll system, Sales software, CRM) on Azure servers.
- Employees just log in from their laptop or tablet.
- It looks like the app is installed on their device, but it's actually running in the cloud.
- This saves time, storage, and makes it secure.





Cloud Computing (BCS071/BCS713)



2. What is Network Virtualization?

- Normally, in a company, to connect computers you need real devices like routers, switches, firewalls, and cables.
- But with Network Virtualization, you don't always need physical devices.
- Instead, you can create virtual (software-based) networks inside one physical network.
- Each virtual network works independently, just like a real one.

Example :

☞ Google Cloud Network Virtualization

A company creates its own virtual private network in Google Cloud.

They set up IP addresses, firewalls, and secure connections using software (no need to buy real routers or switches).

If the company grows, they can expand the network quickly without buying new hardware.





Cloud Computing (BCS071/BCS713)



3. Desktop Virtualization :

- Instead of having one physical desktop computer for each user, **virtual desktops** are created on a server.
- Users can **log in from any device** (laptop, tablet, phone) and still see their desktop.
- The desktop runs in the **cloud or data center**, not on the user's personal device.

Example :

👉 **GeeksforGeeks** uses **Amazon WorkSpaces / Google Cloud Desktops**.

Employees log in from any device and see the same coding setup.

No need for everyone to have a powerful computer.

Easy to **manage, update, and secure**.





Cloud Computing (BCS071/BCS713)



4. Storage Virtualization :

- Combines storage from **many servers/disks** into **one single storage system**.
- Users don't see multiple devices; they just see **one large storage space**.
- Even if hardware changes or fails, storage works smoothly.

Example :

☞ Amazon S3

A multinational company stores **all files and data** in S3.

Employees can access them **from anywhere** securely.

No need to manage separate storage devices.

5. Server Virtualization :

- A **physical server** is divided into **multiple virtual servers (VMs)**.
- Each virtual server runs **independently** with its own OS and apps.
- Saves money by using **one physical machine for many tasks**.





Cloud Computing (BCS071/BCS713)



☞ A startup has one powerful server. Using VMware vSphere or Microsoft Hyper-V:

VM 1 → runs a Web Server 🌐

VM 2 → runs a Database Server 🗄️

VM 3 → runs a File Server 📁

All on the same physical server, but acting like different independent servers.

6. Data Virtualization:

This brings data from different sources together in one place without needing to know where or how it's stored. It creates a unified view of the data, which can be accessed remotely via cloud services.

Example: Companies like Oracle and IBM offer solutions for this.





Cloud Computing (BCS071/BCS713)



❖ Levels of Virtualization

Virtualization can happen at different layers of a computer system. Each level handles virtualization differently and serves a unique purpose.

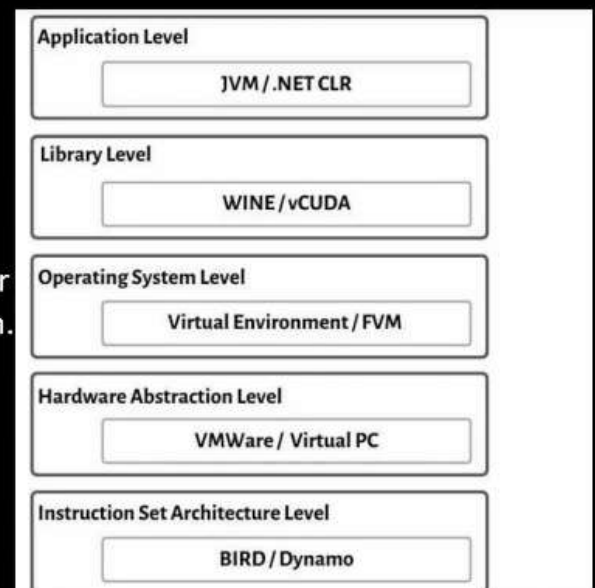
1. Application Level Virtualization

- Here, only the application (software) is virtualized.
- Applications run on a special virtual layer (like JVM or .NET CLR) instead of directly on the operating system.
- This makes apps independent of OS and hardware.

◆ Example:

JVM (Java Virtual Machine): Java programs can run on any operating system (Windows, Linux, Mac) because JVM makes them independent of OS.

.NET CLR (Common Language Runtime): Lets .NET apps





Cloud Computing (BCS071/BCS713)



run on different systems easily.

👉 **Easy Analogy:** It's like having a translator app 📱 —you speak one language, and it translates for everyone, no matter what country (OS) you're in.

2. Library Level Virtualization

- Instead of virtualizing the whole application, only the libraries (functions that apps use) are virtualized.
- Helps run software that was designed for one system on another system.

🔹 Example:

WINE: Lets you run Windows applications on Linux.

vCUDA: Virtualizes GPU libraries to run GPU-based apps on different systems.

👉 **Easy Analogy:** Think of it as using a dictionary 📖. Even if the system doesn't know your language, it looks up words in the library to understand you.





Cloud Computing (BCS071/BCS713)



3. Operating System Level Virtualization

- The entire operating system is virtualized.
- Multiple isolated environments (containers) can run on the same OS.
- Each environment acts like a separate system but shares the same OS kernel.

◆ Example:

- Virtual Environment / FVM (Feather-weight Virtual Machine): Creates separate isolated OS environments for applications.
- Modern example: Docker (containers).

👉 **Easy Analogy:** It's like a shared apartment building 🏢—everyone has their own room (environment), but the building (OS kernel) is the same.





Cloud Computing (BCS071/BCS713)



4. Hardware Abstraction Level Virtualization

The entire hardware (CPU, memory, devices) is virtualized.

A hypervisor is used to create virtual machines (VMs) that run their own OS.

Each VM behaves like a separate physical computer.

◆ Example:

VMware, Virtual PC, VirtualBox – used to run multiple operating systems on one physical machine.

👉 **Easy Analogy:** Like splitting one big house into separate flats 🏠 → 🏢. Each family (VM) thinks they have their own house, but really they share the same building.





Cloud Computing (BCS071/BCS713)



5. Instruction Set Architecture (ISA) Level Virtualization

Here, virtualization happens at the CPU instruction level.

It allows one type of CPU architecture to run programs designed for another.

Often used for compatibility and performance improvements.

🔑 Example:

BIRD, Dynamo: Tools that help applications run across different CPU architectures.

Apple's Rosetta 2 is a modern example—it lets Intel-based apps run on Apple Silicon (M1/M2).

👉 **Easy Analogy:** It's like a real-time language interpreter 🗣️📄👂 —converting every sentence (CPU instruction) into another language instantly so both sides understand.





Cloud Computing (BCS071/BCS713)



Q1. What are the different techniques used for hardware virtualization (AKTU 2022-23) (or Mechanism for virtualization)

Hardware Virtualization means splitting one physical machine (server/computer) into multiple Virtual Machines (VMs) using a hypervisor.

There are **three main** techniques used for hardware virtualization:

1. Full Virtualization

- In this method, the virtual machine (VM) acts exactly like a real physical computer.
- The guest OS (inside VM) doesn't know it's running on virtual hardware—it thinks it's running on a real physical machine.
- The hypervisor traps and translates instructions from the guest OS to the real hardware.

◆ Example:

VMware, Microsoft Virtual PC, VirtualBox use full virtualization.

👉 **Easy Analogy:** It's like a flight simulator 🎮. The pilot (guest OS) feels like they are flying a real plane, but it's just a simulation running on another system.





Cloud Computing (BCS071/BCS713)



2. Para-Virtualization

- In this method, the guest OS knows that it is running in a virtualized environment.
- The OS is modified to talk directly with the hypervisor instead of always thinking it's real hardware.
- Because of this, it's faster and more efficient than full virtualization (less translation work).

◆ Example:

Xen Hypervisor is a well-known para-virtualization software.

👉 **Easy Analogy:** Imagine you're sharing a car with a driver 🚗. Instead of pretending you're driving, you just tell the driver where to go directly—it's faster and more efficient.

3. Hardware-Assisted Virtualization

- Modern CPUs (Intel, AMD) have special instructions built into the processor to support virtualization.
- These CPU features (Intel VT-x, AMD-V) allow the hypervisor to run VMs more efficiently.
- It reduces the overhead of translating instructions because hardware it: virtualization.





Cloud Computing (BCS071/BCS713)



◆ Example:

KVM (Kernel-based Virtual Machine) uses hardware-assisted virtualization. VMware and Hyper-V also use Intel VT-x or AMD-V for performance boost.

👉 **Easy Analogy:** It's like having a car with autopilot 🚗. Instead of manually controlling everything, the car's system (hardware) helps you drive smoothly.

Q.2 What are major three components of virtualized environment (AKTU 2023-24)

1. Host Machine (Physical Hardware)

The actual physical computer (server, desktop, or data center machine).

Provides **CPU, memory, storage, and networking resources**.

All virtualization depends on this base hardware.

2. Hypervisor (Virtualization Layer)

Software that sits between the physical hardware (host) and the virtual machines (guests).

Responsible for **creating, managing, and isolating virtual machines (VMs)**.

Two types:





Cloud Computing (BCS071/BCS713)



- **Type 1 (Bare-metal):** Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, Xen).
- **Type 2 (Hosted):** Runs on top of an OS (e.g., VirtualBox, VMware Workstation).

3. Guest Machines (Virtual Machines / Virtual OS)

The virtual computers created by the hypervisor.

Each VM runs its own operating system and applications.

Completely isolated from each other but share underlying physical resources.





Cloud Computing (BCS071/BCS713)



Tools and Mechanism for Virtualization

Tools for Virtualization

These are the software/programs that help us create and manage virtual machines. Think of them as "apps" that allow you to run one or more computers inside your main computer.

Some popular tools:

- VMware – Most famous tool, lets you create and run multiple VMs easily.
- VirtualBox – Free tool from Oracle, very popular among students for learning.
- Hyper-V – Microsoft's virtualization tool, built inside Windows (Pro/Server editions).
- KVM (Kernel-based Virtual Machine) – Open-source tool, mainly used in Linux.
- Xen – Open-source tool, widely used in cloud platforms.
- Parallels – Popular on macOS to run Windows inside Mac.

Now let discuss these tools in detail →





Cloud Computing (BCS071/BCS713)



◆ 1. VMware

Think of it as the most popular professional tool for virtualization. Companies use it a lot because it is stable, powerful, and easy to manage. You can install many virtual machines (VMs) on one computer/server.

☞ **Example:** A company runs Windows, Linux, and even older systems together using VMware.

◆ 2. VirtualBox

A free tool made by Oracle.

Very famous among students and learners because it's simple.

Works on Windows, Linux, and Mac.

☞ **Example:** You can install Linux on your Windows laptop using VirtualBox for practice.

◆ 3. Hyper-V

Microsoft's own virtualization tool.

Already comes built-in with Windows Pro, Enterprise, and Server editions.

Best for people who use Windows servers.

☞ **Example:** A company using Windows Server can create VMs with Hyper-V without extra software.





Cloud Computing (BCS071/BCS713)



◆ 4. KVM (Kernel-based Virtual Machine)

An open-source tool that is part of Linux.

Works very well for Linux servers.

Mostly used in data centers and cloud (like Google Cloud, Amazon).

☞ **Example:** A Linux server in a data center uses KVM to run hundreds of VMs.

◆ 5. Xen

Another open-source tool, widely used in cloud computing.

Many big companies (like AWS in early days) used Xen.

Very strong for large-scale virtualization.

☞ **Example:** A cloud provider runs thousands of customer servers on Xen.

◆ 6. Parallels

Specially made for Mac users.

Lets you run Windows inside a Mac smoothly.

Great for people who need both macOS and Windows together.

☞ **Example:** A software developer with a Mac runs Windows apps using Parallels.





Cloud Computing (BCS071/BCS713)



1. CPU Virtualization

- ☞ CPU (Central Processing Unit) is the brain of the computer.
In virtualization, the real CPU of the physical server is shared among many virtual machines (VMs).
- A special software called Hypervisor divides the CPU power into parts and gives each VM its share.
- Each VM feels like it has its own CPU, but actually, they are all sharing the same physical CPU.

★ Example:

If your server has 8 cores → Hypervisor can give 2 cores to VM1, 4 cores to VM2, and 2 cores to VM3.



Cloud Computing (BCS071/BCS713)



◆ Key Mechanisms of CPU Virtualization

1. Hypervisor (VMM – Virtual Machine Monitor):

- This is the main software that manages CPU virtualization.
- It **sits between hardware and VMs**, giving each VM a share of the CPU.

Types:

- ✓ **Type 1 (Bare-metal)**: Runs directly on hardware (e.g., VMware ESXi, Hyper-V).
- ✓ **Type 2 (Hosted)**: Runs on top of an OS (e.g., VirtualBox).

2. CPU Scheduling:

- Hypervisor decides **which VM gets CPU time** and for how long.
- Works like a **time-sharing system**:
 - VM1 runs for a few milliseconds → then VM2 → then VM3 → and so on.
- This switching happens so fast that each VM **feels like it has its own CPU**.



Cloud Computing (BCS071/BCS713)



3. vCPU Mapping:

- Each VM is assigned Virtual CPUs (vCPUs).
- The hypervisor maps vCPUs to physical CPU cores in the system.
- Example:
 - A server has 4 physical cores.
 - You can create 8 VMs with 1 vCPU each → Hypervisor distributes CPU time among them.

◆ Benefits of CPU Virtualization

✓ Resource Sharing:

Many VMs can use the same physical CPU → better utilization.

✓ Isolation:

Each VM is independent. Even if one VM uses CPU heavily, others can still run.

✓ Flexibility:

You can allocate more or fewer vCPUs to each VM depending on need.

✓ Cost Saving:



Cloud Computing (BCS071/BCS713)



2. Memory Virtualization

- ☞ Memory (RAM) is like a workspace where programs run.
In virtualization, the physical RAM is divided and managed for different VMs.
- Each VM thinks it has its own private RAM.
- Hypervisor maps virtual memory (fake memory seen by VMs) to physical memory (real RAM in the server).
- If RAM is less, hypervisor can even use techniques like swapping (store temporary data in disk).

✈ Example:

If server has 16 GB RAM → Hypervisor can give 4 GB to VM1, 8 GB to VM2, and 4 GB to VM3.

🔑 How it works:

- ✓ Hypervisor translates VM's memory requests into physical addresses.
- ✓ Prevents one VM from accessing another VM's memory (security).
- ✓ Can optimize by using "memory overcommit" (assign more virtual memory than actual RAM, since not all VMs use full memory at the same time).



Cloud Computing (BCS071/BCS713)



3. I/O Device Virtualization

- ☞ I/O (Input/Output) devices = Hard disk, Keyboard, Mouse, Network card, USB devices, etc.
- In virtualization, these physical devices are shared among VMs.
- Each VM sees its own virtual hard disk, virtual network card, etc.
- But in reality, they are mapped to the same physical devices by the hypervisor.

✦ Example:

VM1 stores data in its "C Drive" → Hypervisor maps it to part of the physical hard disk.

VM2 uses its "Network card" → Hypervisor routes it to the same physical network port.

🔑 How it works:

- Hypervisor creates virtual devices for each VM.
- It translates VM requests (like read/write to disk) into real hardware operations.
- Multiple VMs can share a single network card, printer, or disk without conflict.



Cloud Computing (BCS071/BCS713)



Virtualization Support for Disaster Recovery (DR)

◆ What is Disaster Recovery (DR)?

- Disaster Recovery means bringing back your IT systems, data, and applications after a big failure (like hardware crash, cyber attack, fire, flood, or power outage).
- Goal → Reduce downtime (system not working time) and prevent data loss.

◆ How Virtualization Helps in Disaster Recovery

Virtualization makes DR faster, cheaper, and easier compared to traditional methods.

1. Hardware Independence

- Normally, if a server fails, you need the same kind of hardware to restore it.
- With virtualization, a virtual machine (VM) can run on any hardware because VMs are not tied to physical machines.
 - ✓ Benefit: You don't need to buy identical servers for backup.



Cloud Computing (BCS071/BCS713)



2. Snapshot and Backup

- Virtualization tools let you take snapshots (a frozen copy of VM at a point in time).
- These snapshots can be stored in another location or cloud.
 - ✓ Benefit: If the main system crashes, you can quickly restore the snapshot VM.

3. Live Migration

- Some virtualization platforms (like VMware vMotion, Hyper-V Live Migration) allow moving a running VM from one physical server to another without downtime.
 - ✓ Benefit: If one machine is failing, you can move VMs to a safe machine instantly.

4. Replication

- VMs can be replicated (copied in real-time) to another data center or cloud.
- If the main site is down, you can start VMs in the backup site.
 - ✓ Benefit: Zero or minimal data loss.

5. Faster Recovery Time

Instead of reinstalling OS, applications, and configuring everything, you just boot up the VM backup.



Cloud Computing (BCS071/BCS713)



6. Cost Savings

- No need for a full set of extra physical servers just for backup.
 - Multiple VM backups can run on one physical backup server.
- ✓ Benefit: Saves money 💰.



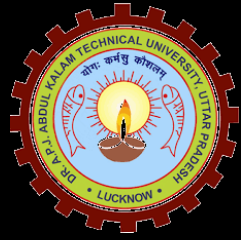
Cloud Computing (BCS071/BCS713)



Thank You....



Download Notes : <https://rzp.io/rzp/tkFQMuOZ>



Cloud Computing (BCS071/BCS713)



Unit – 3 Cloud Architecture, Services & Storage

III

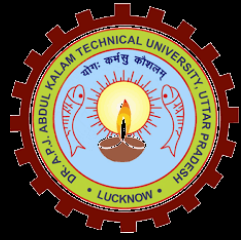
Cloud Architecture, Services And Storage: Layered Cloud Architecture Design – NIST Cloud Computing Reference Architecture – Public, Private and Hybrid Clouds – IaaS – PaaS – SaaS – Architectural Design Challenges – Cloud Storage – Storage-as-a-Service – Advantages of Cloud Storage – Cloud Storage Providers – S3.



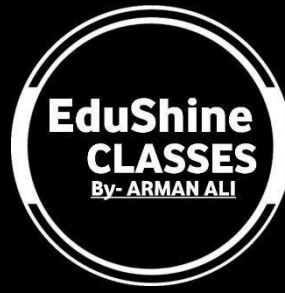
EduShine Classes – Arman Ali



Download Notes : <https://rzp.io/rzp/tkFQMuOZ>



Cloud Computing (BCS071/BCS713)



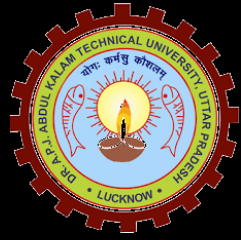
⚡ What is Cloud Architecture?

Cloud architecture means the **design/structure of cloud computing**.

Just like a house has **foundation, walls, rooms, and roof**, cloud has **different layers** that work together.

These layers make it easy to:

- Store data
- Run applications
- Provide services to users
- Cloud architecture is a key element of building in the cloud. It refers to the layout and connects all the necessary components and technologies required for [cloud computing](#).
- Cloud architecture dictates how components are integrated so that you can pool, share, and scale resources over a network. Think of it as a building blueprint for running and deploying applications in cloud environments.



Cloud Computing (BCS071/BCS713)



☁ Layered Cloud Architecture Design

Cloud is built in **layers**. Each layer has its own job, and together they make cloud computing possible.

Think of it like a **shopping mall** 🏢: basement (foundation), shops (services), managers (control), and utilities (electricity, lifts, AC).

1. Physical Layer (Foundation Layer)

This is the **hardware** part of the cloud.

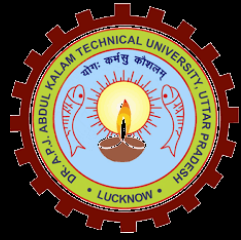
It includes **servers, hard disks, network cables, switches, routers, data centers**.

It is the **backbone** of the cloud.

👉 Example:

The huge **Amazon Data Centers** where thousands of servers are stored.

Like the **basement** of a mall that holds the structure.



Cloud Computing (BCS071/BCS713)



2. Virtual Layer (Magic Layer ⚡)

The physical hardware is divided into **virtual machines (VMs)** using **virtualization software**. This layer makes **one big server act like many small servers**.

Helps in **scalability** (increase capacity anytime) and **efficiency** (use all resources properly).

👉 Example:

AWS EC2 instances → one server can run many virtual computers.

Like in a mall, a **big hall is divided into small shops** so many people can use it.

3. Control Layer (Manager Layer 👤👜)

This layer **manages and monitors everything**.

It decides:

- How much CPU, memory, storage each user gets.
- Security (who can access what).
- Performance monitoring (keep system running smoothly).

Basically, it **allocates resources** and **ensures rules are followed**.



Cloud Computing (BCS071/BCS713)



➡ Example:

Kubernetes, OpenStack, VMware vSphere – they manage virtual resources.

Like in a mall, **mall management** assigns shops, controls electricity, water, lifts, and ensures security.

4. Service Layer (User Layer 🎁)

This is the layer that **users interact with**.

It provides services in **three categories**:

a) IaaS (Infrastructure-as-a-Service)

Raw computing resources: servers, storage, and networking.

You rent infrastructure instead of buying hardware.

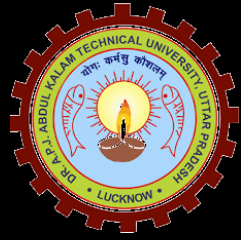
➡ **Example:** AWS EC2, Google Compute Engine.

b) PaaS (Platform-as-a-Service)

Provides a platform to **build and deploy apps**.

You don't manage servers, just focus on your app.

➡ **Example:** Google App Engine, Microsoft Azure App Service.



Cloud Computing (BCS071/BCS713)



c) SaaS (Software-as-a-Service)

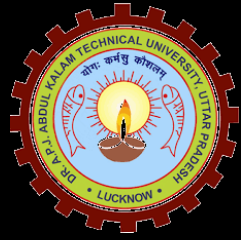
Ready-to-use software delivered over the internet.

You just log in and use.

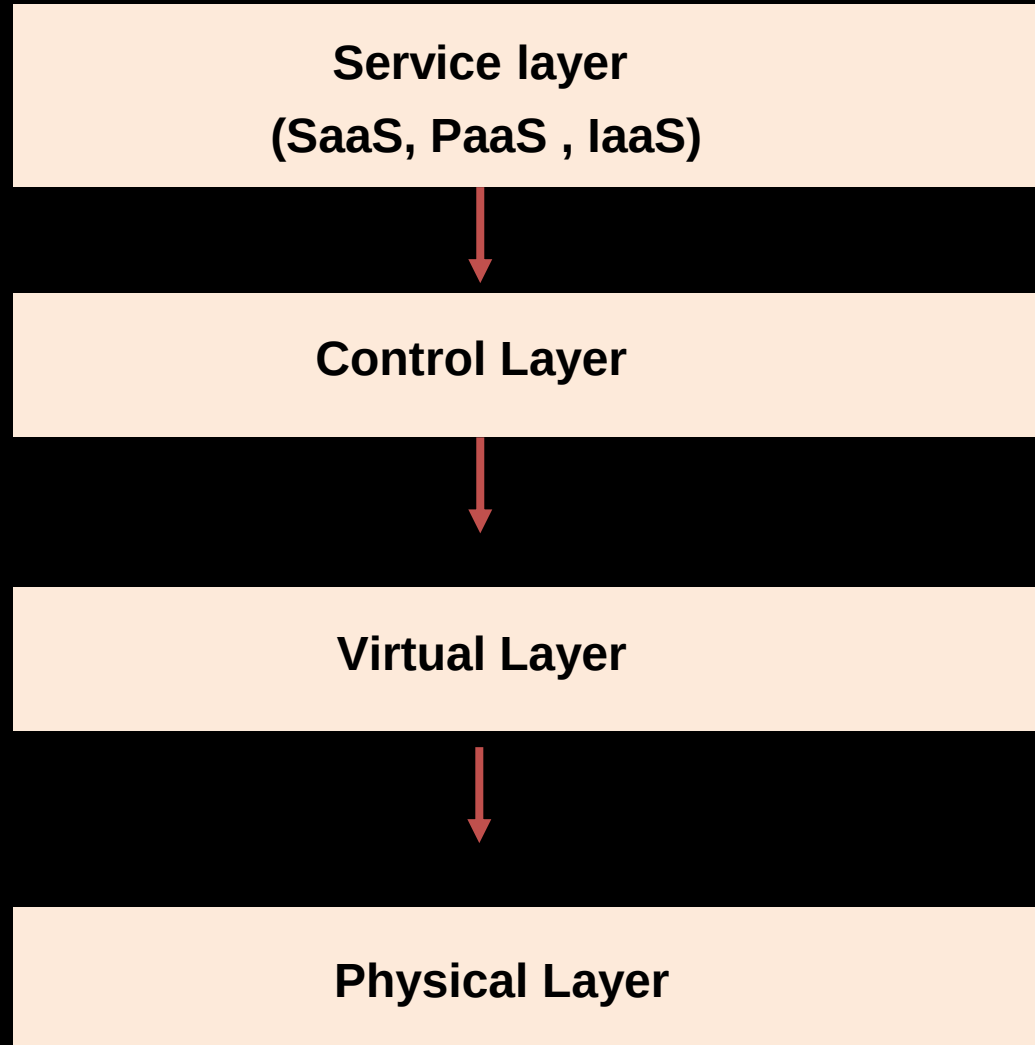
👉 **Example:** Gmail, Zoom, Google Docs.

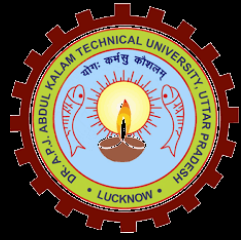
💡 Like in a mall →

- IaaS = Mall provides empty shop space.
- PaaS = Mall provides shop + shelves + electricity.
- SaaS = Mall provides a fully ready shop (you can start business immediately).

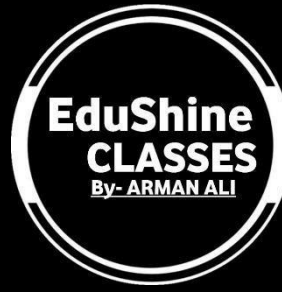


Cloud Computing (BCS071/BCS713)





Cloud Computing (BCS071/BCS713)

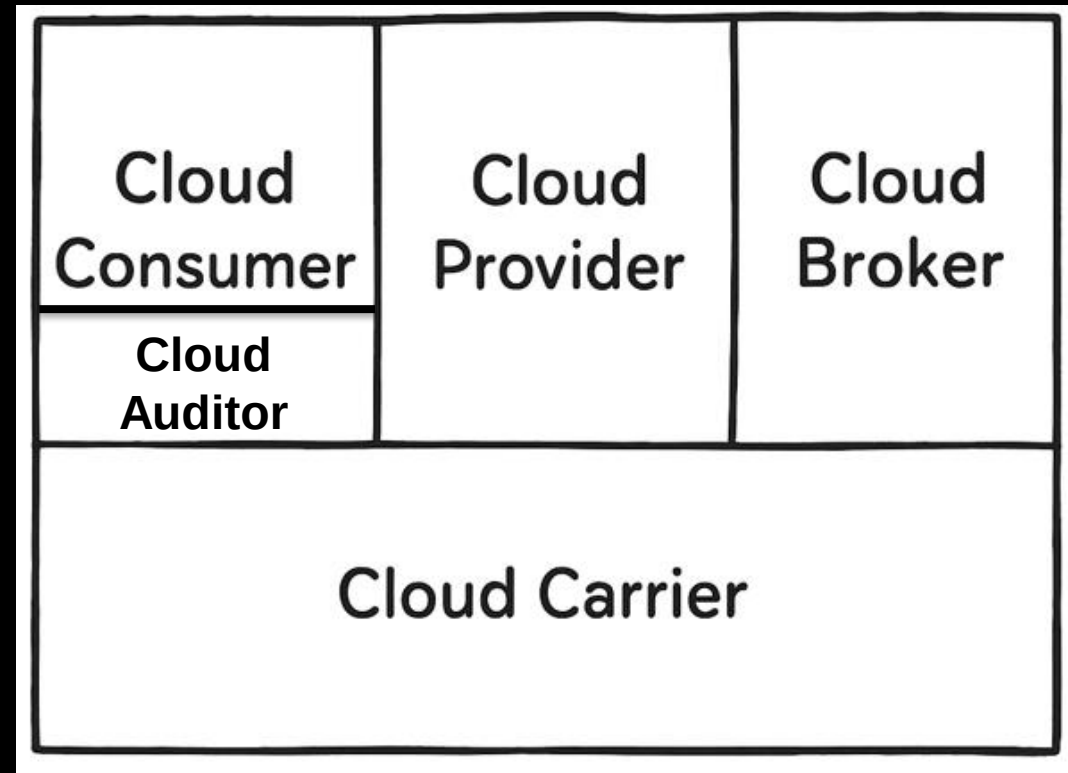


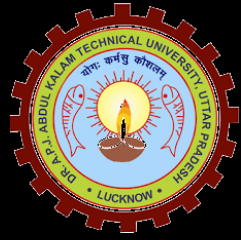
☛ NIST Cloud Computing Reference Architecture

☞ NIST (National Institute of Standards and Technology) created a **standard model** that explains the **main roles/players in cloud computing**.

It helps us understand **who does what** in cloud.

There are **5 main roles**:





Cloud Computing (BCS071/BCS713)



1. Cloud Consumer (User 🧑)

The person or organization that **uses the cloud services**.

They can use SaaS, PaaS, or IaaS depending on their needs.

👉 Example:

You using **Google Drive (SaaS)**.

A developer using **Heroku (PaaS)**.

A company using **AWS EC2 servers (IaaS)**.

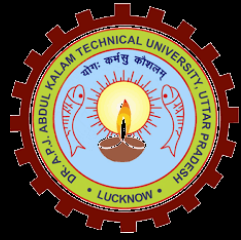
2. Cloud Provider (Owner 🏢)

The company that **owns and manages the cloud infrastructure**.

They provide computing, storage, and services to consumers.

👉 Example:

Amazon AWS, Microsoft Azure, Google Cloud are cloud providers.



Cloud Computing (BCS071/BCS713)



3. Cloud Broker (Middleman)

Acts as a **mediator** between the cloud provider and consumer.

Helps consumers choose the best service, manages multiple providers, and may add extra services like billing or security.

☞ Example:

A company using **RightScale** or **CloudCheckr** to manage AWS + Azure together.
Like a **travel agent** who books flights from different airlines for you.

4. Cloud Auditor (Inspector Q)

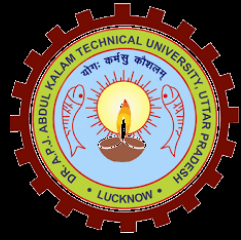
An **independent examiner** who checks the cloud services for:

- Security
- Privacy
- Compliance (rules, regulations)

Ensures the provider is trustworthy.

☞ Example:

Third-party security auditors that certify AWS for GDPR, ISO standards, etc.



Cloud Computing (BCS071/BCS713)



5. Cloud Carrier (Transporter 🚚)

The **network provider** that delivers cloud services from provider to consumer.
Basically, the **Internet or telecom company**.

👉 Example:

Airtel, Jio, AT&T, Verizon → provide the internet connection that carries cloud services.

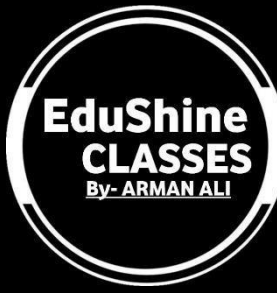
★ Quick Analogy (Easy to Remember)

Think of Cloud as an **Online Shopping Mall** 🛒

- ✓ **Cloud Provider** = The Mall Owner (runs shops).
- ✓ **Cloud Consumer** = The Customer (buys products).
- ✓ **Cloud Broker** = The Shopping Assistant (helps you find the best deal).
- ✓ **Cloud Auditor** = Quality Inspector (checks product quality).
- ✓ **Cloud Carrier** = The Delivery Service (brings products to you).



Cloud Computing (BCS071/BCS713)



☁ Architectural Design Challenges in Cloud Computing

When we design cloud systems, there are some **problems (challenges)** we must handle.

1. Security and Privacy 🔒

Data is stored on remote servers (not on your own computer).

Risk of **hacking, unauthorized access, and data leaks.**

👉 **Example:** If a hacker gets into AWS, they can steal sensitive company data.

2. Performance and Reliability ⚡

Cloud services depend on the **internet.**

If the internet is slow or the provider's server crashes, performance suffers.

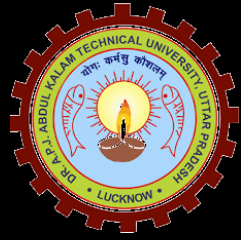
👉 **Example:** A website hosted on cloud may go down if servers fail.

3. Portability and Interoperability ↻

Portability → Can I **move my app/data** from AWS to Google Cloud easily?

Interoperability → Can **different cloud services work together** smoothly?

👉 **Example:** Migrating from AWS S3 to Azure Storage is not very simple.



Cloud Computing (BCS071/BCS713)



4. Cost Management 💰

Cloud seems cheap at first, but costs can grow quickly if not monitored.

Pay-per-use model may lead to **unexpected high bills**.

👉 **Example:** A startup forgets to turn off unused AWS servers and gets a huge bill.

5. Scalability and Elasticity 📈

Cloud should **scale up or down automatically** based on demand.

Designing apps that can handle sudden spikes in traffic is a challenge.

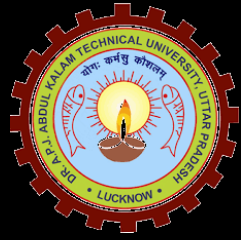
👉 **Example:** An online shopping site crashes during a festival sale because it couldn't scale fast enough.

6. Availability and Disaster Recovery 🌐

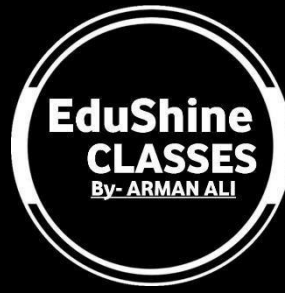
Cloud must be available **24/7**.

Need backup and recovery plans for **natural disasters or cyber-attacks**.

👉 **Example:** Google Cloud provides multi-region backups to prevent data loss.



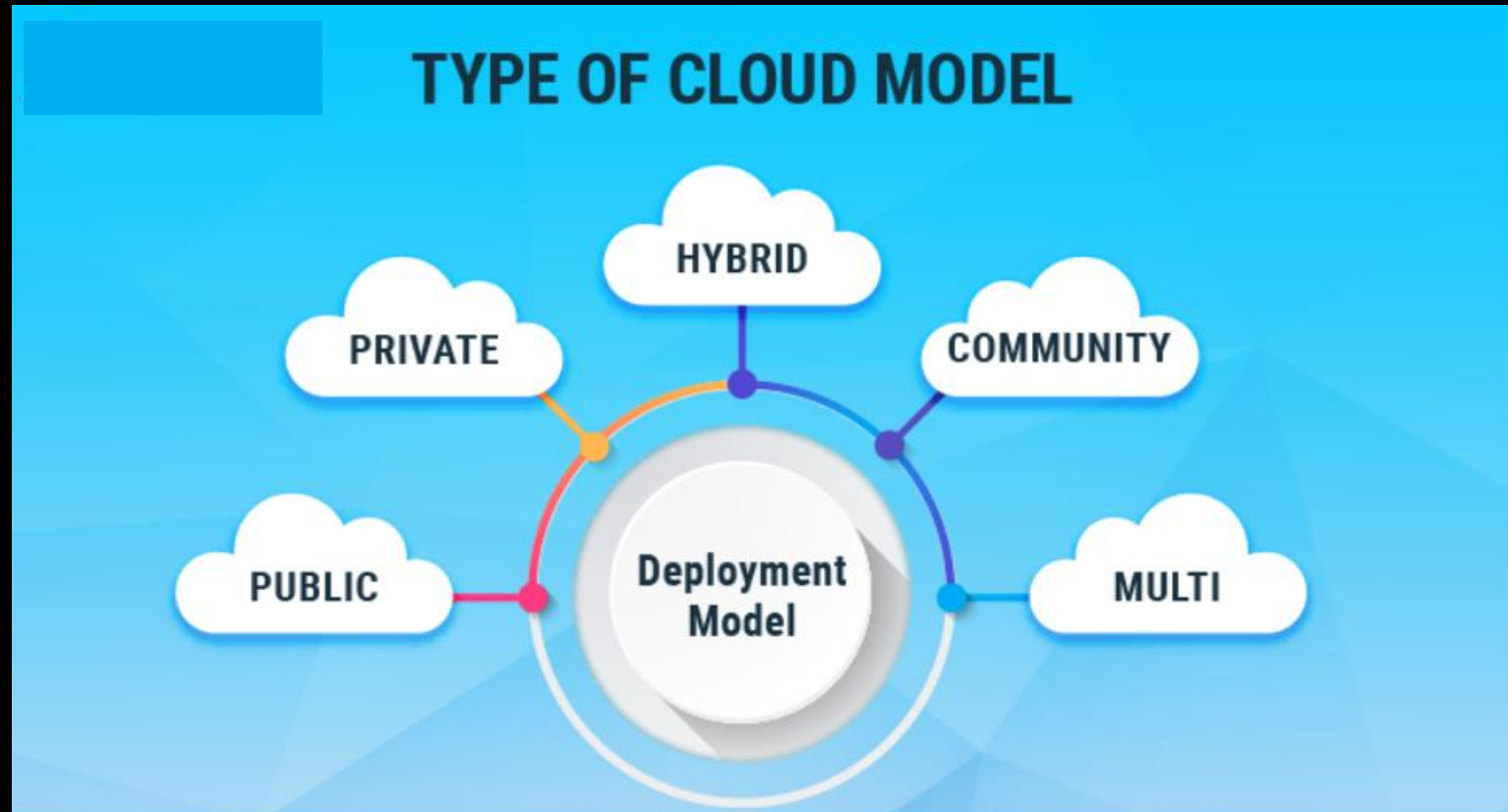
Cloud Computing (BCS071/BCS713)

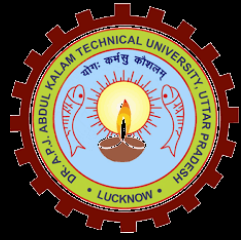


☁️ Cloud Deployment Models

Deployment model = How the cloud is set up and who can use it.

There are 4 main models:



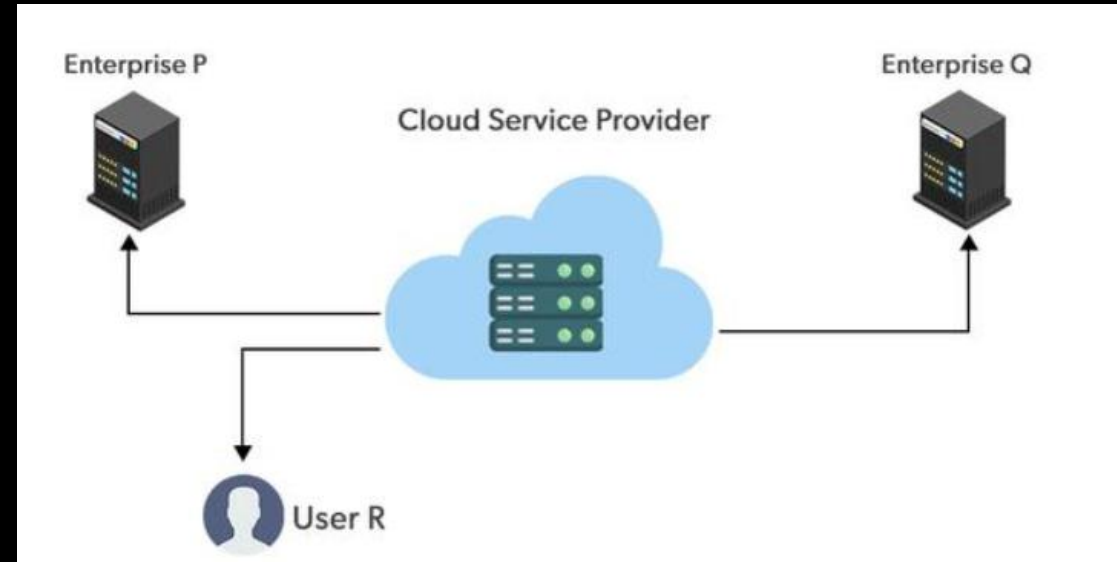


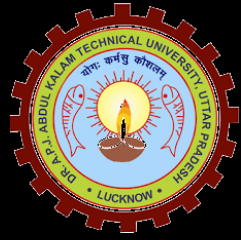
Cloud Computing (BCS071/BCS713)



1. Public Cloud

- Public cloud is a type of cloud **open for everyone** on the internet.
- The cloud is owned and managed by a **Cloud Service Provider (CSP)** like AWS, Azure, Google Cloud.
- Users (individuals, companies) **rent resources** like servers, storage, or applications instead of owning them.
- Payment is usually **Pay-as-you-go** (like electricity bill).





Cloud Computing (BCS071/BCS713)

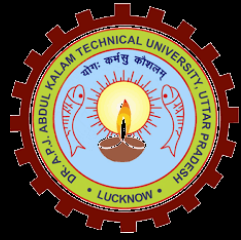


❖ How It Works

- **Cloud Service Provider (CSP)** → owns the servers, data centers, and services.
- **Enterprises P and Q** → different companies that both use the same public cloud.
- **User R** → an end-user who accesses applications hosted on the cloud.
- ☞ All of them share the **same cloud infrastructure** but their data remains **separate and secure**.

❖ Features of Public Cloud

- ✓ **Shared Infrastructure** → Many users share the same physical hardware.
- ✓ **On-demand resources** → Scale up or down anytime.
- ✓ **Low cost** → No need to buy hardware.
- ✓ **Accessible Anywhere** → Available via the internet.



Cloud Computing (BCS071/BCS713)



Advantages ✓

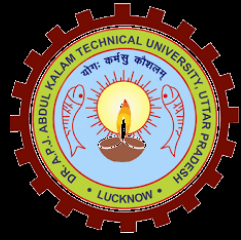
- ✓ **Cost-effective** → No need to invest in hardware.
- ✓ **Scalability** → Increase resources easily.
- ✓ **Reliability** → Data stored across multiple servers.
- ✓ **Maintenance-free** → Provider takes care of updates and security.

Disadvantages ✗

- **Security concerns** → Since it's shared, there's more risk.
- **Less control** → You can't control hardware directly.
- **Internet dependency** → Needs good internet.

Examples

- **AWS (Amazon Web Services)**
- **Microsoft Azure**
- **Google Cloud Platform**
- **IBM Cloud**



Cloud Computing (BCS071/BCS713)



2. Private Cloud

A **private cloud** is used by **only one organization**.

It is **not shared** with others like public cloud.

Can be hosted:

- **On-premises** (inside company's own data center).
- **By a third-party vendor** (but still dedicated only to that organization).

It gives **more control and security**, but it's **costly**.

❖ How It Works

The organization either **owns the servers** or **rents a private cloud** from a provider.

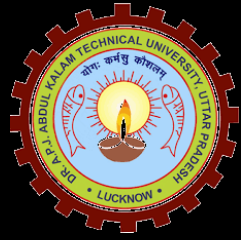
Only employees/customers of that organization can access it.

Data is **completely isolated** from other companies.

☞ Example:

A bank creating its own private cloud for customer financial data.

Government using private cloud for confidential records.



Cloud Computing (BCS071/BCS713)



❖ Features of Private Cloud

- ✓ **Exclusive use** → Only one organization uses it.
- ✓ **High Security** → Sensitive data is protected.
- ✓ **Customization** → Organization controls hardware, software, and policies.
- ✓ **Better performance** → No sharing with others.

Advantages ✓

- ✓ **High security & privacy** (best for sensitive data).
- ✓ **Full control** over infrastructure.
- ✓ **Customization** as per company needs.
- ✓ **Better reliability** since not shared.

Disadvantages ✗

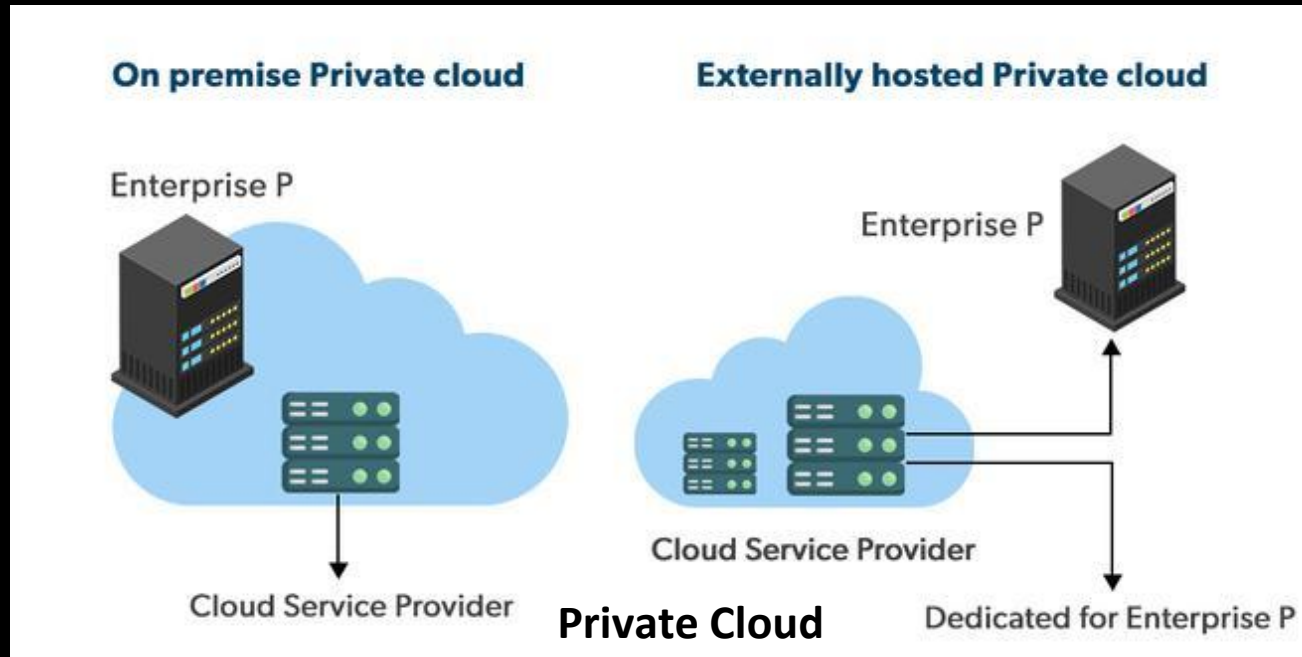
- ✓ **Expensive** → Needs more investment in servers & staff.
- ✓ **Maintenance responsibility** (if on-premises).
- ✓ **Less scalability** compared to public cloud.

Examples

- VMware vCloud
- OpenStack private cloud
- Microsoft Azure Stack (private version of Azure)

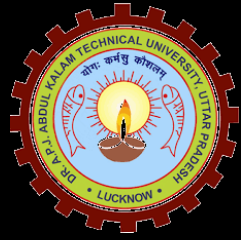


Cloud Computing (BCS071/BCS713)



3. Hybrid Cloud

- A **hybrid cloud** is a combination of **public cloud + private cloud**.
- Some data/applications run on the **private cloud** (secure, sensitive), while others run on the **public cloud** (scalable, cost-effective).
- It gives the **best of both worlds** → security + flexibility.



Cloud Computing (BCS071/BCS713)



How It Works

- An organization uses **private cloud** for sensitive data (like customer info, financial records).
- At the same time, it uses **public cloud** for less sensitive tasks (like hosting apps, big data analysis).
- Both clouds are connected and **work together**.

👉 Example:

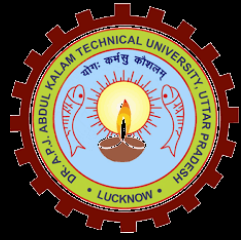
A hospital:

Stores patient data in **private cloud** (secure).

Uses **public cloud** for running healthcare apps and research.

Features of Hybrid Cloud

- **Mix of private + public.**
- **Flexible** → Can move workloads between clouds.
- **Cost efficient** → Keep critical apps private, non-critical apps public.
- **Scalable** → Public cloud handles sudden spikes in traffic.



Cloud Computing (BCS071/BCS713)



Advantages ✓

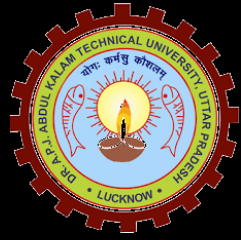
- ✓ **Security + Privacy** for sensitive data (private).
- ✓ **Cost savings** by using public cloud for normal tasks.
- ✓ **Flexibility** to run apps where they fit best.
- ✓ **Scalability** → Handle sudden demand using public cloud.

Disadvantages ✗

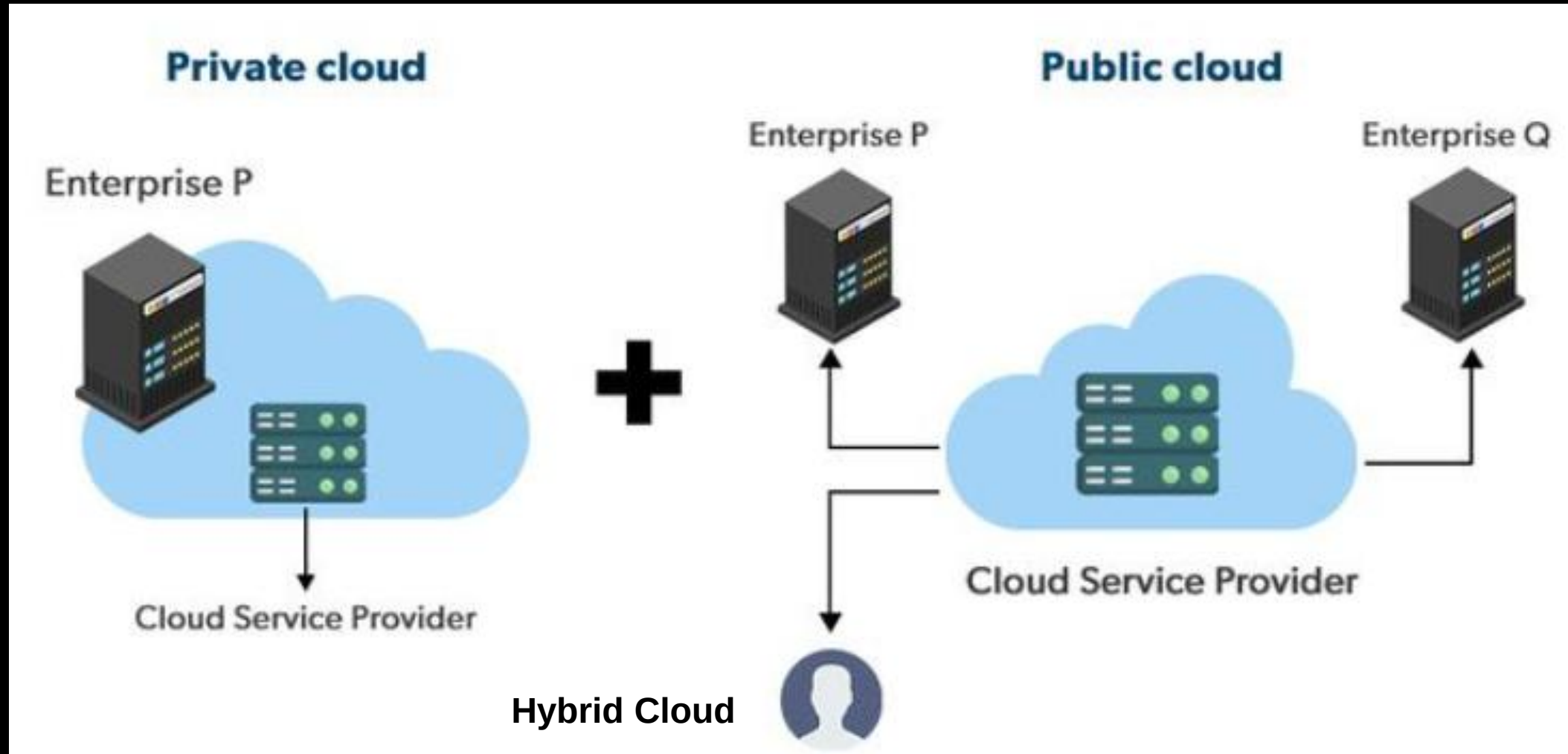
- ✓ **Complex setup** (needs good planning).
- ✓ **Integration issues** between private & public cloud.
- ✓ **Higher management cost** compared to using only one cloud.

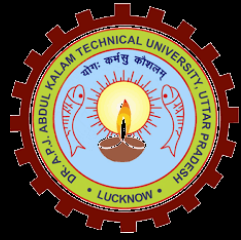
Examples :

- **Microsoft Azure Hybrid Cloud**
- **AWS Outposts** (connects AWS with private data centers)
- **Google Anthos** (manages apps across private + public cloud)



Cloud Computing (BCS071/BCS713)





Cloud Computing (BCS071/BCS713)



4. Community Cloud

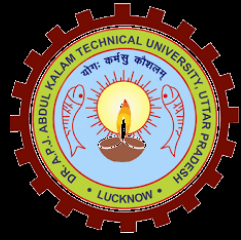
- A **community cloud** is a cloud environment **shared by multiple organizations** that have **similar needs or goals**.
- It is **not public** (open for everyone) and **not private** (used by only one org).
- Instead, it is **shared by a group of organizations** (community).

How It Works

- A group of organizations with **similar requirements** (like security, compliance, or industry rules) **come together**.
- They **share the same cloud infrastructure**.
- The cost and resources are **divided among them**.

☞ Example:

- Several **banks** can use a **community cloud** designed for financial services (with strong security and compliance).
- Several **hospitals** can share a cloud to **store and process medical records** safely.



Cloud Computing (BCS071/BCS713)



❖ Features of Community Cloud

- ✓ **Shared infrastructure** → used by multiple organizations.
- ✓ **Common interest/purpose** → healthcare, government, finance, education.
- ✓ **Cost sharing** → cheaper than private cloud.
- ✓ **More secure than public cloud** (since not open for everyone).

Advantages ✓

- ✓ **Cost effective** → cost shared among organizations.
- ✓ **Security & compliance** → follows industry rules.
- ✓ **Collaboration** → easy data & resource sharing between organizations.
- ✓ **Customizable** → built for a specific community's needs.

Disadvantages ✗

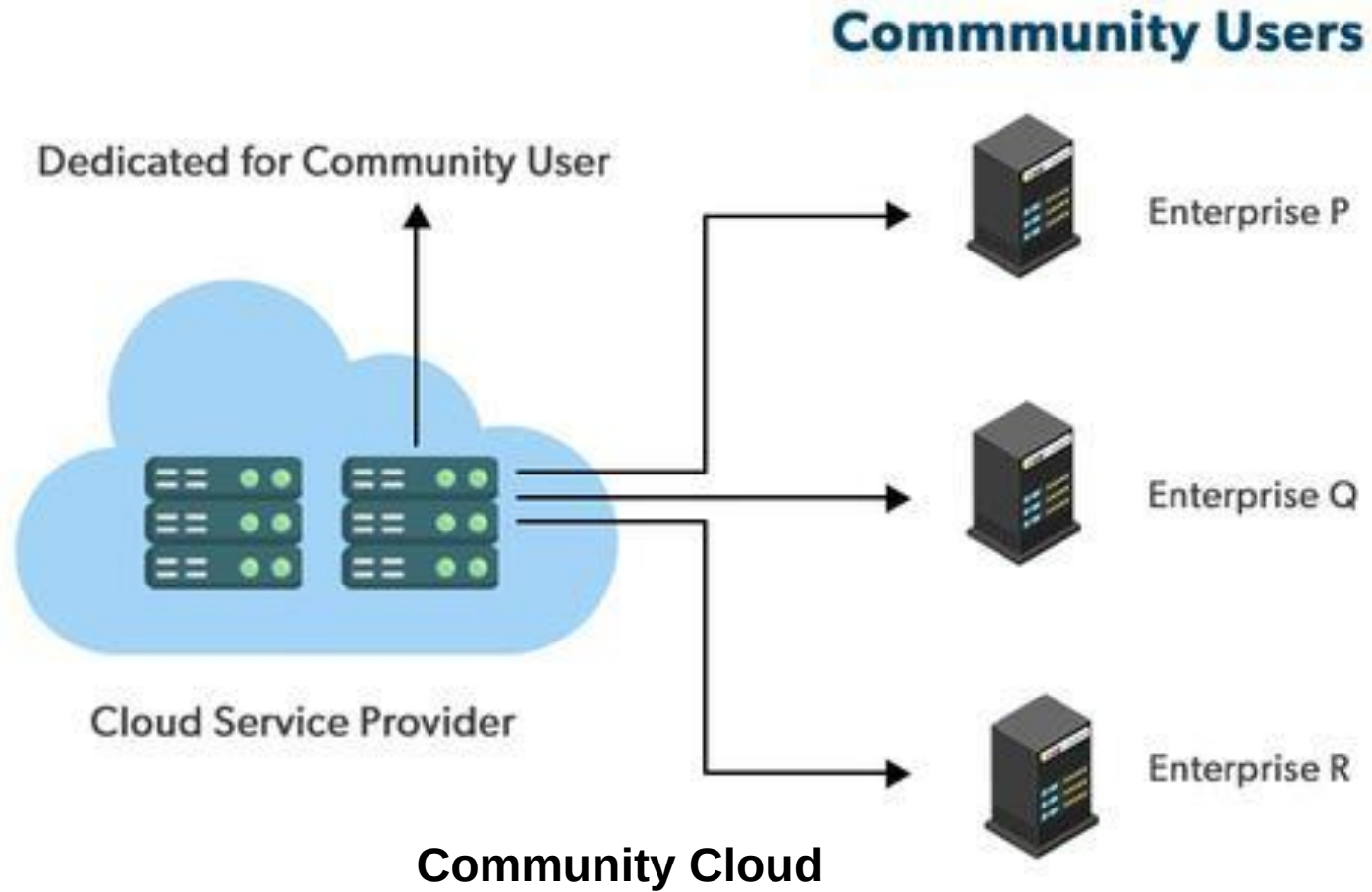
- **Costlier than public cloud** (since it's not open for all).
- **Limited scalability** compared to public cloud.
- **Management complexity** (multiple organizations involved).

Examples

- **Government Community Cloud** (different government departments sharing one cloud).
- **Healthcare Community Cloud** (multiple hospitals sharing patient data securely).
- **Banking/Financial Community Cloud** (several banks following same rules like RBI, PCI DSS).



Cloud Computing (BCS071/BCS713)





Cloud Computing (BCS071/BCS713)



5. Multi-Cloud

Multi-cloud means using **services from more than one cloud provider** at the same time. Instead of depending on only **one cloud provider** (like AWS, Azure, or Google Cloud), an organization uses **two or more** for different purposes.

👉 Example:

A company may use **AWS** for data storage, **Google Cloud** for machine learning, and **Microsoft Azure** for running apps.

Why Multi-Cloud?

- To **avoid vendor lock-in** (not being stuck with only one provider).
- To **use best features** from different providers.
- To **increase reliability** → if one provider fails, another can still serve.
- To **reduce cost** → choose cheaper services for specific tasks.



Cloud Computing (BCS071/BCS713)



Advantages ✓

- ✓ **No vendor lock-in** → not dependent on one provider.
- ✓ **High reliability** → backup from another cloud if one fails.
- ✓ **Best services** → pick the strongest service from each provider.
- ✓ **Cost optimization** → choose cheaper options for different tasks.

Disadvantages ✗

- **Complex management** → managing multiple providers is difficult.
- **Higher skill required** → IT team must know different platforms.
- **Security challenges** → need strong security across all providers.

★ Real-life Analogy

Multi-cloud = **Using multiple online stores** 🛒

You buy clothes from **Myntra**, electronics from **Amazon**, and groceries from **BigBasket**.
You don't depend on **just one store**, you pick the **best one for each need**.



Cloud Computing (BCS071/BCS713)



☁ Principles for Designing Cloud Models :

1. Principles for Designing Public Cloud 🌐

Public cloud is shared by many users → so the design must focus on **security, scalability, and cost efficiency**.

✓ Key Principles:

- ✓ **Multi-tenancy** → Separate data of different users securely.
- ✓ **Elasticity & Scalability** → Handle sudden increase/decrease in demand.
- ✓ **Pay-per-use** → Implement a proper billing system.
- ✓ **High Availability** → Services should run 24/7 across multiple regions.
- ✓ **Security & Privacy** → Strong encryption and access control.
- ✓ **Standardization** → Use common APIs and interfaces for easy access.

☞ **Example:** AWS S3 stores files of millions of users but ensures **data isolation + scalability**.



Cloud Computing (BCS071/BCS713)



2. Principles for Designing Private Cloud 🗝️

Private cloud is for one organization → focus on **control, customization, and security**.

✓ Key Principles:

- ✓ **Security-first design** → Strict access control, firewalls, monitoring.
- ✓ **Customization** → Tailored to organization's needs.
- ✓ **Resource Optimization** → Use virtualization to avoid waste.
- ✓ **Compliance** → Follow legal and regulatory requirements (e.g., healthcare, banking).
- ✓ **Integration** → Should connect smoothly with on-premise systems.
- ✓ **Automation** → Automate provisioning, scaling, and monitoring.

👉 **Example:** A bank's private cloud ensures **data security + regulatory compliance** (RBI rules, PCI-DSS).



Cloud Computing (BCS071/BCS713)



3. Principles for Designing Hybrid Cloud ☁

Hybrid cloud combines private + public cloud → focus on balance, flexibility, and seamless movement of data/apps.

✓ Key Principles:

Workload Distribution → Place sensitive data in private, normal apps in public.

Interoperability → Ensure smooth communication between public & private cloud.

Data Portability → Easy to move data between clouds.

Security Consistency → Apply same security rules in both private & public.

Scalability & Flexibility → Public cloud handles extra demand.

Disaster Recovery → Backup in multiple environments.

🔗 **Example:** A hospital stores patient data in private cloud but uses public cloud for analytics and apps.



Cloud Computing (BCS071/BCS713)



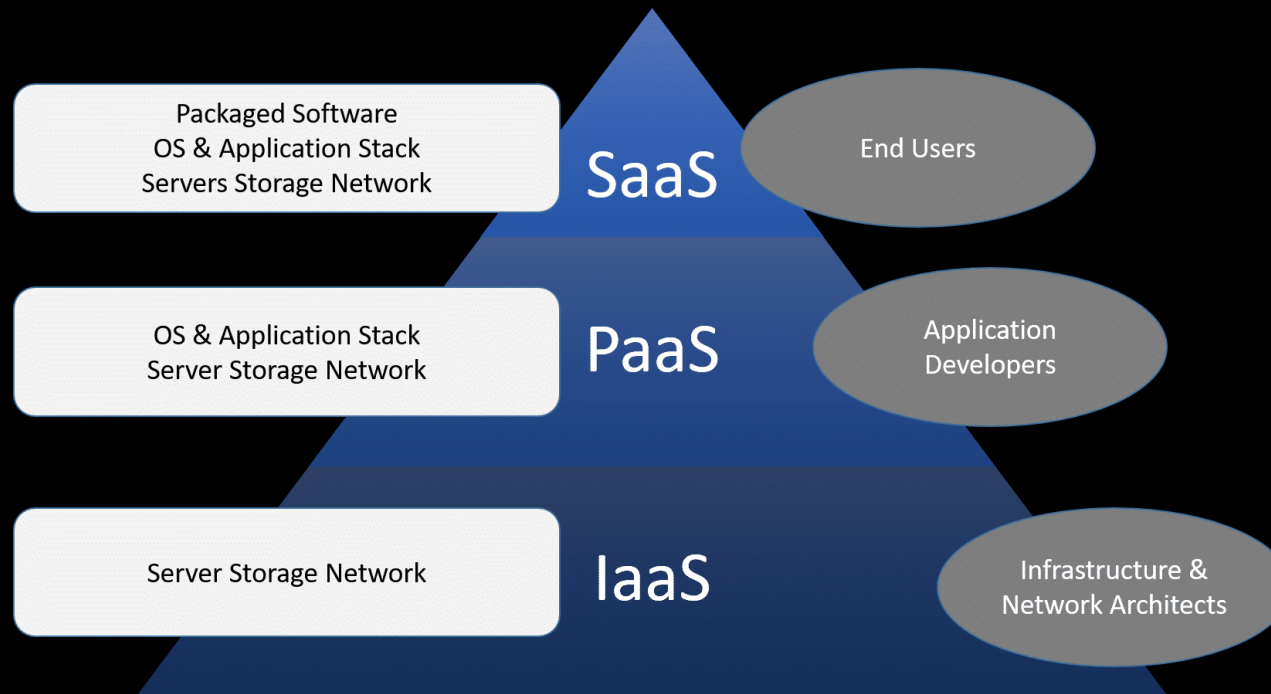
☁️ The 3 Cloud Service Models :

Think of them as different levels of “how much the provider does for you.”

IaaS = You rent raw infrastructure (virtual machines, networks, storage).

PaaS = You get a ready platform to run your code (runtime, OS, autoscaling handled).

SaaS = You use finished software (just log in and use).





Cloud Computing (BCS071/BCS713)



1) IaaS — Infrastructure as a Service

“You get virtual computers and disks. You install and manage everything on top.”

☞ **You get raw materials.**

Cloud gives you: computer (VM), storage, network.

You must install OS, software, and your app.

✈ **Example:** AWS EC2 (you rent a computer in the cloud).

🔑 **Use when:** You want **full control** and can manage servers yourself.

❖ **You manage:**

- Operating system (Windows/Linux), updates & patches
- Runtime (.NET/Java/Node), web servers, DB engines you install
- Your application code & data
- Security settings (firewalls/NSGs, IAM roles), backups you choose to take
- Scaling rules (unless you add managed services)



Cloud Computing (BCS071/BCS713)



❖ Cloud provider manages:

- Data centers (power, cooling, physical security)
- Physical servers, storage hardware, network cables/switches
- Virtualization layer (hypervisor) and VM availability
- Basic building blocks: virtual networks, load balancers, block/object storage
- Hardware failures, replacement, base SLAs

When to use:

- Maximum control (custom OS, custom networking)
- Lift-and-shift of existing apps
- You have ops skills and want flexibility

Examples:

AWS EC2, EBS, VPC

Azure Virtual Machines, Disks, VNet

Google Compute Engine, Persistent Disks, VPC



Cloud Computing (BCS071/BCS713)



2. PaaS (Platform as a Service)

☞ You get a ready kitchen 🍳, you just cook your food.

Cloud gives you: OS, runtime, database, scaling system.

You only bring your code (your app) and run it.

✈ **Example:** Google App Engine, Azure App Service.

🔑 **Use when:** You just want to write code and not worry about OS or servers.
“You push code. The platform runs it. No OS or runtime headaches.”

You manage:

- Your application code, configs, and data
- App-level security (who can use your app), secrets/keys
- Scaling preferences (e.g., min/max instances)
- Cost governance

Cloud provider manages:

- OS installation + patching
- Language runtime & middleware (Java/.NET/Node, web server)





Cloud Computing (BCS071/BCS713)



- Built-in autoscaling, load balancing, health checks
- CI/CD hooks, logs/metrics, rollbacks
- Underlying infrastructure & virtualization
- Security of platform components & patching

When to use:

Faster development, less ops

Standard web/mobile APIs, event workers, microservices

You want auto-scaling and easy deployments

Examples:

AWS Elastic Beanstalk, AWS Fargate, AWS Lambda* (serverless, PaaS-like)

Azure App Service, Azure Functions*

Google App Engine, Cloud Run*, Cloud Functions*

*Serverless is usually considered PaaS (even more managed).



Cloud Computing (BCS071/BCS713)



3. SaaS (Software as a Service)

☞ You get food delivery 🍷, just eat it.

Cloud gives you: full app ready to use.

You don't build anything, just login and use.

✈ **Example:** Gmail, Zoom, Netflix, MS Office 365.

🔑 **Use when:** You just need ready software (no coding, no setup).

“You just use the app in your browser. No servers, no code deployment.”

You manage:

- Your users & access (who gets accounts/permissions)
- Your data (what you upload, retention settings)
- App configuration (settings, integrations)
- Compliance use (how your org uses the tool)

Cloud provider manages:

The entire application (features, updates, bug fixes)

All platform layers: OS, runtime, middleware, scaling



Cloud Computing (BCS071/BCS713)



- Full infrastructure, storage, networking, DR/backup of the service
- Security of the service, monitoring, availability SLAs

When to use:

You need a business tool quickly (email, CRM, docs)

No dev/ops needed, predictable pricing per user

Examples:

Google Workspace (Gmail, Docs), Microsoft 365

Salesforce, Slack, Zoom, Notion



Cloud Computing (BCS071/BCS713)



❖ What is Cloud Storage?

- Cloud storage means saving your files, data, photos, or videos on the internet (cloud) instead of your computer's hard disk.
- The files are stored on big servers managed by companies like Google, Amazon, Microsoft.
- You can access your files anytime, anywhere, on any device (phone, laptop, tablet) with internet.

☞ Example:

Google Drive, Dropbox, iCloud, OneDrive.

If you upload a photo to Google Drive, you can see it later on your phone, computer, or even share it with friends.



Cloud Computing (BCS071/BCS713)



❖ Advantages of Cloud Storage ✓

1. Access from anywhere 🌐

You just need internet, no need to carry USB or hard disk.

2. Data backup & recovery ↺

If your laptop crashes, your data is still safe in cloud.

3. Sharing is easy 📁

You can share files/folders with others in one click.

4. Scalable storage 📦

Start with free storage, buy more when needed (like 15GB free in Google Drive).

5. Cost-effective 💰

No need to buy big hard drives; pay only for what you use.

6. Collaboration 👤👤👤👤

Multiple people can work on the same file (Google Docs).

7. Automatic sync ↺

Changes you make on one device appear on all devices.



Cloud Computing (BCS071/BCS713)



8. Security

Cloud providers use encryption and strong security to protect your data.

★ Simple Analogy

Cloud storage is like a locker in a bank .

You keep your valuables (files) there.

The bank (cloud provider) keeps them safe.

☁ Storage as a Service (STaaS)

Storage as a Service means renting storage space from a cloud provider instead of buying your own hard drives or servers.

You pay only for what you use, just like paying electricity or water bills.

The provider takes care of hardware, backup, scaling, and security.

☞ Example:

Google Drive, Dropbox, iCloud (for individuals)

Amazon S3, Microsoft Azure Blob Storage, Google Cloud Storage (for businesses).



Cloud Computing (BCS071/BCS713)



Storage System in the Cloud

Cloud storage is not just “one type.” Providers offer different storage systems based on use cases. Let’s break it down:

1. Object Storage

Data is stored as objects (file + metadata + unique ID).

Very good for storing images, videos, backups, big data.

Unlimited scalability, accessed via API/HTTP.

✦✦ **Example:** Amazon S3, Azure Blob Storage, Google Cloud Storage.

2. File Storage

Data stored in folders & files, like your computer.

Good for shared documents, content management, home directories.

Uses NFS/SMB protocols.

✦✦ **Example:** Amazon EFS, Azure Files, Google Filestore.



Cloud Computing (BCS071/BCS713)



3. Block Storage

Data is split into blocks (like pieces of a puzzle).

Works like a virtual hard disk attached to a server.

Very fast → used for databases, virtual machines, transactional apps.

✦ **Example:** Amazon EBS, Azure Disk Storage, Google Persistent Disk.

✓ Advantages of Storage as a Service

- Cost saving 💰 – no need to buy servers.
- Scalability 📈 – add more storage anytime.
- Accessibility 🌐 – access from anywhere with internet.
- Reliability ↻ – automatic backup and replication.
- Security 🔒 – encryption and strong protection.
- Pay-per-use 🏷️ – only pay for what you store.



Cloud Computing (BCS071/BCS713)



❖ What is a Cloud Storage Provider?

A cloud storage provider is a company that gives you storage space on their servers. They handle hardware, backup, scaling, and security, so you don't need to buy or manage your own storage devices.

You just upload your files/data to their cloud and pay only for what you use.

🔑 Main Cloud Storage Providers :

1. Amazon Web Services (AWS) – Amazon S3

❑ Amazon S3 (Simple Storage Service):

Stores unlimited data as objects (photos, videos, backups, logs).

Very durable (99.999999999% reliability).

Provides lifecycle policies (automatic move to cheaper storage).

❑ Other AWS storage:

Amazon EBS (Elastic Block Storage): Block storage for VMs/databases.

Amazon EFS (Elastic File System): File storage for apps that need shared

Glacier: Cheap long-term archive.



Cloud Computing (BCS071/BCS713)



Think of **Amazon S3 (Simple Storage Service)** as a giant, safe, online **hard drive** on the internet. You upload files there and can get them back from anywhere.

Key words (learn these first)

- **Bucket** → like a **folder** at the top level. You create it and give it a globally unique name. Example: edushine-notes.
- **Object** → your **file** inside a bucket (PDF, image, video, JSON, etc.).
- **Key** → the **full path/name** of the object in the bucket. Example: notes/dsa/stack.pdf.
- **Region** → the **geographic area** where your bucket lives (e.g., ap-south-1 = Mumbai).
- **Prefix** → the “folder path” part of the key (e.g., notes/dsa/).

What can you do with S3?

- Store and share files (photos, backups, logs, datasets).
- Host a **static website** (HTML/CSS/JS).
- Trigger actions when files arrive (resize images, process logs) using **S3 Events + Lambda**.
- Keep old versions of files (turn on **Versioning**).
- Auto-move old files to cheaper storage (set **Lifecycle rules**).



Cloud Computing (BCS071/BCS713)



2. Microsoft Azure Storage

Azure Blob Storage: Object storage for unstructured data (like S3).

Azure Disk Storage: Block storage for VMs and databases.

Azure File Storage: File storage using SMB/NFS protocols.

Azure Archive Storage: For backup and long-term data.

💡 Azure integrates easily with Office 365 & Windows systems.

3. Google Cloud Storage

❑ Google Cloud Storage (GCS):

Highly scalable **object storage** (like Amazon S3).

Multiple storage classes (Standard, Nearline, Coldline, Archive).

Good for **AI/ML data, big data analytics**.

❑ Other Google storage:

Persistent Disk (Block storage): For Compute Engine VMs.

Filestore (File storage): For applications that need shared file systems.



Cloud Computing (BCS071/BCS713)



4. IBM Cloud Storage

- Offers Object Storage, Block Storage, File Storage.
- Strong in hybrid cloud solutions.
- Known for enterprise-level compliance and security.

5. Oracle Cloud Storage

- Oracle Cloud Infrastructure (OCI) Object Storage: for backups, big data.
- Block Volumes: for databases and enterprise apps (good for Oracle DB).
- File Storage: shared file systems.
- 💡 Often used by enterprises already using Oracle Databases.

✓ Advantages Cloud Storage Providers Give

- ✓ Global availability 🌐 → Access data anywhere.
- ✓ High reliability ↻ → Automatic replication.
- ✓ Scalability 📈 → Store GBs or PBs easily.
- ✓ Security 🔒 → Encryption, firewalls, compliance.
- ✓ Pay as you go 💰 → Only pay for storage used.



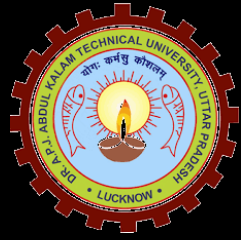
Cloud Computing (BCS071/BCS713)



Thank You....



Download Notes : <https://rzp.io/rzp/tkFQMuOZ>



Cloud Computing (BCS071/BCS713)

Unit – 4 Resource Management & Security In Cloud

IV

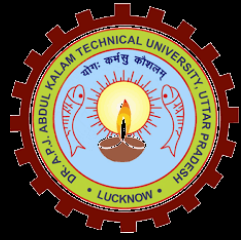
Resource Management And Security In Cloud: Inter Cloud Resource Management – Resource Provisioning and Resource Provisioning Methods – Global Exchange of Cloud Resources – Security Overview – Cloud Security Challenges – Software-as-a-Service Security – Security Governance – Virtual Machine Security – IAM – Security Standards.



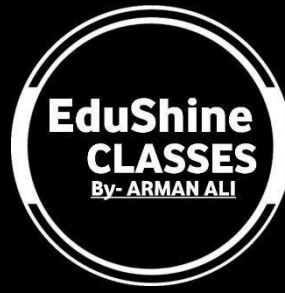
EduShine Classes – Arman Ali



Download Notes : <https://rzp.io/rzp/tkFQMuoZ>



Cloud Computing (BCS071/BCS713)



Resource Management in Cloud

In cloud, resources = CPU, memory, storage, bandwidth, applications.

Resource management means **distributing, controlling, and monitoring** these resources so every user gets what they need **without waste**.

Example: If 100 students use Google Drive at the same time, Google must manage its storage servers so that everyone gets smooth service.

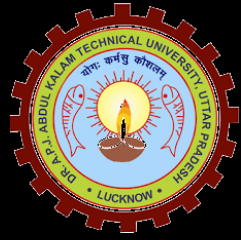
Inter-Cloud Resource Management

- **Inter-cloud** = between multiple cloud providers.
- Companies don't always depend on just one cloud (like AWS). They may use **AWS + Azure + Google Cloud** together.
- **Inter-cloud resource management** = managing and sharing resources across **different cloud providers**.

Example:

Netflix uses AWS for storage + Google Cloud for analytics.

They must manage resources across both to keep services running smoothly.



Cloud Computing (BCS071/BCS713)



❖ Why is it Needed? (Need for Inter-Cloud Resource Management)

1. High Demand Handling 📈

If one cloud runs out of resources, another cloud can provide them.

2. Cost Optimization 💰

Different clouds have different prices. Companies choose the cheapest option for a resource.

3. Better Performance ⚡

User gets service from the **nearest cloud server** for faster speed.

4. Avoid Vendor Lock-in 🔒

Companies don't depend only on one provider; they can switch easily.

5. Business Continuity ↻

If AWS has downtime, resources from Azure or Google Cloud can be used → no service interruption.

6. Flexibility & Scalability 📦

Easy to increase/decrease resources using multiple clouds.



Cloud Computing (BCS071/BCS713)



Benefits of Inter-Cloud Resource Management ✓

1. Reliability

Even if one cloud fails, others keep the system running.

2. Cost Savings

Choose best price cloud provider at any time.

3. Global Access

Resources can be served from the nearest location worldwide.

4. Improved Security

Data can be divided across clouds (multi-cloud security).

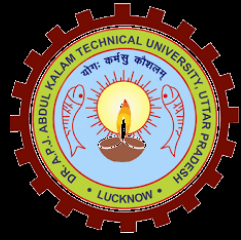
5. Efficient Resource Usage

No wastage, resources are balanced across providers.

★ Simple Analogy

Think of Inter-cloud resource management like using multiple food delivery apps (Swiggy + Zomato + UberEats):

If Swiggy has no restaurant open, you can order from Zomato.



Cloud Computing (BCS071/BCS713)



☁ Types of Inter-Cloud Resource Management

There are 2 main types:

1. 🌐 Federation Clouds

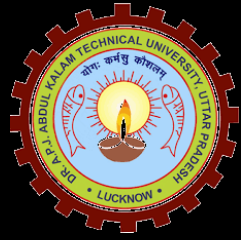
In federation cloud, many cloud providers agree to work together. They share their resources (storage, computing, bandwidth) voluntarily. It's like a teamwork of cloud providers.

👉 Example:

- Suppose Govt. of India has its own private cloud.
- It can join with another state government's cloud → so both can share resources (data, apps, storage).
- Universities and research centers also use **federated clouds** to share computing power.

✓ Key points of Federation Cloud:

- Cloud providers **cooperate**.
- Resources are **openly exchanged**.
- Used by **governments, NGOs, universities, private clouds**.



Cloud Computing (BCS071/BCS713)



2. ☁ Multi-Cloud

In multi-cloud, one company or client uses many different clouds (AWS, Azure, Google Cloud, etc).

But here, cloud providers do not cooperate with each other.

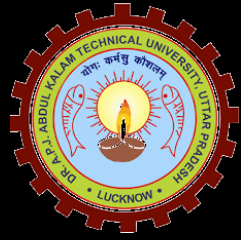
The client (user) must manage resources between them.

👉 Example:

- Netflix uses AWS for video streaming + Google Cloud for analytics + Azure for AI.
- But AWS and Google are not “working together”; Netflix itself manages resources.

✓ Key points of Multi-Cloud:

- Providers do not share resources voluntarily.
- User (or their software tools) decides how to use different clouds.
- Mix of public + private clouds often.
- Popular for flexibility, backup, and cost savings.



Cloud Computing (BCS071/BCS713)



🌐 Topologies in Inter-Cloud Architecture(How Inter Cloud Arch. Works)

There are 4 main types:

1. Peer-to-Peer Inter-Cloud Federation

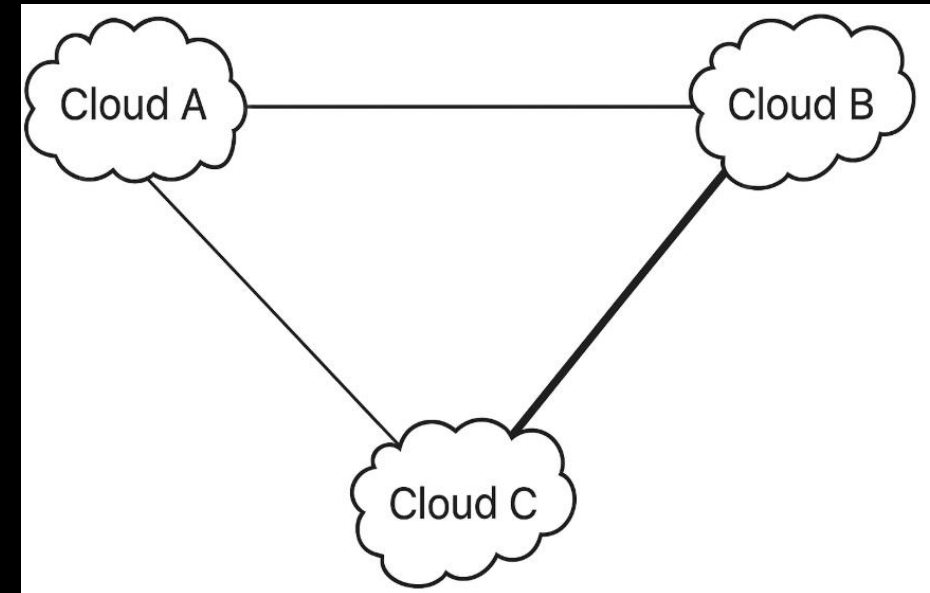
- Clouds **connect directly with each other**.
- They **negotiate and share resources** without any middleman.
- Sometimes, they use a small directory/broker just to find other clouds, but most work is **direct communication**.

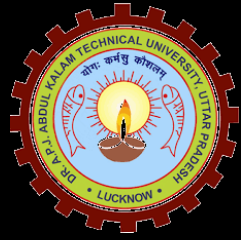
👉 Example:

Two universities (University A & University B) connect their private clouds directly to share computing resources for research.

RESERVOIR Project is based on this.

✓ **Key Idea:** Direct cloud-to-cloud friendship.





Cloud Computing (BCS071/BCS713)



2. 🏢 Centralized Inter-Cloud Federation

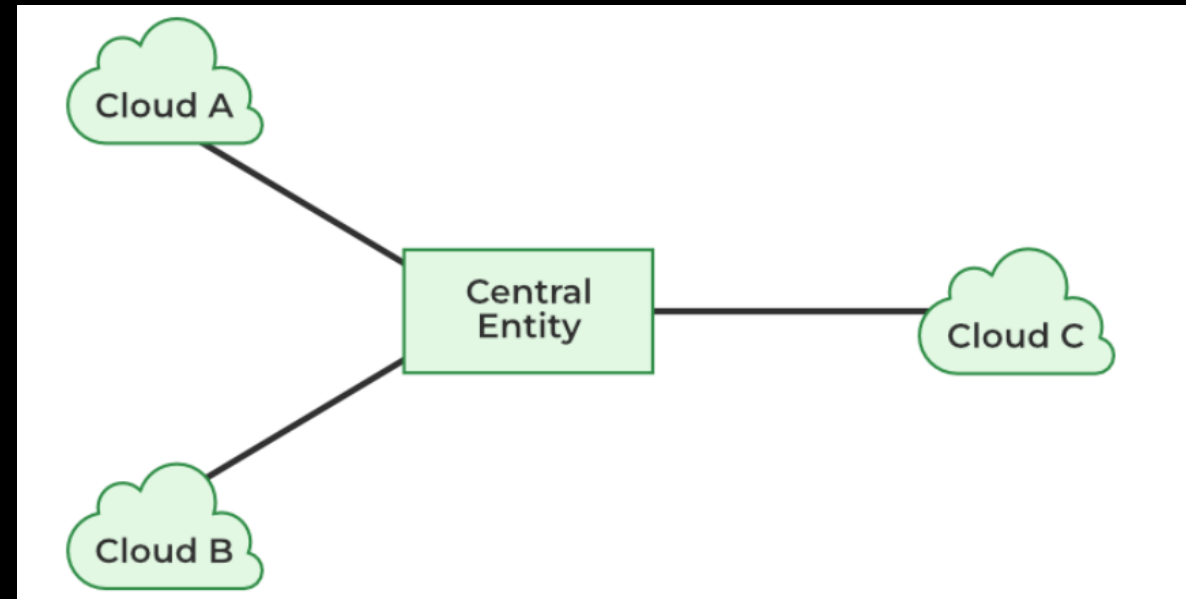
- A **central authority** controls and manages resource sharing.
- This central body keeps a **registry (database)** of available resources from all clouds.
- When a cloud needs resources, it asks the **central manager**.

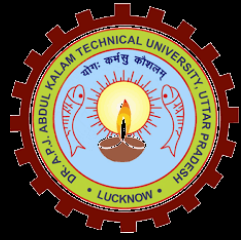
👉 Example:

Think of a **job portal** like Naukri.com. Companies (clouds) post jobs (resources), and students (clouds in need) apply through the central website.

Projects like **Dynamic Cloud Collaboration (DCC)** and **Federated Cloud Management** use this.

✓ **Key Idea:** One central manager coordinates everything.





Cloud Computing (BCS071/BCS713)



3. ☁ Multi-Cloud Service

- Here, a **client** uses a **service** that gives access to **multiple clouds**.
- A **broker service** helps the client choose and manage resources from different clouds.

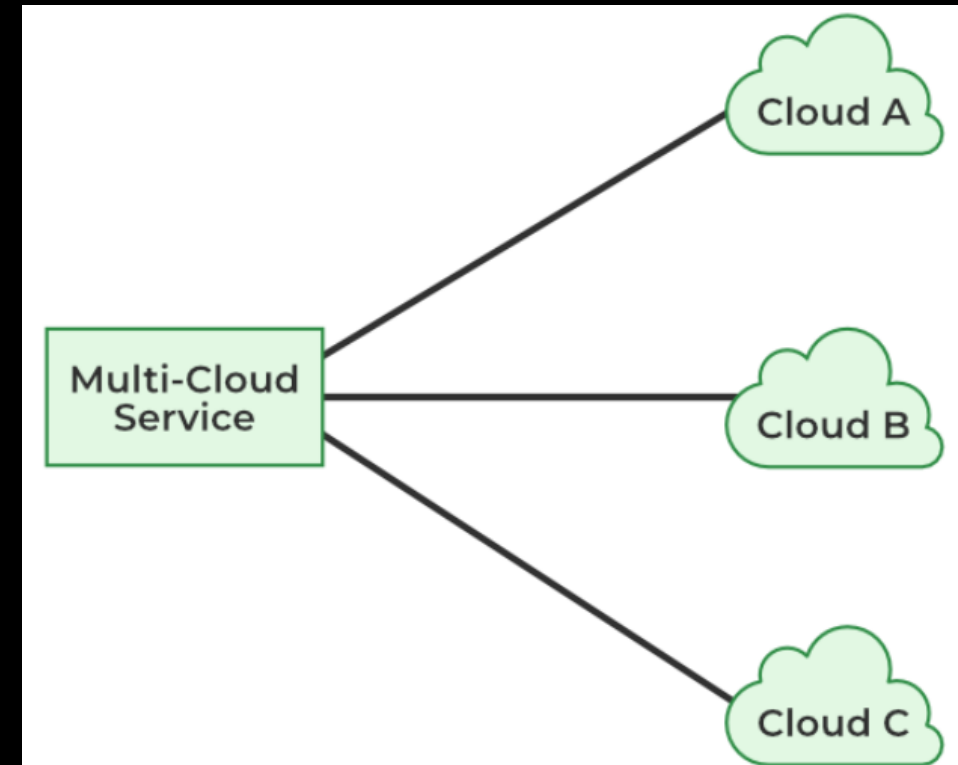
👉 Example:

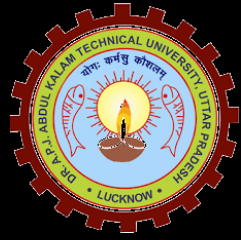
A travel booking site (like MakeMyTrip) checks flights from **IndiGo, Air India, Vistara** at once
→ you don't go to each airline's site.

Similarly, a **multi-cloud service** helps you use AWS, Azure, and Google Cloud together.

Examples: OPTIMUS, Contrail, MOSAIC, STRATOS.

✓ **Key Idea:** A **broker service** helps clients connect to many clouds.





Cloud Computing (BCS071/BCS713)



4. 📁 Multi-Cloud Libraries

Instead of using an external broker, the **client builds its own broker** using **libraries (APIs)**. Libraries give a **common way (standard API)** to access different clouds. Developers write code once and use it with any cloud.

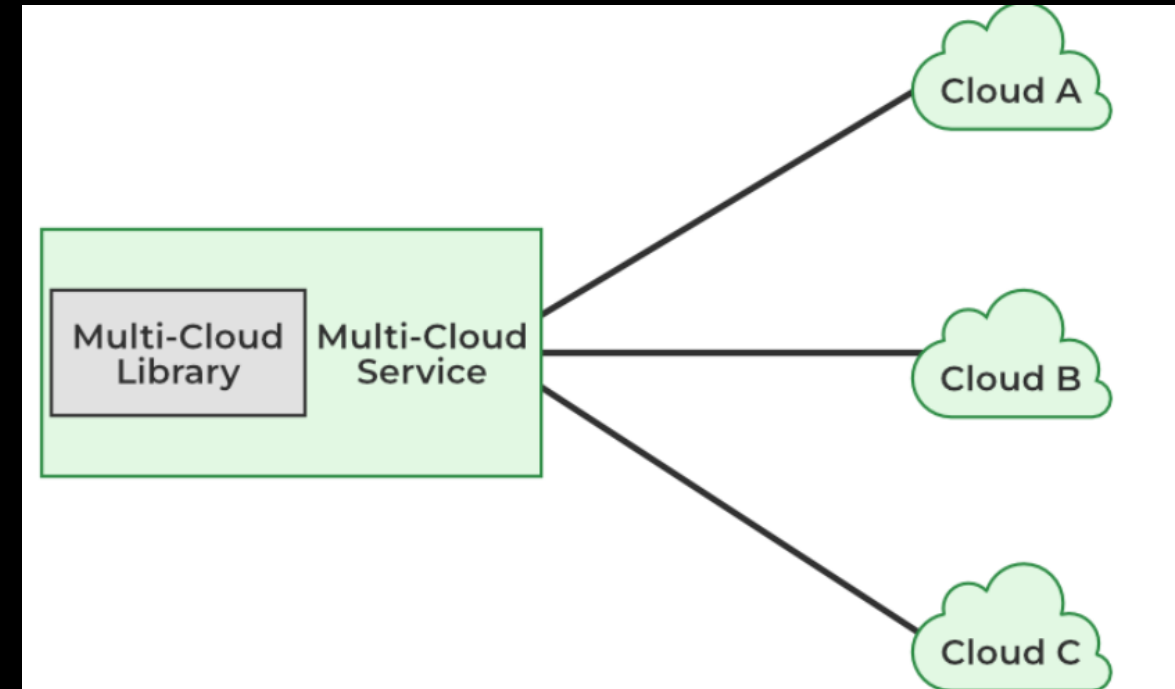
🔗 Example:

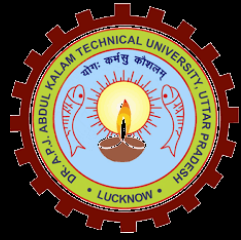
If you are coding in Java/Python/Ruby, you can use special libraries like:

- JClouds (Java)
- Apache LibCloud (Python)
- Apache DeltaCloud (Ruby)

These libraries help you connect to AWS, Azure, Google Cloud in the same way.

✓ **Key Idea:** Client uses **APIs/libraries** to make cloud usage simple and uniform.



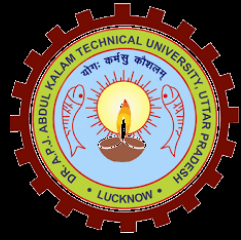


Cloud Computing (BCS071/BCS713)



★ Super Simple Analogy

- ❖ **Peer-to-Peer** → Like two friends sharing books directly 📖.
- ❖ **Centralized** → Like a library 📖 where all books are registered, and you borrow through the librarian.
- ❖ **Multi-Cloud Service** → Like a travel app ✈️ showing flights from many airlines.
- ❖ **Multi-Cloud Libraries** → Like having a **universal charger** 🔌 that works with any phone brand.



Cloud Computing (BCS071/BCS713)



🌐 Resource Provisioning in Cloud :

- Provisioning = assigning and managing resources (CPU, storage, memory, network, apps) to cloud users when they need it.
- It's like giving resources on demand.

👉 Example:

When you create a new Gmail account, Google **provisions storage (15GB)** for you.

When you buy Netflix subscription, it **provisions servers & bandwidth** to stream videos for you.

So provisioning = making resources ready and available.

✂️ Types of Resource Provisioning

There are **3 main types**:

1. **Static Provisioning**
2. **Dynamic Provisioning**
3. **Self service provisionig**



Cloud Computing (BCS071/BCS713)



1. Static Provisioning

- Resources are **fixed** and given in advance.
- Even if the user does not use them fully, they **cannot change**.
- **No automatic scaling** (increase/decrease).

☞ Example:

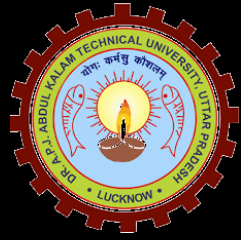
A company buys **5 servers permanently** from a cloud provider. Even if it uses only 2 servers, it still pays for all 5.

Like buying a **fixed-size water tank** 🛢 – whether you use it or not, it's always there.

✓ **Good for:** Workloads that are **predictable and stable** (e.g., small websites).

2. ⚡ Dynamic Provisioning

- Resources are **allocated automatically** based on demand.
- If demand increases → more resources are added.
- If demand decreases → resources are reduced.
- This saves money and improves efficiency.



Cloud Computing (BCS071/BCS713)



☞ Example:

Amazon shopping website adds **extra servers on Black Friday** sale and reduces them later. Like a **water tap connected to a pipeline** 🚰 – water (resources) increases or decreases as you need.

✓ **Good for:** Workloads that **change frequently** (e.g., e-commerce sites, video streaming).

3. Self-Service Provisioning

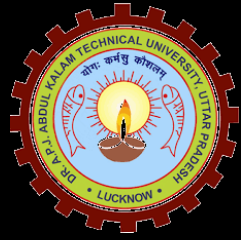
- Users can **create, manage, and delete resources themselves** without asking the cloud provider.
- A **dashboard or portal** is provided where users can choose what they need.

☞ Example:

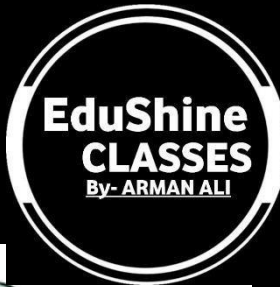
On AWS or Azure, you can log in, choose a virtual machine, storage, or database, and create it instantly.

Like **online shopping** 🛒 – you select what you want, pay, and it's delivered immediately.

✓ **Good for:** Developers, companies, and anyone who wants **full control and flexibility**.

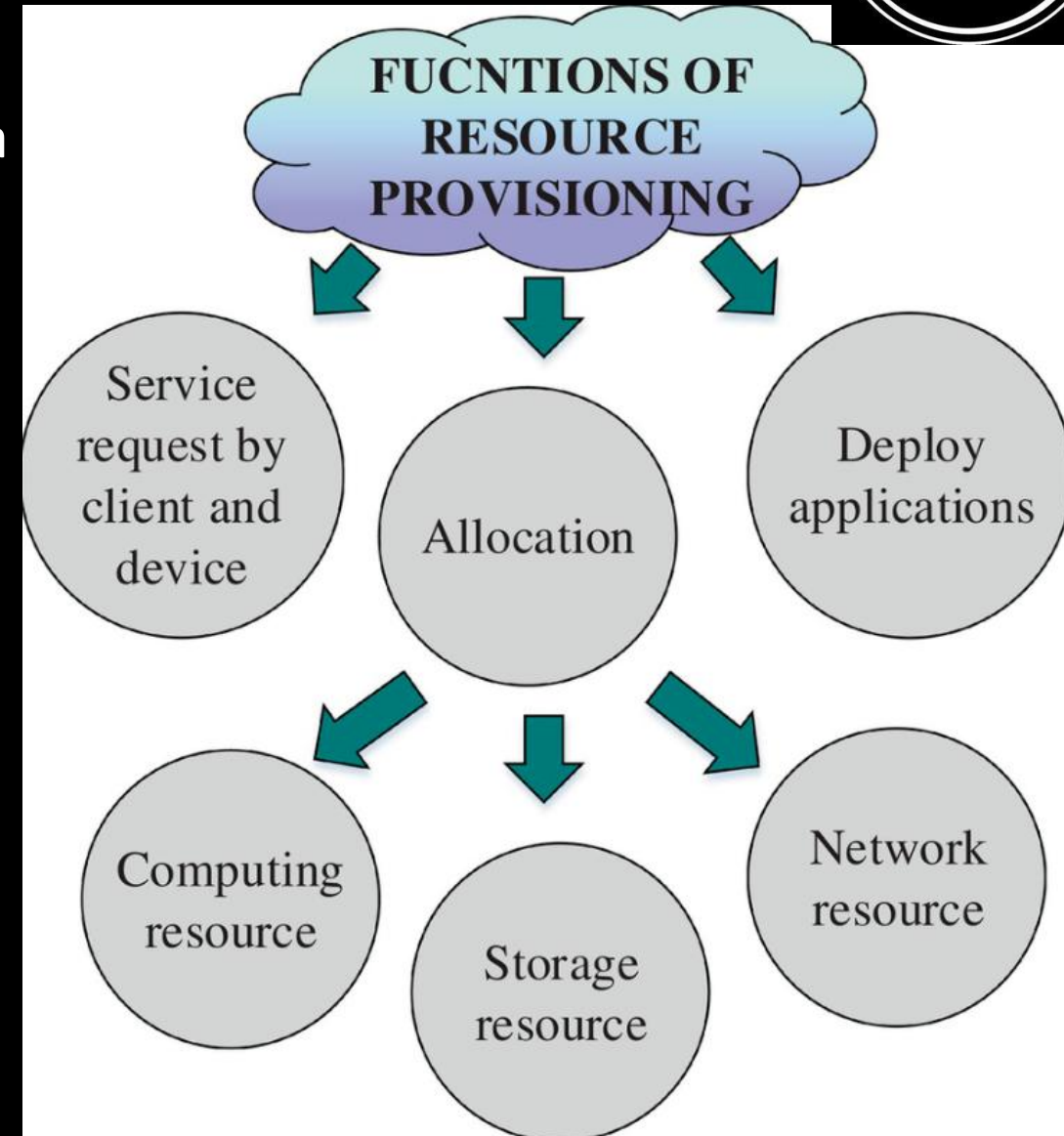


Cloud Computing (BCS071/BCS713)



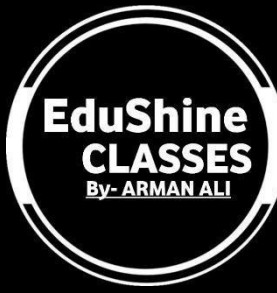
🔑 Analogy to Remember

- ✓ **Static** = Prepaid SIM card 📱 (you buy fixed data in advance, whether you use or not).
- ✓ **Dynamic** = Postpaid SIM card 📱 (you pay according to how much you use).
- ✓ **Self-Service** = Recharge app 📱 (you can recharge or change plan anytime yourself).





Cloud Computing (BCS071/BCS713)



🌐 Demand-Driven vs Event-Driven Resource Provisioning

Both are types of **Dynamic Provisioning** (where resources increase or decrease automatically).

1. 📈 Demand-Driven Resource Provisioning

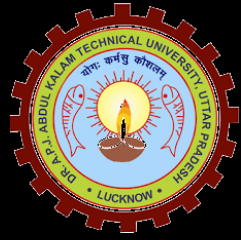
- Resources are added or removed **based on user demand (workload)**.
- The system looks at **how many users are active** or **how much load is on servers**.
- If demand increases → more resources are given.
- If demand decreases → extra resources are removed.

👉 Example:

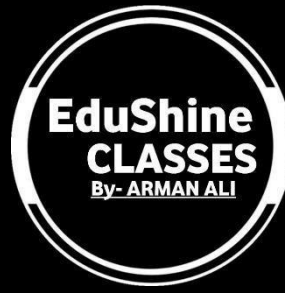
Amazon website adds more servers during **Black Friday sales** when many users visit.
YouTube adds extra bandwidth when too many people watch videos at the same time.

✓ Easy Analogy:

Like a **restaurant hiring extra chefs** when more customers come, and letting them go when business is slow.



Cloud Computing (BCS071/BCS713)



2. ⚡ Event-Driven Resource Provisioning

- Resources are added or removed **based on specific events or triggers**, not just demand.
- An “event” can be: system error, sudden traffic spike, new user sign-up, scheduled job, or an attack.
- System reacts **immediately** when the event occurs.

👉 Example:

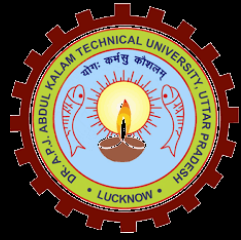
A news website automatically adds servers when **breaking news** is published (event = news release).

A company’s backup server turns on when **main server crashes** (event = failure).

Auto-scaling triggers if CPU usage goes beyond 80% (event = CPU threshold crossed).

✓ Easy Analogy:

Like **fire alarms** 🔊 — they trigger instantly when smoke (event) is detected, and fire sprinklers turn on.



Cloud Computing (BCS071/BCS713)

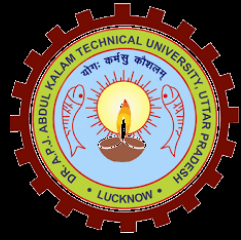


❖ What is a Global Cloud Exchange?

A **Global Cloud Exchange** is like a big market or stock-exchange — but for cloud resources (CPU, storage, bandwidth, special services).

- Cloud providers **offer** spare resources.
- Customers or other clouds **buy/borrow** those resources when they need them.
- A central system (an **exchange** or brokers) helps find, match, negotiate and settle deals between providers and consumers.

Analogy: imagine an electricity market where power plants sell extra power and shops buy it when demand is high. The cloud exchange does the same for computing power, storage, and network.

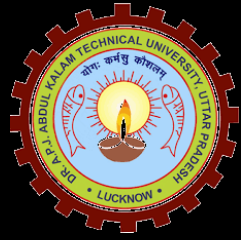


Cloud Computing (BCS071/BCS713)



❖ Why do we need a Global Cloud Exchange?

1. **Handle Peaks & Bursts** – If your website suddenly gets millions of visitors, you can take extra capacity from the market immediately.
2. **Cost Optimization** – Pick the cheapest provider for a specific time or region.
3. **Better Performance / Locality** – Serve users from a nearby datacenter (lower latency).
4. **Failover & Resilience** – If one cloud fails, others can take over quickly.
5. **Access to Specialized Services** – Get a special GPU farm, or a region-specific service, without buying it yourself.
6. **Avoid Vendor Lock-in** – You can move between providers easily because resources are traded.
7. **Global Resource Pooling** – Small clouds or universities can share resources with others to run big jobs.



Cloud Computing (BCS071/BCS713)



↻ How Global Cloud Exchange Works :

1. Provider Registers Resources

- Cloud providers (like AWS, Azure, Google Cloud, or smaller clouds) tell the exchange:
- What resources they have (CPU, storage, servers, network).
- Location (India, US, Europe).
- Price and capacity.
- SLA (Service Level Agreement = performance promises like 99.9% uptime).

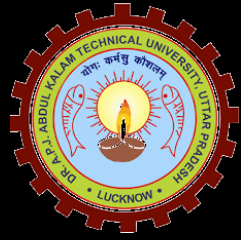
☞ **Example:** AWS says: *"I have 100 servers available in Mumbai at \$0.05/hour each, with 99.9% uptime."*

2. Consumer (or Broker) Requests Resources

A company (consumer) or a **broker** (middleman) goes to the exchange and requests resources.

They specify:

- What they need (like 20 CPUs, 500GB storage).
- How long (time window → 1 day, 1 week).



Cloud Computing (BCS071/BCS713)



- Quality of Service (QoS → like speed, latency).

☞ **Example:** Flipkart asks: *“I need 20 servers in India for 5 days with fast response under 50ms.”*

3. Matchmaking (Finding the Best Provider)

- The exchange compares **consumer’s request** with **providers’ offers**.
- It finds the best match (correct location, price, speed, capacity).

☞ **Example:** The exchange checks AWS, Azure, and Google, and finds AWS Mumbai fits Flipkart’s needs.

4. SLA Negotiation (Agreement on Service Level)

- Both sides agree on terms:
- Uptime (99.9% or 99.99%).
- Latency (how fast service responds).
- Price and penalties if SLA breaks.

☞ **Example:** Flipkart and AWS agree: *“If uptime goes below 99.9%, AWS w*



Cloud Computing (BCS071/BCS713)



5. Provisioning & Orchestration

The resources are now **allocated and set up** for the consumer.

Orchestration = connecting everything together (servers, storage, network).

☞ **Example:** Flipkart's new 20 servers are launched automatically and connected to their website system.

6. Monitoring

The exchange keeps checking whether the provider is following SLA (uptime, speed, availability).

If something goes wrong → alerts are sent.

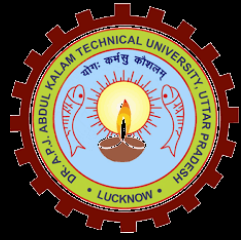
☞ **Example:** Exchange monitors AWS servers to ensure Flipkart's website stays online and fast.

7. Billing & Settlement

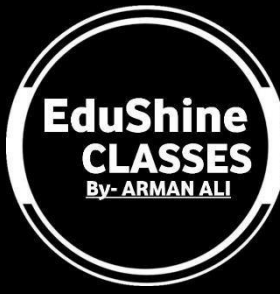
At the end, usage is measured (metering).

Bills are created and payments are settled.

☞ **Example:** Flipkart used 20 servers for 5 days = $20 \times 24 \times \$0.05 = \24 .



Cloud Computing (BCS071/BCS713)



★ Super Simple Analogy

Think of it like **Uber for Cloud** 🚗:

- Drivers (providers) register cars (resources).
 - Rider (consumer) requests a ride (resources).
 - Uber app (exchange) matches rider and driver.
 - They agree on fare (SLA negotiation).
 - Ride happens (provisioning).
 - Uber tracks the ride (monitoring).
 - At the end, payment is made (billing).
-



Cloud Computing (BCS071/BCS713)



⚠ Challenges in Global Cloud Exchange

1. Interoperability (Compatibility Issue)

- Different clouds (AWS, Azure, Google) use different systems and APIs.
- Hard to connect or move apps between them.

☞ **Example:** An app built for AWS may not run easily on Azure.

2. Security & Trust

When you use another provider's resources, you must trust them with your data.

☞ **Example:** What if they don't encrypt your files properly? Or misuse your data?

3. Data Privacy & Laws

Many countries have rules: data must stay inside the country.

☞ **Example:** Indian user data may not be allowed to move to US servers.

4. SLA (Service Level Agreement) Issues

If provider promises 99.9% uptime but fails, who takes responsibility?

☞ **Example:** If Flipkart's site crashes, who pays — AWS or the exchange?



Cloud Computing (BCS071/BCS713)



☁ Cloud Security Overview :

Cloud security means **protecting data, applications, and resources** that are stored and used in the cloud.

It makes sure:

- ✓ **Confidentiality** – Only the right people can access the data.
- ✓ **Integrity** – Data cannot be changed by attackers.
- ✓ **Availability** – Cloud services must be available all the time (no downtime).

🔑 **Example:** Just like you lock your house with keys, CCTV, and security guard — in the cloud, we use encryption, firewalls, and monitoring.

⚠ Security Challenges in Cloud Computing :

1. Data Breach (Data Theft)

Hackers may steal sensitive data from cloud.

🔑 **Example:** Credit card info stolen from a shopping site's cloud.



Cloud Computing (BCS071/BCS713)



2. Data Loss

Data may get deleted accidentally or due to attack.

☞ **Example:** A company loses customer records because backup failed.

3. Account Hijacking

If hackers steal login/password, they can control cloud resources.

☞ **Example:** Hacker logs into AWS account and uses it to mine cryptocurrency.

4. Insider Threats

Employees of cloud provider or company misuse access.

☞ **Example:** A staff member copies secret project files.

5. Insecure APIs

Cloud provides APIs (software interfaces). If APIs are weak, hackers can attack easily.

☞ **Example:** Poorly protected API lets hacker access user database.

6. Compliance & Legal Issues

Different countries have different rules about data storage.

☞ **Example:** EU's GDPR law says data of Europeans must stay safe inside



Cloud Computing (BCS071/BCS713)



7. Shared Technology Risks

In cloud, many users share the same hardware. If isolation fails, data can leak.

☞ **Example:** One company's data becomes visible to another due to a bug.

8. Denial of Service (DoS) Attacks

Attackers overload the cloud system so it crashes.

☞ **Example:** Flooding Flipkart's website with fake traffic so real customers can't shop.

🔒 Security Governance in Cloud Computing

Security Governance = A set of rules, policies, and controls that guide how security is managed in the cloud.

☞ It ensures cloud services are safe, legal, and aligned with business goals.

🛡️ Key Parts of Cloud Security Governance

1. Policies & Rules

Define what is allowed and what is not.

☞ **Example:** "All data must be encrypted" or "Only managers can access financial reports."



Cloud Computing (BCS071/BCS713)



2. Roles & Responsibilities

Clear assignment of who manages what (cloud provider vs customer).

☞ **Example:** Provider secures the hardware; customer secures passwords.

3. Compliance & Regulations

Follow laws (like GDPR, HIPAA, IT Act in India).

☞ **Example:** A hospital must protect patient data according to medical laws.

4. Risk Management

Identify possible risks (data loss, hacking) and prepare solutions.

☞ **Example:** Backup plans in case of server crash.

5. Monitoring & Auditing

Keep checking the system to make sure rules are followed.

☞ **Example:** Regular security scans, log reviews, audits.

6. Incident Response

Define steps if something goes wrong (attack or data leak).

☞ **Example:** If hacker attacks, system should lock accounts and notify admin.



Cloud Computing (BCS071/BCS713)



☁ SaaS(Software as a service) Security :

SaaS = Software delivered over the internet (like Gmail, Google Drive, Zoom, Dropbox).

SaaS Security = Protecting data, users, and applications inside SaaS platforms from theft, loss, or misuse.

👉 **Example:** Keeping your Google Drive safe from hackers or unauthorized users.

★ Importance of SaaS Security

1. Protects sensitive data

👉 Example: A company's financial reports on Google Drive must stay private.

2. Ensures business continuity

👉 If SaaS is attacked, business apps (like Zoom or Salesforce) can stop working.

3. Compliance with laws

👉 Companies must follow legal rules like GDPR (Europe) or HIPAA (healthcare).

4. Builds trust

👉 Customers feel safe when they know their data is secure.



Cloud Computing (BCS071/BCS713)



⚠ Key Challenges in SaaS Security

1. Data Breaches

Hackers may steal company or personal data.

👉 **Example:** Hackers stealing emails from Gmail.

2. Account Hijacking

Weak or stolen passwords let attackers take over accounts.

👉 **Example:** Someone steals your Netflix account login.

3. Insider Threats

Employees (inside company or SaaS provider) may misuse access.

👉 **Example:** A worker downloads confidential files without permission.

4. Shadow IT

Employees use SaaS apps without IT approval.

👉 **Example:** Staff storing work files on personal Dropbox.

5. Data Loss / Lack of Backup

If SaaS fails or is deleted, data may be lost.



Cloud Computing (BCS071/BCS713)



☞ **Example:** Accidentally deleting important files from Google Docs.

6. Compliance Issues

SaaS data stored in other countries may break legal rules.

☞ **Example:** A company storing Indian data on US servers against law.

🛡️ SaaS Security Best Practices

1. Strong Authentication

Use **Multi-Factor Authentication (MFA)** → password + OTP or fingerprint.

2. Data Encryption

Encrypt data **at rest (stored)** and **in transit (moving over internet)**.

3. Access Control

Give users **only necessary access** (least privilege).

☞ Example: Interns can't access financial records.

4. Regular Monitoring & Auditing

Track logins, unusual activity, and generate security reports.

5. Backup & Recovery Plans



Cloud Computing (BCS071/BCS713)



Keep regular backups in case of accidental deletion or attack.

6. Employee Training

Teach employees to avoid phishing, weak passwords, and unsafe apps.

7. Compliance Checks

Ensure SaaS providers follow rules like GDPR, HIPAA, etc.

☁ What is Virtualized Security?

Virtualized Security = Security designed for virtual environments (where many virtual machines run on the same physical server using a hypervisor).

👉 In simple words:

When one computer (server) is divided into many virtual machines (VMs), normal security tools (like firewalls, antivirus) may not work properly.

So, virtualized security provides special protection for these VMs, hypervisors, and virtual networks.





Cloud Computing (BCS071/BCS713)



⚙️ How Virtualized Security Works?

1. **Monitors Virtual Machines (VMs):** Keeps an eye on each VM to prevent attacks.
2. **Protects Hypervisor:** Secures the software (hypervisor) that controls all VMs.
3. **Isolates Resources:** Stops one VM from accessing or attacking another VM.
4. **Virtual Firewalls & IDS/IPS:** Creates security tools inside the virtual environment (not only on physical hardware).

🔑 **Example:** Imagine one big apartment building 🏢 (server) with many flats (VMs). Virtualized security = CCTV, locks, and guards inside the building to keep each flat safe.

🔒 Types of Virtualization Security :

1. Network Virtualization Security -

- Protects virtual networks (software-based networks inside the cloud).
- Uses virtual firewalls, VPNs, intrusion detection systems (IDS).

🔑 **Example:** Just like Wi-Fi password protects your home network, network virtualization security protects data moving between VMs.



Cloud Computing (BCS071/BCS713)



2. Storage Virtualization Security

Protects data stored in virtualized storage systems (like SAN, cloud storage).

Uses encryption, access control, backup.

🔑 **Example:** Like locking your cupboard (encryption) so only you can open it, even if it's inside a shared room.

3. Server Virtualization Security

Protects virtual machines (VMs) running on a server.

Uses antivirus, VM isolation, snapshots for recovery.

🔑 **Example:** If one flat in an apartment catches fire, walls (isolation) stop it from spreading to other flats.

4. Application Virtualization Security

- Protects apps running in a **virtual environment**.
- Uses **sandboxing (isolating apps), patching, access control**.

🔑 **Example:** Running a risky app inside a “sandbox” so even if it has a virus, it cannot damage other apps.



Cloud Computing (BCS071/BCS713)



☁ What is Cloud IAM?

Cloud IAM (Identity and Access Management) = A system that controls who can access cloud resources, what they can do, and under what conditions.

👉 **Example:** In Google Drive, you can decide:

- Arman = can view only
- Abhishek = can edit files
- Admin = can add/remove users

That's Cloud IAM in action.



Identity and Access
Management
(IAM)



Cloud Computing (BCS071/BCS713)



🔑 Key Components of Cloud IAM

1. Identities (Users & Groups)

People or services using the cloud.

👉 **Example:** Employee, Admin, Application.

2. Authentication

Verifying “Who are you?”.

👉 **Example:** Logging in with password, OTP, or fingerprint.

3. Authorization (Access Control)

Deciding “What can you do?”.

👉 **Example:** View file, edit file, delete file.

4. Roles & Policies

Define permissions clearly.

👉 **Example:**

Viewer = Read-only.

Editor = Read + Write.



Cloud Computing (BCS071/BCS713)



Admin = Full control.

5. Audit & Monitoring

Track who accessed what and when.

🔑 **Example:** Log showing “Ali downloaded report at 5PM.”

★ Benefits of Cloud IAM

1. Stronger Security 🛡️

Prevents unauthorized access.

2. Least Privilege 👤

Users get only what they need, nothing extra.

3. Compliance 📋

Meets legal rules (like GDPR, HIPAA).

4. Centralized Control 📄

One dashboard to manage access across all cloud services.

5. Scalability 📈

Easy to add/remove thousands of users in large companies.



Cloud Computing (BCS071/BCS713)



🛡 Best Practices for Cloud IAM

1. Use Multi-Factor Authentication (MFA)

☞ Password + OTP = more security.

2. Follow Least Privilege Principle

☞ Give only minimum access required.

3. Regularly Review Access

☞ Remove access for employees who leave.

4. Strong Password Policies

☞ Force users to set strong, unique passwords.

5. Role-Based Access Control (RBAC)

☞ Assign roles (Admin, User, Viewer) instead of manual permissions.

6. Enable Logging & Monitoring

☞ Keep track of suspicious activity.

7. Use Identity Federation

☞ Allow login with trusted IDs (like Google, Microsoft).



Cloud Computing (BCS071/BCS713)



☁ What is Cloud Security Standard?

- Cloud Security Standards = Rules and guidelines made by international organizations to ensure cloud systems are safe, reliable, and compliant.

👉 **Example:** Just like food safety standards (FSSAI, FDA) ensure food is safe to eat, cloud security standards ensure data in the cloud is safe to store and process.

☀ Why Cloud Security Standards are Needed?

- To protect data from theft or loss.
- To meet government regulations (laws).
- To build trust between cloud providers and users.
- To ensure all providers follow the same security practices.



Cloud Computing (BCS071/BCS713)



🔑 Common Cloud Security Standards (Easy)

1. ISO/IEC 27001 & 27017 & 27018

a) ISO/IEC 27001 → Standard for Information Security Management.

☞ **Example:** Ensures cloud provider has strong security controls.

b) ISO/IEC 27017 → Focuses on cloud-specific security guidelines.

☞ **Example:** Rules for VM isolation, cloud firewalls.

c) ISO/IEC 27018 → Protects personal data in cloud.

☞ **Example:** Ensures user privacy in Google Drive/OneDrive.

2. NIST (National Institute of Standards and Technology)

US-based standard with a Cybersecurity Framework (CSF).

Defines Identify, Protect, Detect, Respond, Recover steps.

☞ **Example:** Cloud provider must have monitoring, backups, and recovery plans.

3. CSA STAR (Cloud Security Alliance – Security Trust Assurance and Risk)

A program that rates cloud providers' security practices.

☞ **Example:** Like a “security certificate” that shows AWS/Azure/Google fol



Cloud Computing (BCS071/BCS713)



4. PCI DSS (Payment Card Industry Data Security Standard)

Standard for protecting payment and cardholder data in the cloud.

☞ **Example:** PayPal or Stripe must follow PCI DSS to protect credit card numbers.

5. HIPAA (Health Insurance Portability and Accountability Act)

US standard for healthcare data security.

☞ **Example:** Cloud apps storing hospital patient records must follow HIPAA.

6. GDPR (General Data Protection Regulation – EU Law)

Protects personal data of European citizens.

☞ **Example:** A cloud provider must ask permission before storing your data and allow you to delete it (“Right to be Forgotten”).

7. FedRAMP (Federal Risk and Authorization Management Program)

US government cloud security standard.

☞ **Example:** If a cloud provider wants to provide services to the US govt, it must be FedRAMP certified.



Cloud Computing (BCS071/BCS713)



What is a Honeypot?

A **honeypot** is a **fake system, server, or resource** created to attract hackers and study their activities.

 **Example:** Like putting a **jar of honey**  **in the forest** to attract bees — in cybersecurity, we put a **fake target** to attract hackers.

It **looks real** but it's actually a trap.

Types of Honeypots

1. Low-Interaction Honeypot

Simulates only **basic services** (like fake login page, fake database).

Easy to set up, less risky.

 **Example:** A fake email server that just records login attempts.

2. High-Interaction Honeypot

A **real system with operating system & applications**, but isolated.

Lets attackers spend more time → gives deep insights.

 **Example:** A real Windows server left vulnerable to watch how hackers attack.



Cloud Computing (BCS071/BCS713)



3. Production Honeypot

Placed inside real company network.

Used mainly for detection and diversion (not for research).

☞ **Example:** A fake database inside a bank's cloud to distract hackers.

4. Research Honeypot

Used by security researchers to study hacker behavior.

Collects data about attack patterns, tools, and malware.

☞ **Example:** A university runs a fake network to learn about new viruses.

5. Malware Honeypot

Special honeypot designed to trap malware (viruses, worms, ransomware).

☞ **Example:** A fake Android app store used to catch malware apps.

6. Spam Honeypot

Designed to trap spammers.

Creates fake email accounts to collect spam messages.

☞ **Example:** A system that records spammers sending junk mail.



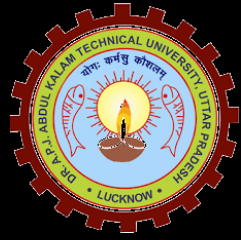
Cloud Computing (BCS071/BCS713)



Thank You....



Download Notes : <https://rzp.io/rzp/tkFQMuOZ>



Cloud Computing (BCS071/BCS713)

Unit – 5 Cloud Technologies & Advancement Hadoop

V

Cloud Technologies And Advancements Hadoop: MapReduce – Virtual Box — Google App Engine – Programming Environment for Google App Engine — Open Stack – Federation in the Cloud – Four Levels of Federation – Federated Services and Applications – Future of Federation.



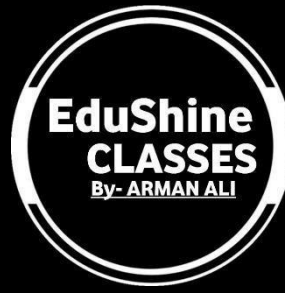
EduShine Classes – Arman Ali



Download Notes : <https://rzp.io/rzp/tGaErCk>



Cloud Computing (BCS071/BCS713)



What is Hadoop?

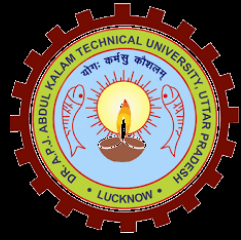
Hadoop is an **open-source framework** designed to store and process **large datasets** across multiple computers in a **distributed manner**. It uses cheap hardware to handle big data efficiently. The **two main components** of Hadoop are:

- **HDFS (Hadoop Distributed File System)**: For storing data.
- **MapReduce**: For processing data.

❖ Apache Hadoop :

Apache Hadoop is an open-source platform used to store and analyze large datasets using distributed systems. It is scalable, meaning you can add more computers to handle more data. It has three main components:

- **HDFS**: A file system that splits data into blocks and stores them across computers.
- **YARN**: Manages resources in the cluster.
- **MapReduce**: Processes data in parallel across the cluster.



Cloud Computing (BCS071/BCS713)



❖ What is Hadoop Distributed File System (HDFS)?

HDFS is the storage system used in Hadoop. It is designed to store very large files by breaking them into smaller pieces (blocks) and distributing them across multiple computers in a cluster. HDFS is fault-tolerant, meaning it keeps your data safe even if some computers fail.

How Does HDFS Work?

1. Splitting the File:

Large files are broken into **blocks** (default size is 128 MB or 256 MB).

For example, if you have a 1GB file and the block size is 128MB, it will be split into 8 blocks.

2. Storing Blocks on Nodes:

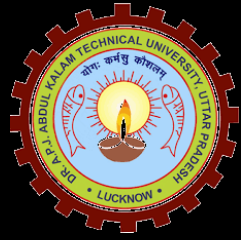
The blocks are stored across the computers (called **DataNodes**) in the cluster.

Each block is **replicated** (copied) to multiple nodes (default: 3 copies) for fault tolerance.

3. Components of HDFS:

NameNode: Acts like a "manager" or "directory" that keeps track of where each block is stored. It doesn't store data itself, just the metadata (information about blocks).

DataNodes: The "workers" that store the actual blocks of data.



Cloud Computing (BCS071/BCS713)



4. Writing Data:

When you upload a file, HDFS splits it into blocks.

These blocks are sent to DataNodes and stored with replication.

The NameNode updates its metadata to record where the blocks are stored.

5. Reading Data:

When you request a file, the NameNode provides the location of the blocks.

The system retrieves these blocks from the DataNodes and combines them to give you the complete file.

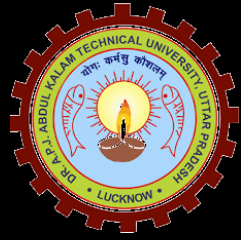
6. Fault Tolerance:

If a DataNode fails, the other copies (replicas) of the block are used.

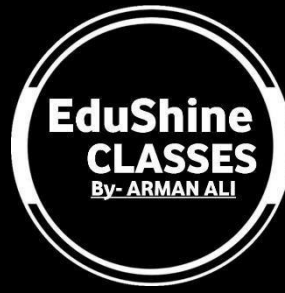
The system automatically creates a new copy of the lost block on another DataNode.

✓ Key Features of HDFS

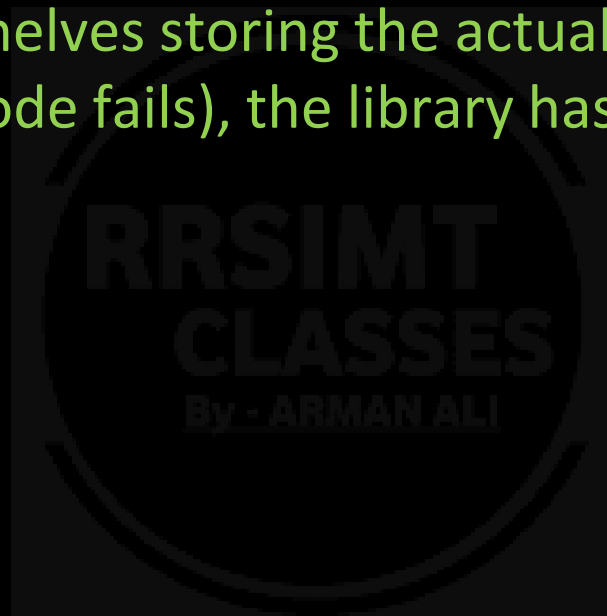
- **Scalable:** Easily add more computers to store more data.
- **Fault-Tolerant:** Data is safe even if some computers fail, thanks to replication.
- **Distributed Storage:** Files are split and spread across multiple computers for efficiency.



Cloud Computing (BCS071/BCS713)



- **High Throughput:** Optimized for large data reads and writes, not small files. In simple terms, HDFS works like a library where:
The **NameNode** is the librarian who knows where every book (block) is stored.
The **DataNodes** are the bookshelves storing the actual books (data blocks).
If a book is damaged (a DataNode fails), the library has backup copies to replace it.





Cloud Computing (BCS071/BCS713)



❖ Components of Hadoop :

Hadoop has four main components that work together to store and process big data. Let's think of Hadoop as a "data factory" and understand each part like sections of the factory.

1. HDFS (Storage)

- Stores big files by breaking them into small parts and spreading them across many computers.
- **NameNode**: Tracks where files are stored.
- **DataNodes**: Stores the actual data.

2. YARN (Manager)

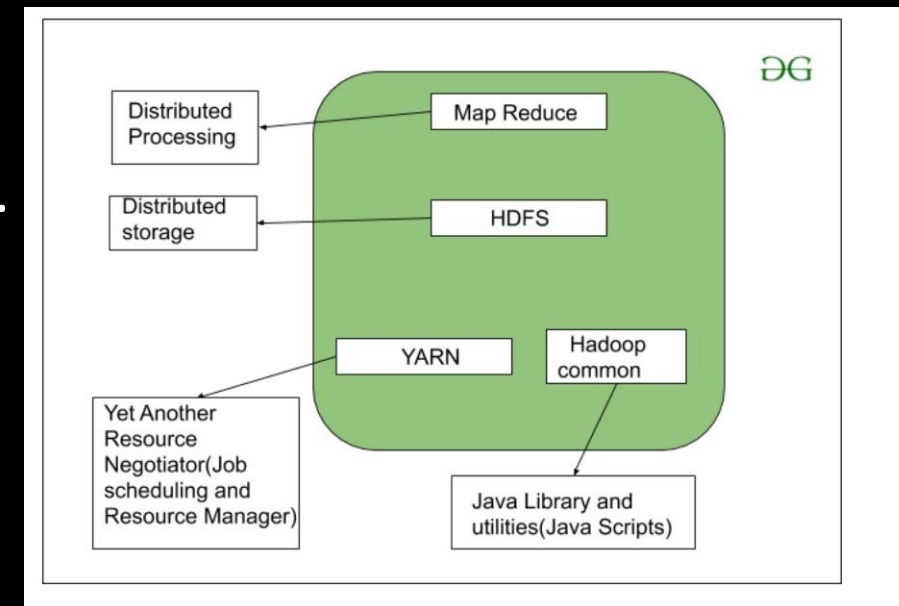
Manages resources like memory and CPU across the cluster. Decides which computer does what job.

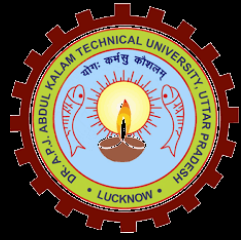
3. MapReduce (Processing)

Processes big data in two steps:

Map: Breaks work into smaller tasks.

Reduce: Combines results for the final output.





Cloud Computing (BCS071/BCS713)



4. Hadoop Common (Toolbox)

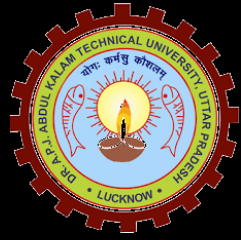
Provides tools and libraries to help all the components work together.

Simple Example

Think of it like a pizza shop:

- **HDFS:** Stores the ingredients.
- **YARN:** Decides who prepares, bakes, and delivers the pizza.
- **MapReduce:** Prepares and combines ingredients to make the pizza.
- **Hadoop Common:** Tools like ovens and knives to get the job done.





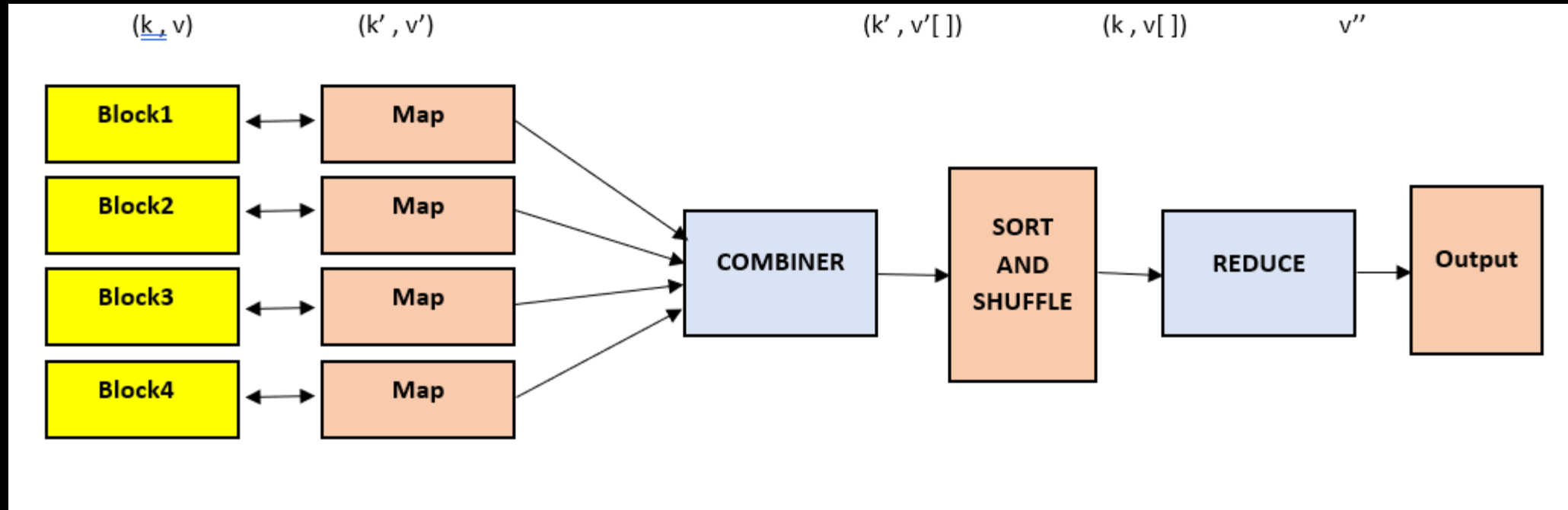
Cloud Computing (BCS071/BCS713)

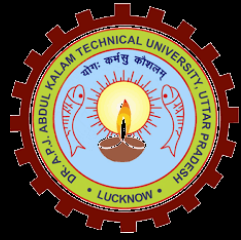


❖ What is MapReduce?

MapReduce is a programming model in Hadoop that processes large amounts of data by dividing it into small tasks that run on many computers at the same time. It has two main steps:

- **Map:** Breaks data into smaller parts and processes it.
- **Reduce:** Combines the results from the Map step to produce the final output.





Cloud Computing (BCS071/BCS713)



✓ Phases (Stages) of MapReduce

1. Map Phase (Splitting and Mapping)

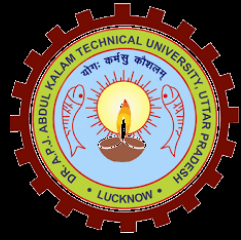
- **What Happens:**

- Input data is split into small chunks (blocks).
- Each chunk is processed by a **Mapper** to produce key-value pairs.
- **Example:** If analyzing a document, Mapper counts how many times each word appears and outputs results like:
 - word1 -> 1
 - word2 -> 1

2. Shuffle and Sort Phase (Grouping)

- **What Happens:**

- The key-value pairs from the Map phase are grouped by their key (like sorting all values for the same word together).
- **Example:**
 - All results for word1 are grouped: word1 -> [1, 1, 1]

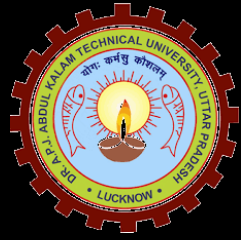


Cloud Computing (BCS071/BCS713)



3. Reduce Phase (Combining)

- **What Happens:**
 - The grouped data is processed by the **Reducer**, which combines the values for each key.
- **Example:**
 - For word1 -> [1, 1, 1], the Reducer adds the values:
 - word1 -> 3



Cloud Computing (BCS071/BCS713)



❖ How MapReduce Works :

1. **Input:** Splits big data into smaller parts (blocks).
2. **Map:** Processes each part to create **key-value pairs** (e.g., ("word", 1)).
3. **Shuffle & Sort:** Groups data by key (e.g., ("word", [1, 1, 1])).
4. **Reduce:** Combines values for each key to produce the final result (e.g., ("word", 3)).
5. **Output:** Saves the combined result.

Example: Counting words in a file: Input:

"Hadoop is cool. Hadoop is fast."

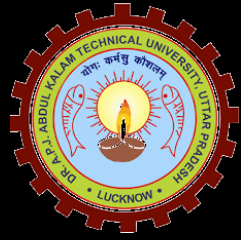
Output:

Hadoop -> 2

is -> 2

cool -> 1

fast -> 1



Cloud Computing (BCS071/BCS713)



❖ Phases of Developing a MapReduce Application :

Here are the main steps involved in creating a MapReduce application:

1. Understand the Problem

Identify what you want to achieve with your data.

Example: Counting how many times each word appears in a document.

2. Set Up Input Data

Prepare the data you want to process (e.g., a text file).

Example: A file with sentences like "Hadoop is amazing. Hadoop is fast."

3. Write the Mapper Function

- Break the input data into small pieces and process each piece.
- Mapper outputs **key-value pairs**.
- Example:
 - Input: "Hadoop is amazing"
 - Mapper output:
 - ("Hadoop", 1), ("is", 1), ("amazing", 1)



Cloud Computing (BCS071/BCS713)



4. Write the Reducer Function

- Combine the key-value pairs from the Mapper.
- Reducer processes grouped data and produces the final output.
- Example:
 - Input: ("Hadoop", [1, 1])
 - Reducer output:
 - ("Hadoop", 2)

5. Configure the Job

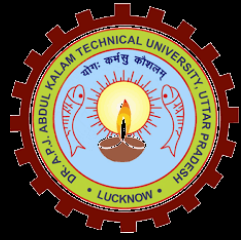
Specify the **input file**, **output location**, **Mapper**, and **Reducer** in your code.

Set the number of Mappers and Reducers if needed.

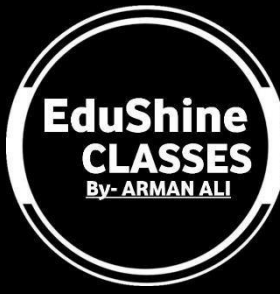
6. Run the Application

Submit the job to a Hadoop cluster.

Hadoop handles splitting data, running Mappers and Reducers, and saving the output.



Cloud Computing (BCS071/BCS713)



7. Verify Output

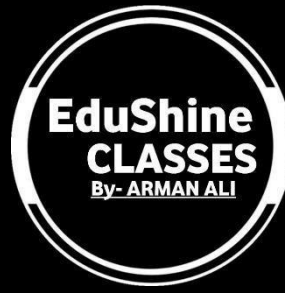
- Check the output file to ensure the results are correct.
- Example:
 - Output:
 - Hadoop -> 2
 - is -> 2
 - amazing -> 1

Summary

1. **Understand** the problem.
2. **Prepare** the input data.
3. Write the **Mapper** and **Reducer** functions.
4. **Configure** and run the job.
5. **Verify** the output.



Cloud Computing (BCS071/BCS713)



❖ What is VirtualBox?

VirtualBox is a free software (by Oracle) that allows you to run **multiple operating systems (OS)** on a single computer.

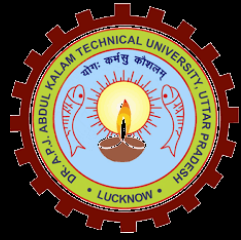
Example: On your Windows laptop, you can install **Linux, Ubuntu, Kali, or even another Windows** inside VirtualBox.

This is possible because VirtualBox creates a **Virtual Machine (VM)** → a computer inside your computer.

◆ Role in Virtualization

- **Virtualization** = Technology that allows one physical computer to run many virtual computers.
- VirtualBox is a **Type-2 Hypervisor** (runs on top of an OS like Windows/Mac/Linux).
- It divides your computer's CPU, RAM, storage, and network so that each VM feels like it has its own computer.

☞ In short: VirtualBox is a tool that makes virtualization easy for desktops and developers.



Cloud Computing (BCS071/BCS713)

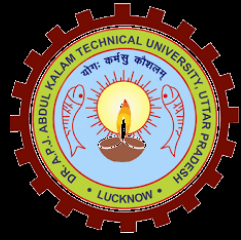


◆ How it Works

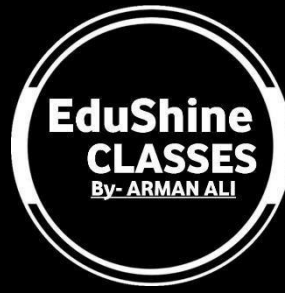
1. **Install VirtualBox** on your main computer (Host OS).
2. **Create a Virtual Machine (VM)** → choose how much RAM, CPU, and storage to give.
3. **Install Guest OS** inside the VM (e.g., Linux, Ubuntu).
4. Now, both **Host OS (your original OS)** and **Guest OS (inside VM)** can run at the same time.
5. You can pause, start, stop, or delete VMs whenever you want.

✓ Installation of Virtual Box

- Download it from its official website at Oracle VM Virtual Box site.
- Select your platform package.
- Click on the installer.exe.
- Follow all the Virtual Box installer.exe instruction.
- Start your Virtual Box and create your OS file.



Cloud Computing (BCS071/BCS713)

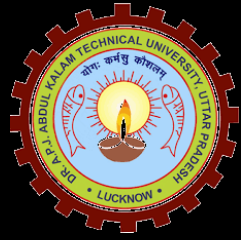


◆ Advantages of VirtualBox

- ✓ **Free & Open Source** → No cost, widely available.
- ✓ **Cross-Platform** → Works on Windows, Mac, Linux.
- ✓ **Multiple OS** → Run many operating systems at once.
- ✓ **Safe Testing** → You can test new software, apps, or even viruses inside VM without harming host computer.
- ✓ **Snapshots** → Take “backup states” of VM and restore later.
- ✓ **Networking Features** → You can simulate network setups (LAN, server-client).
- ✓ **Portable** → You can copy VM files to another system and run them.

◆ Disadvantages of VirtualBox

- ✗ **Performance Overhead** → Slower than real hardware (since resources are shared).
- ✗ **Needs Powerful Hardware** → If your PC has low RAM/CPU, multiple VMs will lag.
- ✗ **Limited Graphics Support** → Not good for heavy gaming/3D rendering.
- ✗ **Not Enterprise Grade** → Better for students/developers; companies prefer VMware/Hyper-V for large scale.



Cloud Computing (BCS071/BCS713)



✗ **Battery & Heat** → On laptops, running VMs drains battery faster and heats the system.

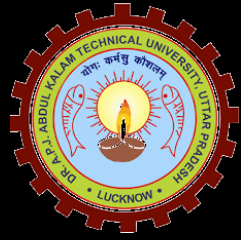
◆ Simple Example

Imagine your **computer** is like a house 🏠.

Normally, only **one family (one OS)** can live inside.

VirtualBox divides the house into **many flats (VMs)**.

Now multiple families (Windows, Linux, Mac) can live together inside one house.



Cloud Computing (BCS071/BCS713)



◆ What is Google Cloud Platform (GCP)?

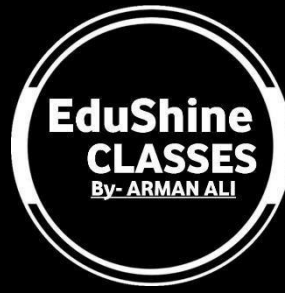
- **Google Cloud Platform (GCP)** is Google's cloud computing service.
- It provides everything you need to **build, store, and run applications on the cloud** instead of your local computer.
- GCP is the same cloud that runs **Google Search, YouTube, and Gmail**.

☞ Think of it like a **giant online toolbox** where you get:

- Virtual machines (servers)
- Databases
- Storage (like Google Drive but for apps/data)
- AI/ML services
- Networking & security tools



Cloud Computing (BCS071/BCS713)



◆ Main Services of GCP :

- i. **Compute** → Run apps (e.g., Compute Engine, App Engine, Kubernetes Engine).
- ii. **Storage & Databases** → Store files and data (e.g., Cloud Storage, Firestore, BigQuery).
- iii. **Networking** → Connect apps securely (e.g., Cloud VPN, Load Balancer).
- iv. **AI & Machine Learning** → Ready-made AI (e.g., Vision API, Translation API).
- v. **Tools for Developers** → Cloud Functions, APIs, CI/CD pipelines.

◆ How to Interact with Google Cloud Services?

You can use GCP in 3 simple ways:

1. Google Cloud Console (Web UI)

A website → console.cloud.google.com

You log in, click buttons, create servers, databases, buckets, etc.

Easiest way for beginners.

2. Cloud SDK (gcloud CLI)

- A command-line tool → gcloud command.

Example:



Cloud Computing (BCS071/BCS713)



gcloud compute instances create my-vm

- Used by developers/admins for automation.

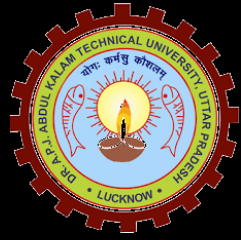
3. Client Libraries / APIs

- You can interact directly from your **code** (Python, Java, Node.js, etc.).
- Example (Python code to store file in Cloud Storage):

```
from google.cloud import storage

client = storage.Client()
bucket = client.bucket("my_bucket")
blob = bucket.blob("hello.txt")
blob.upload_from_filename("hello.txt")
```

- Best for apps that need cloud features.



Cloud Computing (BCS071/BCS713)



◆ Advantages of GCP

- ✓ Same infrastructure as **Google Search & YouTube** (super fast & reliable).
- ✓ **Pay as you go** → Only pay for what you use.
- ✓ **Global network** → Data centers all over the world.
- ✓ **AI & Big Data power** → Google's strength.
- ✓ **Secure & Scalable.**
- ✓ **In short:**

GCP = Google's toolbox for cloud computing.

You can interact with it via:

Web Console (UI),

gcloud CLI (commands),

APIs/Libraries (in your code).



Cloud Computing (BCS071/BCS713)



◆ What is Google App Engine?

Google App Engine (GAE) is a Platform as a Service (PaaS) by Google.

It allows you to **build and run web apps** without worrying about servers, storage, or networking.

You just **write code** → **upload** → **Google takes care of everything** (scaling, security, updates).

👉 **Example:** If you build a web app for online notes, normally you need a server, database, security, etc.

With GAE → you only write the code for your app, and Google manages the backend.

◆ Features of Google App Engine

1. **Automatic Scaling** → If 10 users or 1 lakh users come, GAE automatically adds or reduces resources.
2. **Multiple Languages** → Supports Python, Java, PHP, Go, Node.js, Ruby, etc.
3. **Built-in Services** → Datastore (database), Memcache (caching), Task Queues, User Authentication.
4. **Pay-as-you-go** → You pay only for the resources you use.



Cloud Computing (BCS071/BCS713)



- 5. Versioning** → You can run multiple versions of the same app (easy updates/rollbacks).
- 6. Monitoring & Logging** → Built-in tools to track errors, performance, and usage.
- 7. Security** → Google manages patches, firewalls, and secure connections.
- 8. Integration with Google Cloud** → Can easily use Google's AI, ML, Storage, BigQuery, etc.

◆ Advantages of Google App Engine

- ✓ **No Server Management** → Developers focus only on coding, not on maintaining servers.
- ✓ **Highly Scalable** → Handles sudden traffic spikes automatically.
- ✓ **Free Tier Available** → Good for students, startups to test apps.
- ✓ **Fast Deployment** → Just one command and your app is live.
- ✓ **Reliable** → Runs on Google's global infrastructure (same as Gmail, YouTube).
- ✓ **Secure** → Automatic updates & security handled by Google.
- ✓ **Cost Effective** → Pay only for what you use, no upfront cost.



Cloud Computing (BCS071/BCS713)



◆ Limitations (for balance)

- ✗ Limited control on hardware (Google manages it, not you).
- ✗ Vendor lock-in → App is tied to Google's ecosystem.
- ✗ Pricing may increase for very large apps.

✓ In short:

Google App Engine = A platform where you only write code and Google takes care of running, scaling, and securing your app.

It is **easy, scalable, secure, and cost-effective**, especially good for startups and students.



- Receives requests from users around the world.
- Decides where to send each request (which region, which app version).

2. App Engine Frontend (Service & Versions)



Cloud Computing (BCS071/BCS713)



- The frontend is where your code runs (web handlers, APIs).
- You can deploy different versions for testing and switch traffic easily.

3. Instances

- Small containers/servers that actually run your code.
- App Engine creates more when traffic goes up and removes them when traffic drops.

4. Autoscaling

- Automatically adds instances when your app gets busy and reduces them when it's quiet. Saves cost and handles spikes.

5. Cloud Storage

- Used to store files like images, videos, PDF. Good for static content.
- Can be served through CDN for fast delivery.

6. Cloud SQL

- Managed relational database (MySQL/Postgres). Use it when you need structured tables and transactions.



Cloud Computing (BCS071/BCS713)



7. Cloud Datastore / Firestore

- NoSQL database for flexible, scalable storage (good for JSON-like objects, user profiles, etc).

8. Memcache

- Very fast memory store (cache) to keep frequently-read data so your app doesn't always hit the database.

9. Task Queues (Cloud Tasks)

- For background jobs (send email, resize image). These run outside the user request so response is fast.

10. Cron Jobs

- Scheduled jobs (e.g., every night run a cleanup task).

11. Monitoring & Logging

- Tracks errors, performance and usage (Cloud Logging / Cloud Monitoring).

12. IAM & Security / VPC

- Controls who can access what, and allows secure network connections if your app



Cloud Computing (BCS071/BCS713)



- needs to reach private resources.

❖ How a typical request flows (example: user uploads an image)

1. User clicks “Upload” in the browser → request goes to Load Balancer.
2. Load Balancer forwards to the right App Engine service/version.
3. An Instance receives the HTTP request and the app code runs.
4. App code stores the image in Cloud Storage (fast, cheap for files).
5. App writes metadata (URL, user id) to Cloud SQL or Datastore.
6. App pushes a job into Task Queue to create thumbnails (background work).
7. The task worker instance picks the job, resizes image, saves thumbnails to Cloud Storage, updates DB, maybe updates Memcache.
8. The user gets a quick response that upload succeeded — heavy work runs in background.
9. If many users upload at once, App Engine spins up more instances automatically.



Cloud Computing (BCS071/BCS713)



❖ Programming Environment in Google App Engine (GAE)

It means → what tools, languages, and environment Google gives us to **develop and run apps**.

1. Supported Languages

Google App Engine supports many programming languages.

- Java
- Python
- PHP
- Go

(Through custom runtimes, almost any language can run, like Node.js, Ruby, etc.)

2. Development Tools

- You write code on your local computer using any **IDE** (like PyCharm, Eclipse, VS Code).
- Google gives an **App Engine SDK (Software Development Kit)** for local testing.
- You can test your app on your system before sending it to Google Cloud.



Cloud Computing (BCS071/BCS713)



3. Deployment

- After testing, you deploy your app using **Google Cloud Console** (web dashboard) or **gcloud command line tool**.
- Once deployed, your app runs on Google's infrastructure automatically.

4. App Engine Services

The environment provides:

- **App Engine Frontend** → handles user requests.
- **Datastores (Cloud SQL, Datastore, Firestore)** → for storing data.
- **Memcache** → for caching data.
- **Task Queues / Cron jobs** → for background tasks.
- **Scaling** → auto increase or decrease resources.



Cloud Computing (BCS071/BCS713)



5. Programming Model

- App is divided into services (modules).
- Each service has handlers (functions) for requests.
- You use Google's APIs (for datastore, email, authentication, etc.) directly in your code.

✓ In Short :

The programming environment of Google App Engine =

☞ A set of languages, tools, APIs, and services given by Google so that you can write code (like in Python/Java), test it locally, and then run it on Google's servers without worrying about hardware, scaling, or servers.





Cloud Computing (BCS071/BCS713)



❖ What is OpenStack?

☞ OpenStack is an open-source cloud computing platform.

- It helps organizations build their own private cloud or public cloud (like AWS, Google Cloud but self-hosted).
- With OpenStack, you can manage compute (VMs), storage, networking, and other cloud resources through a dashboard or APIs.

Think of it as a toolbox for making your own cloud.

❖ Features of OpenStack

1. **Open-source** – Free to use, anyone can modify.
2. **Multi-tenant** – Multiple users can share the same cloud safely.
3. **Scalable** – Can grow from a few machines to thousands.
4. **Dashboard (Horizon)** – Easy web interface for management.
5. **API support** – Developers can control resources programmatically.
6. **Pluggable architecture** – You can add extra modules if needed.
7. **Supports virtualization & bare-metal** – Works with hypervisors like KVM, VMware, etc.



Cloud Computing (BCS071/BCS713)



8. Community support – Large active community for updates.

❖ Main Components of OpenStack

OpenStack has many components (services). Each does one job in the cloud.

1. Nova (Compute Service)

- Manages virtual machines (VMs).
- Works with hypervisors (like KVM, Xen, VMware).

Example: If you want to create 10 new servers, Nova handles it.

2. Neutron (Networking Service)

- Manages networking in cloud.
- Creates virtual networks, subnets, IP addresses, firewalls, load balancers.

Example: Connecting VMs securely, giving each one its own IP.

3. Cinder (Block Storage)

- Provides block storage (like hard disks).
- You can attach these volumes to VMs.

Example: Extra storage for a database server.



Cloud Computing (BCS071/BCS713)



4. Swift (Object Storage)

- Stores large amounts of unstructured data (files, images, backups, videos).

Example: Like Google Drive or AWS S3.\

5. Glance (Image Service)

- Stores and retrieves VM images (OS snapshots).

Example: You can boot a VM with Ubuntu or CentOS image from Glance.

6. Keystone (Identity Service)

- Handles authentication and authorization.

Example: Username & password login, assigns

7. Horizon (Dashboard)

- Web-based GUI to manage OpenStack resources.

Example: Create VM, attach volume, create networks using mouse clicks.

8. Heat (Orchestration)

- Automates the creation of cloud resources using templates.

Example: Deploying a whole app stack (VM + DB + Network) in one go.



Cloud Computing (BCS071/BCS713)



9. Ceilometer (Telemetry/Monitoring)

Collects metering and monitoring data.

Example: Tracks CPU usage, billing, resource usage.

10. Trove (Database as a Service)

Manages relational and NoSQL databases.

Example: Quickly launch a MySQL or MongoDB database.

❖ Easy Flow (How OpenStack Works)

- User logs in → via Keystone (Identity).
- User requests a VM → Nova creates it.
- VM needs a network → Neutron provides it.
- VM needs storage → Cinder (block) or Swift (object).
- VM image comes from Glance.
- User manages everything via Horizon Dashboard or API.



Cloud Computing (BCS071/BCS713)



✓ Summary :

OpenStack = A platform to create your own cloud.

It has different components (Nova, Neutron, Swift, etc.), each managing one part of the cloud.

You can launch VMs, connect networks, store data, and manage users easily.



Cloud Computing (BCS071/BCS713)



☁️ What is Cloud Federation?

☞ Cloud Federation means connecting multiple cloud providers (different clouds) together so that users can access them as if they were one single cloud.

Ek hi cloud kabhi kabhi enough nahi hota (storage, compute, cost, location problems).

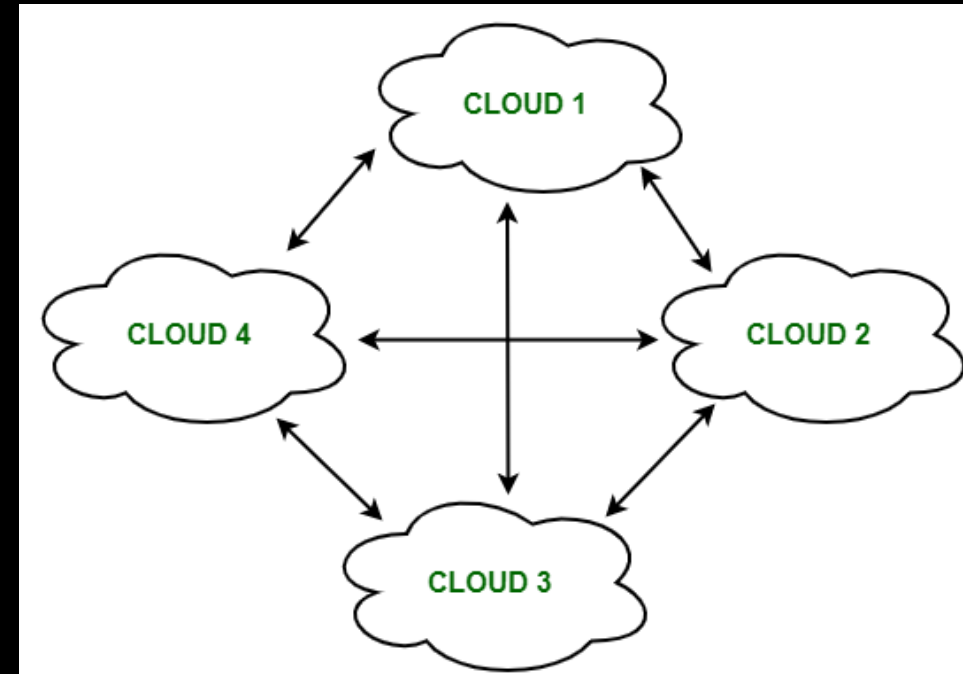
Federation allows different clouds (Google Cloud, AWS, Microsoft Azure, or private clouds) to work together.

🌐 Example:

Imagine you are a student:

- College cloud gives you 50GB storage.
- University cloud gives you 2 virtual machines.
- Research center cloud gives you AI GPU service.

☞ With Federation, you can use all three clouds together through a single login and broker, without separately managing each one.





Cloud Computing (BCS071/BCS713)



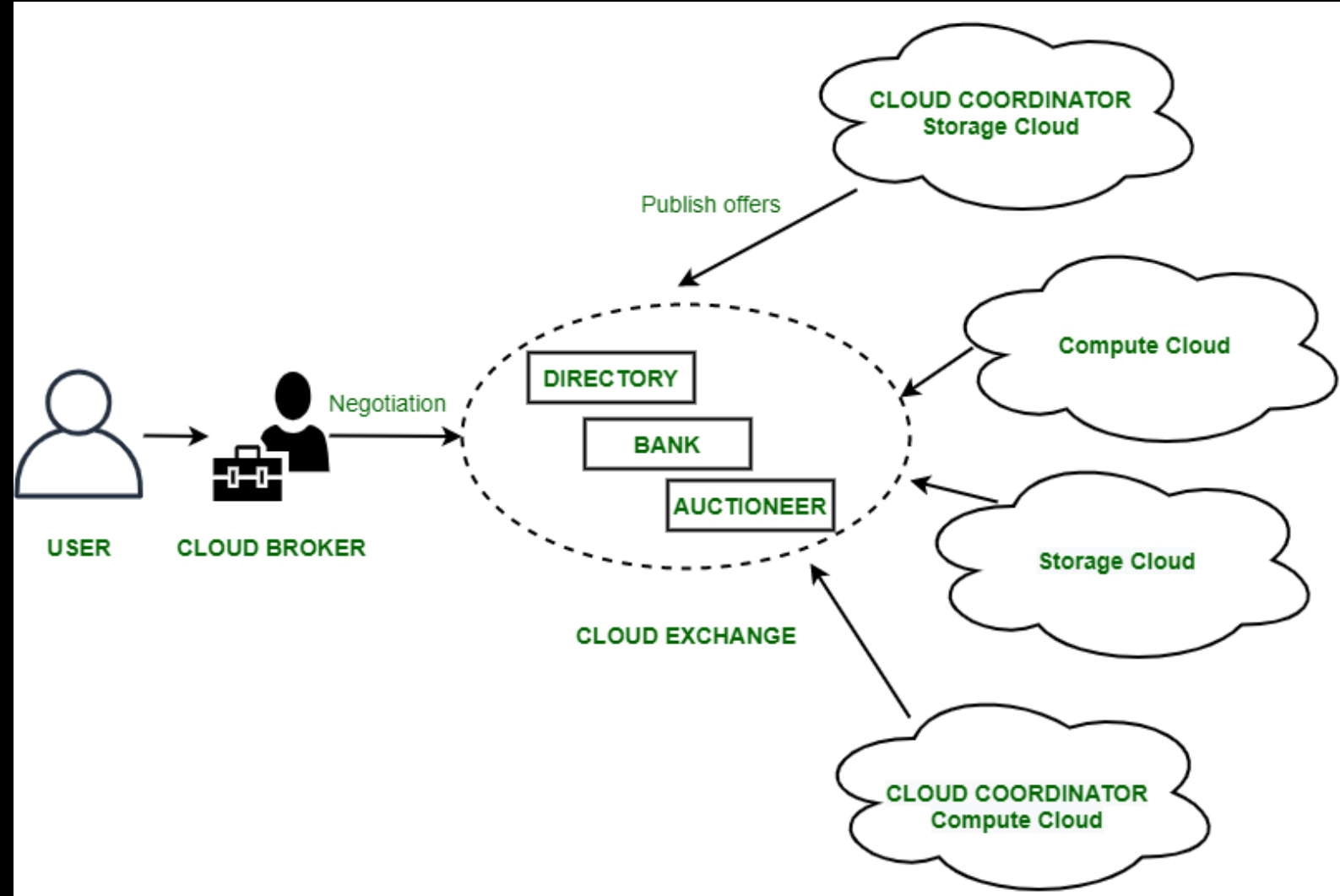
Architecture of Federated Cloud :

1. User

- Normal person/company who needs cloud service.

2. Cloud Broker

- Acts like a middleman.
- User doesn't directly talk to different clouds (AWS, Azure). Instead, the broker helps in negotiation, pricing, and selecting best service.





Cloud Computing (BCS071/BCS713)



3. Cloud Exchange

This is like a marketplace for clouds.

It has:

- Directory → List of available cloud services.
- Bank → Handles payments & credits.
- Auctioneer → Helps choose the best provider if many options exist (like bidding).

4. Cloud Coordinator (for each cloud)

- Each cloud (Compute Cloud, Storage Cloud, etc.) has its own coordinator.
- Coordinator publishes its offers (price, resources) to the Cloud Exchange.

5. Compute/Storage Clouds

Actual cloud providers (AWS, GCP, Azure, or private data centers).



Cloud Computing (BCS071/BCS713)



🔄 How it works :

- User → asks broker → “I need 5 servers and 1TB storage”.
- Broker → sends request to Cloud Exchange.
- Cloud Exchange → checks directory & auctioneer → finds best cloud(s).
- Bank in Cloud Exchange → handles payments.
- Cloud Coordinator → responds with offers (price, capacity).
- User → gets resources from multiple clouds but feels like **one single cloud**.

✓ Advantages of Cloud Federation

- ✓ Access to **multiple clouds** from one place.
- ✓ Better **cost optimization** (choose cheapest provider).
- ✓ **Load balancing** across providers.
- ✓ Avoids **vendor lock-in** (not stuck with only AWS/only GCP).
- ✓ Users get **more resources** when one cloud alone is not enough.
- ✓ 🖱 **So, Cloud Federation = Marriage of multiple clouds working as one big cloud.**



Cloud Computing (BCS071/BCS713)



🌐 Four Levels of Cloud Federation

Cloud Federation means connecting multiple clouds (AWS, Google Cloud, Azure, or private clouds) so they can work together and share resources.

The federation happens in 4 levels – from very loose connection to full integration.

1. Permissive Federation

This is the basic / lowest level.

One cloud trusts the user of another cloud without much checking.

Example: You log in to Cloud A with credentials from Cloud B, and Cloud A just accepts it.

Risk: It is less secure because there is no strong verification.

☞ Think of it like: *“Anyone from your friend’s group can enter my home without showing ID.”*

2. Verified Federation

Here, the cloud verifies the identity before giving access.

It checks if the user really belongs to the other cloud.

Example: Cloud A will confirm with Cloud B’s database that you are a valid user.

☞ Like: *“If your friend wants to enter my home, I will call you first to confirm.”*



Cloud Computing (BCS071/BCS713)



3. Encrypted Federation

- In this level, secure communication is used between the two clouds.
- The identity information is exchanged in encrypted (secret) form so no one can steal it.

Example: When Cloud B sends your login info to Cloud A, it is encrypted.

☞ Like: *“Your friend shows me an ID card, but it is locked inside a sealed envelope. Only I can open it.”*

4. Trusted Federation

- This is the highest and strongest level.
- The clouds have a deep trust relationship.
- Both sides are confident that the other cloud's users are authentic and secure.

Example: Cloud A and Cloud B work like close partners, maybe even sharing security policies.

☞ Like: *“I fully trust your family, so anyone you send is always welcome in my home.”*



Cloud Computing (BCS071/BCS713)



🚀 Future of Federation – Key Directions

1. Multi-Cloud & Hybrid Cloud Federation

- Companies will not depend on just one cloud (like AWS or GCP).
- They will use many clouds together (multi-cloud) or mix private + public cloud (hybrid).
- Federation will make it possible for users and workloads to move smoothly between these clouds.

🔑 **Example:** A bank runs sensitive apps in its private cloud but uses AWS for analytics. Federation allows employees to log in once and use both.

2. Stronger Security & Zero-Trust Federation

- Current federation often assumes some level of trust.
- In the future, Zero-Trust models will be used → meaning every request will be verified, no matter where it comes from.
- Features like multi-factor authentication (MFA), biometric login, and AI-based anomaly detection will be part of federation.



Cloud Computing (BCS071/BCS713)



3. Global Identity Standards

- Right now, there are many standards: SAML, OAuth, OpenID Connect, etc.
- In the future, there will be unified global standards to make federation smoother across industries.

👉 **Example:** One universal digital ID that works across schools, hospitals, banks, and government portals.

4. Blockchain-Based Federation

- Blockchain can provide decentralized identity (DID).
- No single company (Google, Facebook, etc.) controls your login.
- Users own their identity and can use it across all federated systems.

👉 **Example:** Instead of using Gmail login, you use your blockchain wallet ID to access different apps.



Cloud Computing (BCS071/BCS713)



5. AI-Powered Federation

- AI will analyze login patterns and detect suspicious access.
- Federation systems will become intelligent → automatically adapting trust levels depending on risk.

☞ **Example:** If you log in from your usual laptop, access is smooth. If someone tries from a new country, extra verification is required.

6. Cross-Industry Federation

- Federation will not be limited to tech companies.
- Governments, healthcare, education, and finance will use federated identity systems.

☞ **Example:** Using your Aadhaar/Passport ID to log in to airline sites, bank portals, and university portals without separate accounts.

7. Seamless User Experience (Single Global Identity)

Goal = One secure login → access everything.

No more remembering 20 different usernames and passwords.

Federation will give a smooth, passwordless future (using biometrics, passkeys, or hardware keys).



Cloud Computing (BCS071/BCS713)



Thank You....



Download Notes : <https://rzp.io/rzp/tGaErCk>